Lean from First Principles

Dependent Types, the Calculus of Constructions, and the Architecture of Leant

A beginner's path from "a term has a type" to verified type-directed synthesis

Repository snapshot

Leant	`49c8f9f100a6` (main, inspected 16 August 2026)
Djex used by Leant	`7d65eba6f753` (the `lib/Djex` submodule)
Djex standalone	`0cf82f2aa22d` (main, inspected for the current shared architecture)
Lean 4	`79c4cd09fdab` (master)

Prepared 16 August 2026

Preface

This guide has a deliberately narrow destination and a deliberately broad runway. The destination is practical: after reading it, an absolute beginner should be able to open the Leant and Djex repositories, understand what problem `:synth` is trying to solve, follow the major data transformations, and judge correctly what a successful candidate or a negative result means. The runway is broad because those facts make little sense until Lean's central idea is internalized: *types may depend on values, propositions are types, and proofs are ordinary typed terms checked by a small kernel.*

The guide is not a general catalogue of Lean tactics, a mathlib tutorial, or a replacement for the official Lean books. It concentrates on the pieces that determine Leant's design:

- the Curry–Howard correspondence and type inhabitation;
- dependent function types, dependent pairs, universes, inductive families, equality, and transport;
- the Calculus of Constructions and the inductive extension used by Lean;
- the separation between surface syntax, elaboration, metaprogramming, and kernel expressions;
- the gap between Lean's rich dependent type theory and Djex's deliberately smaller Haskell-like synthesis language;
- the translation, search, rendering, verification, and evidence boundaries in Leant.

The existing LaTeX documents in the Leant repository use two useful idioms. The user manual in `docs/Leant_Overview/Leant_Overview.tex` uses LuaLaTeX and Unicode-safe transcript boxes; the longer architecture reports use chapters, pinned revisions, source trails, diagrams, and explicit trust-boundary boxes. This document combines those idioms. All Lean snippets are presented as Unicode-capable verbatim material, and source-specific statements are tied to the pinned repository state above.

The one-sentence thesis

Lean provides the rich language and the final type checker; Leant projects selected Lean goals into a smaller synthesis problem for Djex, turns Djex's answers back into Lean syntax, and asks Lean to accept or reject every displayed candidate.

How to read this guide

For the shortest route to the Leant implementation, read Chapters 1–4, then Chapters 6–11, and finally Parts III and IV. The formal-rule boxes can be skipped on a first pass. They are there to remove ambiguity when one later reads kernel code or reasons about completeness.

A few notational conventions recur:

- $\Gamma \vdash t : A$ means that in context Γ, term t has type A.
- $A \to B$ is a non-dependent function type.
- $\Pi(x : A), B(x)$ is a dependent function type; Lean writes it as `(x : A) → B x` or `∀ x : A, B x`.
- $t \equiv u$ means *definitionally equal*: Lean can see the equality by computation and built-in conversion rules, without an explicit equality proof.
- "kernel" always means the trusted checker for fully elaborated terms, not the parser, elaborator, tactic engine, or synthesis engine.
- "elaborator" means the untrusted front end that turns Lean source text into a fully explicit kernel term: it resolves notation, inserts the implicit arguments and universe levels the user left out, runs type-class search and tactics, and only then hands a complete term to the kernel.

- Where a Lean box shows output in a comment (`-- ...`), it is the text a current Lean 4 toolchain (4.33.0) printed for that command; snippets shown as *rejected* quote Lean's actual error message.

Contents

Part I

The Core Idea

A First Encounter with Lean

1.1 Lean is both a language and a checker

Lean is a functional programming language, a language for stating mathematical propositions, and an interactive theorem prover. These are not three unrelated subsystems bolted together. They share one typed term language. A natural number expression is a term. A function is a term. A proposition is a type. A proof is a term whose type is that proposition.

Start with ordinary programming:

Lean: data and functions

```
def twice (f : Nat → Nat) (x : Nat) : Nat :=
  f (f x)

#check twice
#eval twice (fun n => n + 1) 40
```

The declaration says that `twice` accepts a function from natural numbers to natural numbers, accepts a natural number, and returns a natural number. The body is checked against that declared type. Lean answers `#check` with the signature `twice (f : Nat → Nat) (x : Nat) : Nat`, and `#eval` prints 42.

Now look at a theorem written in the same language:

Lean: a proof as a function

```
theorem andSwap (P Q : Prop) : P ∧ Q → Q ∧ P :=
  fun h => ⟨h.right, h.left⟩
```

The body is again a term checked against a type. This time the type is a proposition. The input `h` contains proofs of both P and Q; the output pair puts them in the opposite order. Nothing in the kernel says "switch into proof mode now." It checks one typed term.

Read the punctuation literally

In `P ∧ Q → Q ∧ P`, the connective `∧` binds more tightly than `→`, so the type parses as `(P ∧ Q) → (Q ∧ P)`: "given a proof of $P \wedge Q$, produce a proof of $Q \wedge P$." (Arrows themselves associate to the right, so `P → Q → R` means `P → (Q → R)`; that rule is not what is at work here.) The term `fun h => ...` is a lambda expression: a function accepting that proof.

1.2 Declarations: `def`, `theorem`, `example`, and `opaque`

The most common top-level forms differ mainly in how the result is named and how it may compute.

`def`
: Introduces a named definition with a body. Lean may unfold it during computation and definitional equality, subject to transparency policy in higher layers.

`theorem`
: Introduces a named proof. Its type must be a proposition (Lean rejects a `theorem` whose type is data-valued with "`type`

`of theorem ‘compose‘ is not a proposition`”), and its body is checked exactly like a definition body. Afterwards the proof is used through its type alone: Lean’s elaborator never unfolds a theorem body during reduction, and although the kernel’s definitional-equality check is permitted to unfold theorems as a last resort, proof irrelevance (Chapter 8) makes that practically unnecessary. The proof’s type is the public contract.

`example`
Checks a declaration without adding a durable user-facing name. It is ideal for experiments and, notably, for Leant’s candidate-verification command.

`opaque`
Introduces a checked constant whose implementation does not unfold during ordinary reduction. It is useful when only a signature should be visible.

`axiom`
Introduces a constant with a stated type and no body. The kernel checks later reasoning relative to that assumption; it does not prove the axiom.

Leant verifies a rendered candidate by asking Lean to elaborate a noncomputable example of exactly the requested type:

The shape of Leant’s verification command

```
set_option autoImplicit true in
noncomputable example : (GOAL) := (CANDIDATE)
```

That small command is architecturally decisive. The candidate is not accepted because Djex says it has the right type, nor because a string renderer believes it is valid. Lean elaborates it in the live environment and the kernel checks the resulting expression.

1.3 Terms, types, and types of types

Lean’s `#check` command asks the elaborator to report a term’s type:

Lean: inspecting types

```
#check 17
#check Nat
#check Nat → Nat
#check fun x : Nat => x + 1
#check Prop
#check Type
#check Type 1
```

Typical results are conceptually:

Term	Type
`17`	`Nat`
`Nat`	`Type`
`Nat → Nat`	`Type`
`fun x : Nat => x + 1`	`Nat → Nat`
`Prop`	`Type`
`Type`	`Type 1`
`Type 1`	`Type 2`

A type is itself a term, classified by a universe. This is the first hint of the Calculus of Constructions: the language does not stop at functions between data values. It can quantify over types, compute types, and form functions whose results are types.

1.4 What Leant adds

Lean already has holes, tactics, simplifiers, type-class search, decision procedures, and powerful proof automation. Leant adds a particular interactive workflow inspired by GHCi and Djinn:

A representative Leant interaction

```
:synth ((A → B → C) → (A → B) → A → C)
```

A natural answer is:

Synthesized Lean term

```
fun f g x => f x (g x)
```

The type determines a wiring diagram. Given $f : A \to B \to C$, $g : A \to B$, and $x : A$, the only direct way to build C is to give f an A and a B; x supplies the first, and $g\,x$ supplies the second.

Leant's job is not merely to print a plausible lambda term. It must:

1. ask Lean to elaborate the requested goal into a precise internal expression;
2. decide which structure can be translated into a synthesis fragment and which must remain opaque;
3. run one or both Djex engines under explicit search policies;
4. reconstruct Lean syntax, including binders, constructors, global names, and implicit arguments erased by the search model;
5. re-elaborate each candidate against the exact original goal;
6. report negative results only with the strength justified by both translation completeness and search completeness.

Why this matters for Leant

The beginner's temptation is to think of Leant as "Lean plus a clever autocomplete." A more accurate model is a verified compiler pipeline with a lossy middle language. Candidate soundness comes from checking the trip back into Lean; negative evidence depends on proving that the lossy projection was not lossy for this particular query.

Source trail

The current Leant overview and examples are in https://github.com/VladimirReshetnikov/Leant/blob/49c8f9f100a644b5a32b3c942bc6565a3a3ba6a0/README.md. The exact candidate command is generated by `src/Leant/Synth/Fragment.hs`; process management is in `src/Leant/Backend.hs`; orchestration and interactive state are in `src/Main.hs`.

Chapter checkpoint

You should now be able to read "a proof of P" as "a term of type P," and "synthesize a program of type T" as "search for an inhabitant of T." Lean owns the exact type theory and final check. Leant owns an interactive translation/search/reconstruction pipeline around that checker.

Reading Lean Syntax Without Fear

Lean's notation is dense because the same core constructs serve programming and proof. This chapter decodes the subset needed throughout the guide and in the Leant source.

2.1 Function types and function values

A function type is written with an arrow:

Function signatures

```
Nat → Nat
String → Nat → Bool
(A → B) → A → B
```

Arrows associate to the right:

$$A \to B \to C \quad \text{means} \quad A \to (B \to C).$$

Function application is written by juxtaposition and associates to the left:

$$f\ x\ y \quad \text{means} \quad (f\ x)\ y.$$

A lambda expression introduces arguments:

Equivalent lambda spellings

```
fun x : Nat => x + 1
fun (x : Nat) => x + 1
fun x y => x + y
```

Multiple arguments are syntactic convenience for nested one-argument functions. This is *currying*:

$$\lambda x.\lambda y.\, t$$

has type $A \to B \to C$ when $x : A$, $y : B$, and $t : C$.

2.2 Named parameters and binder styles

Lean has four binder styles that matter both to users and to Leant's renderer:

Surface form	Binder information	Normal use
`(x : A)`	explicit	caller supplies an ordinary argument
`{x : A}`	implicit	elaborator usually infers the argument
`{{x : A}}`	strict implicit	inferred only once a later explicit argument is present
`[x : C]`	instance implicit	type-class synthesis supplies evidence

For example:

Implicit and instance arguments

```
def ident {α : Sort u} (x : α) : α := x

class Named (α : Type u) where
  name : α → String

def display {α : Type u} [Named α] (x : α) : String :=
  Named.name x
```

The braces around α say that Lean should infer the type from `x`. The square brackets say that Lean should search the current local and global instance environment for a `Named α` value.

Several snippets in this guide use variables such as `α`, `A`, `B` without declaring them. That works because core Lean's option `autoImplicit` defaults to `true`: an unbound identifier in a declaration *header* is quietly turned into an implicit argument, at the most general sort Lean can assign. Two consequences, both confirmed with Lean 4.33.0:

What autoImplicit really adds

```
def append : List α → List α → List α
  | [], ys => ys
  | x :: xs, ys => x :: append xs ys

#check @append
-- @append : {α : Type u_1} → List α → List α → List α

def app2 (xs : List Nat) : List Nta := xs
-- error: Type mismatch
--   xs
-- has type
--   List Nat
-- but is expected to have type
--   List Nta
```

Auto-bound implicits: convenient, and a source of silent surprises

First, `α` became `{α : Type u_1}`, complete with a fresh universe variable; the sort is chosen from how the variable is used, so an unused `A` becomes `Sort u` rather than `Type` or `Prop`. That is exactly why `theorem compose (f : B → C) ...` without `(A B C : Prop)` fails with "`type of theorem 'compose' is not a proposition`" (Section 2.6). Second, the typo `Nta` is not reported as an unknown name; it becomes a new type variable, and the error surfaces one step later as a mismatch between `List Nat` and `List Nta`. Mathlib disables the option for this reason; Leant's verification command deliberately enables it (Chapter 1) so that goals written with free type variables elaborate.

Why binder visibility survives translation

Djinn's logical model does not natively distinguish every Lean binder visibility. Leant therefore records whether a quantified type binder was explicit, implicit, or instance-implicit, and its renderer may insert lambdas or _ placeholders that the search term itself never contained. A single semantic candidate can consequently produce several textual variants, only one of which elaborates.

2.3 Definitions by equations and pattern matching

Lean lets functions be written as pattern-matching equations:

Structural recursion on lists

```
def append : List α → List α → List α
  | [], ys => ys
  | x :: xs, ys => x :: append xs ys
```

This notation is elaborated into recursors and auxiliary definitions. The kernel does not trust the equation compiler; it checks the generated term.

Constructor patterns expose the data carried by an inductive value:

Pattern matching on a sum

```
def either (f : A → C) (g : B → C) : A ⊕ B → C
  | .inl a => f a
  | .inr b => g b
```

A `match` expression gives the same structure inline:

Inline match

```
fun s : A ⊕ B =>
  match s with
  | .inl a => Sum.inr a
  | .inr b => Sum.inl b
```

2.4 Products, structures, and constructor notation

The type `A × B` is an ordinary product. A value can be written `(a, b)` or, when the expected type identifies a one-constructor structure, with anonymous constructor brackets `⟨a, b⟩`. Fields can be projected by names or indices.

Products and projections

```
def swap (p : A × B) : B × A :=
  ⟨p.2, p.1⟩
```

The same anonymous constructor notation works for conjunction because `And` is a proposition-valued inductive type with one constructor:

The same shape in Prop

```
theorem andSwap' (P Q : Prop) : P ∧ Q → Q ∧ P :=
  fun h => ⟨h.2, h.1⟩
```

Leant exploits expected-type elaboration here. Its renderer can use universe-agnostic shapes such as `⟨a, b⟩`, and Lean decides whether the target is `Prod`, `And`, `PProd`, or another compatible structure.

2.5 Unicode and ASCII

Lean's standard notation uses symbols such as `→`, `∀`, `∧`, `∨`, `¬`, `⟨`, and `⟩`. Editors insert them through abbreviations; ASCII alternatives exist for many constructs. The important point is semantic, not typographic:

Lean	Read as	Core idea
`A → B`	function / implication	non-dependent Π
`∀ x : A, P x`	for every x	dependent Π
`P ∧ Q`	conjunction	product-like inductive in `Prop`
`P ∨ Q`	disjunction	sum-like inductive in `Prop`
`¬ P`	not P	definitionally $P \to$ `False`
`∃ x, P x`	there exists x	inductive existential in `Prop`

2.6 Term mode and tactic mode

A theorem body can be a direct term:

Direct proof term

```
theorem compose (A B C : Prop) (f : B → C) (g : A → B) : A → C :=
  fun x => f (g x)
```

or a tactic script beginning with `by`:

Tactic proof producing the same kind of term

```
theorem compose' (A B C : Prop) (f : B → C) (g : A → B) : A → C := by
  intro x
  exact f (g x)
```

(The binders `(A B C : Prop)` are not decoration. Left out, Lean would auto-bind `A B C` as arbitrary sorts and reject the declaration with "`type of theorem ‘compose‘ is not a proposition`"; a `def` would accept it. See the warning box in Section 2.2.)

Tactics manipulate goals and construct a term behind the scenes. The kernel receives only the final elaborated term. Leant's `:prove` mode participates at the goal-manipulation layer, while `:synth` chiefly asks for a whole inhabitant of a target type.

2.7 Seeing what the elaborator inserted

Everything above is *surface* syntax. Two options make Lean print what a term became after elaboration: `pp.explicit` shows the implicit arguments, and `pp.notation false` turns off infix and bracket notation. The following outputs are verbatim from Lean 4.33.0.

Surface syntax versus elaborated term

```
set_option pp.explicit true
set_option pp.notation false
variable (P Q : Prop) (h : P ∧ Q) (p : Nat × Bool) (x : Nat)

#check x + 1
-- @HAdd.hAdd Nat Nat Nat (@instHAdd Nat instAddNat) x
--   (@OfNat.ofNat Nat (nat_lit 1) (instOfNatNat (nat_lit 1))) : Nat
#check ¬ P
-- Not P : Prop
#check P ∧ Q
-- And P Q : Prop
#check h.1
-- @And.left P Q h : P
#check p.2
-- @Prod.snd Nat Bool p : Bool
#check (⟨h.2, h.1⟩ : Q ∧ P)
-- @And.intro Q P (@And.right P Q h) (@And.left P Q h) : And Q P
#check ∃ n : Nat, n = 0
-- @Exists Nat fun n => @Eq Nat n (@OfNat.ofNat Nat (nat_lit 0) (instOfNatNat (nat_lit 0))) : Prop
#check {n : Nat // n < 10}
-- @Subtype Nat fun n => @LT.lt Nat instLTNat n (@OfNat.ofNat Nat (nat_lit 10) (instOfNatNat (nat_lit
 ↪  10))) : Type
```

The table summarizes the pattern. Note how much of the elaborated term was never typed: instance arguments (`instHAdd`, `instAddNat`), the numeral machinery (`OfNat.ofNat` applied to a raw literal), the propositions `P`, `Q` passed to `And.left`, and the constructor chosen for `⟨...⟩` from the expected type.

Surface form	Elaborated head	Remark
`A → B`, `∀ x : A, B x`	`Expr.forallE`	one kernel constructor for both
`fun x => t`	`Expr.lam`	binder type inferred if omitted
`x + 1`	`HAdd.hAdd _ _ _ inst x 1`	instance found by type-class search
`1`	`OfNat.ofNat Nat (nat_lit 1) _`	literals go through `OfNat`
`¬ P`	`Not P`	a definition; unfolds to `P → False`
`P ∧ Q`, `A × B`, `A ⊕ B`	`And P Q`, `Prod A B`, `Sum A B`	ordinary constant applications
`∃ x, P x`	`Exists (fun x => P x)`	the binder becomes a lambda argument
`{x : A // P x}`	`Subtype (fun x => P x)`	likewise
`⟨a, b⟩`	`And.intro`, `Prod.mk`, `Exists.intro`, `Subtype.mk`, ...	chosen from the expected type
`h.1`, `p.2`	`And.left h`, `Prod.snd p`	numeric projections become named ones
`[1, 2]`	`List.cons 1 (List.cons 2 List.nil)`	list notation is nested constructors

Chapter checkpoint

Arrows and applications have opposite associativity; binders carry visibility information; pattern matching is compiled; products and conjunctions can share constructor-shaped syntax; tactic scripts are programs that build proof terms. These facts explain many seemingly fussy steps in Leant's renderer.

Propositions Are Types, Proofs Are Terms

3.1 The Curry–Howard dictionary

The correspondence used by Lean can be summarized as follows:

Logic	Type theory
proposition P	type $P : \mathsf{Prop}$
proof of P	term $p : P$
$P \Rightarrow Q$	function type $P \to Q$
$P \wedge Q$	type with a proof of P and a proof of Q
$P \vee Q$	type tagged with either a proof of P or a proof of Q
$\top$	inhabited unit-like proposition
$\bot$	empty proposition
$\forall x : A,\ P(x)$	dependent function returning a proof for every x
$\exists x : A,\ P(x)$	package containing a witness and a proof
proof normalization	program reduction

This is not merely an analogy used in documentation. The kernel type checks proofs using the same mechanisms used to type check functions.

3.2 Implication is a function

To prove $P \to Q$, assume a proof of P and construct a proof of Q:

Implication introduction

```
theorem idProof (P : Prop) : P → P :=
  fun p => p
```

To use a proof of $P \to Q$, apply it to a proof of P:

Implication elimination

```
theorem modusPonens (P Q : Prop) : (P → Q) → P → Q :=
  fun hpq hp => hpq hp
```

The words “introduction” and “elimination” name the two directions of constructor use. Introduction builds a value of a type; elimination consumes one. Djinn’s LJT proof search is largely a disciplined automation of these structural moves.

3.3 Conjunction is product-like

To prove $P \wedge Q$, provide both components. To use it, project or pattern-match:

Conjunction

```
theorem reassoc (P Q R : Prop) :
    (P ∧ Q) ∧ R → P ∧ (Q ∧ R) :=
  fun h => ⟨h.1.1, ⟨h.1.2, h.2⟩⟩
```

The exact type `And P Q` lives in `Prop`; `Prod P Q` would live in a data universe if P and Q were data types. Their logical shapes are similar, but Lean keeps the nominal families distinct.

3.4 Disjunction is sum-like

A proof of $P \vee Q$ contains a tag saying which side holds. Introduction chooses a tag; elimination handles both cases:

Disjunction

```
theorem orSwap (P Q : Prop) : P ∨ Q → Q ∨ P :=
  fun h =>
    match h with
    | Or.inl hp => Or.inr hp
    | Or.inr hq => Or.inl hq
```

This is the same computational shape as swapping `Sum A B`, though proof irrelevance changes what may later be observed from proposition-valued data.

3.5 Truth, falsity, and negation

`True` has a constructor, so it is inhabited. `False` has no constructors, so a proof of it cannot be built in a consistent context. But if a hypothesis already has type `False`, it can be eliminated into any proposition or type:

Explosion

```
theorem absurdFromFalse (P : Prop) : False → P :=
  fun h => False.elim h
```

Negation is defined as implication to falsity:

$$\neg P \quad \text{is} \quad P \to \text{False}.$$

So a proof of `¬ P` is a function that turns any hypothetical proof of P into contradiction.

3.6 Universal and existential quantification

A universal proof is a dependent function:

Universal quantification

```
theorem eqSelf : ∀ n : Nat, n = n :=
  fun n => Eq.refl n
```

An existential proof packages a witness and evidence:

Existential witness

```
theorem existsEven : ∃ n : Nat, n % 2 = 0 :=
  ⟨0, rfl⟩
```

The witness can affect the type of the second component. That dependency is why quantifiers are not reducible to ordinary simple function and pair types.

3.7 Constructive logic and classical additions

The core proof interpretation is constructive: a proof should carry enough information to construct its conclusion. For example, a proof of $P \vee Q$ identifies a side. The law of excluded middle, $P \vee \neg P$, is not derivable for an arbitrary proposition from constructive rules alone.

Lean's libraries can add classical principles through named axioms and derived theorems. Once such a constant is in the term, the kernel can check the term relative to it. The distinction matters in Leant:

- Djinn's LJT engine searches intuitionistic logic.
- Leant can optionally offer classical premises after a constructive route is exhausted or refuted.
- A candidate using classical principles is still type-correct Lean, but its axiom dependencies differ.

Lean can report the assumptions on which a declaration depends with `#print axioms name`.

Core Lean has exactly three axioms: `propext` (propositional extensionality, `(a ↔ b) → a = b`), `Quot.sound` (the quotient axiom identifying related elements), and `Classical.choice` (`Nonempty α → α`, a choice function). Excluded middle is not an axiom but a theorem, `Classical.em`, derived from all three (Diaconescu's argument). `#print axioms` makes the dependence visible:

Classical versus constructive dependence

```
theorem dne (P : Prop) (h : ¬¬P) : P :=
  Classical.byContradiction (fun hn => h hn)

theorem dneConstr (P : Prop) (hp : P) : ¬¬P :=
  fun hn => hn hp

#print axioms dne
-- 'dne' depends on axioms: [propext, Classical.choice, Quot.sound]
#print axioms dneConstr
-- 'dneConstr' does not depend on any axioms
```

Double-negation *elimination* needs the classical route; double-negation *introduction* is a plain lambda. Both are accepted by the same kernel; only their axiom footprints differ.

Type correctness is relative to the environment

The kernel proves no theorem "from reality." It checks that a term has a type using the declarations and axioms in its environment. Sound engineering therefore tracks not only whether a term checks, but which opaque constants, axioms, instance declarations, and imported definitions it relies on.

Chapter checkpoint

Logical connectives have data-like introduction and elimination rules. This turns proof search into typed term synthesis. Djinn can automate a propositional structural fragment because that fragment has a finite, well-understood proof calculus; Lean's full dependent language is richer and requires additional care.

Type Inhabitation Is Program Synthesis

4.1 The question asked by a type

Given a type T, the inhabitation problem asks:

> Does there exist a term t such that $t : T$? If so, construct one.

For a proposition, this is theorem proving. For a data type, it is program construction. For a mixed dependent type, it can be both at once.

Some types have an obvious canonical inhabitant:

Canonical inhabitants

```
fun x : A => x

fun f : A → B => fun g : B → C => fun x : A => g (f x)

fun p : A × B => (p.2, p.1)
```

Other types are uninhabited without additional assumptions:

A structurally impossible polymorphic function

```
∀ A B : Type, A → B
```

After the caller chooses A, B, and a value of A, no source of a B is available. In a total, parametric language, no term can manufacture an arbitrary value of an arbitrary type.

Lean will nevertheless accept a declaration of this uninhabited type if the body is the placeholder `sorry`:

sorry is accepted, with a warning

```
example : ∀ A B : Type, A → B := sorry
-- warning: declaration uses `sorry`
```

"It elaborated" is not "it is inhabited"

`sorry` elaborates to the constant `sorryAx`, which has every type; the kernel checks the term as usual and only a *warning* records the gap. Any tool that decides success by "Lean did not report an error" must therefore also reject warnings of this kind, or inspect the final term for `sorryAx`.

4.2 A type constrains wiring, not behavior completely

A type can determine a surprising amount of structure. The type

$$(A \to B \to C) \to (A \to B) \to A \to C$$

essentially forces composition. But many types admit several behaviors:

Many inhabitants, different behavior

```
Nat → Nat

List α → List α

Bool → Bool
```

For `List α → List α`, possible total inhabitants include identity, reverse, constant empty, duplicate each element, drop every element, and many more. Type-directed synthesis alone cannot know which behavior the user intended.

This gives two distinct kinds of specification:

Structural specification
: Which inputs, outputs, polymorphic variables, sums, products, and constraints must fit together. This is what Djinn and Exference principally exploit.

Behavioral specification
: Equations, invariants, refinement predicates, resource bounds, or examples that distinguish extensionally different inhabitants of the same type.

Current Leant is strongest on structural synthesis. Its optional Length layer is a deliberately narrow example of behavioral assessment: after ordinary Lean type verification, certain list-spine candidates can be interpreted under a sealed length contract and checked through bounded, replayed arithmetic reasoning.

4.3 Soundness and completeness are different promises

Suppose a tool prints a candidate. Candidate *soundness* asks whether the printed term really has the requested Lean type. Leant answers this by re-elaboration and kernel checking.

Suppose instead the tool prints "uninhabited." That is a *completeness* claim about the search and translation. To justify it, the tool must know that:

- every relevant part of the Lean goal was represented faithfully;
- every allowed premise and constructor was included;
- no hidden type-class dictionary or opaque constant could carry useful data;
- the search procedure was complete for the represented logic;
- no configured budget, depth bound, queue pruning, or finite instantiation plan left unexplored cases.

A verifier can easily reject a bad positive candidate. It cannot rescue an unjustified negative claim, because there is no candidate to check.

The asymmetry that shapes Leant

A lossy translation can still produce sound positive answers if every answer is checked in the original language. The same lossy translation cannot support a sound "no answer exists" conclusion unless the loss is proven irrelevant for that query.

4.4 Search as proof construction

For the propositional fragment, the type's outer constructor suggests the next proof step:

Goal shape	Introduction move	Resulting work
$A \to B$	introduce $x : A$	synthesize B in extended context
$A \wedge B$	build a pair	synthesize both A and B
$A \vee B$	choose an injection	synthesize one branch
$\top$	use unit constructor	done
$\bot$	no introduction rule	must derive from context
atomic P	use a hypothesis or premise	apply something whose result is P

Left rules explain how hypotheses can be consumed. For a hypothesis $h : A \wedge B$, split it. For $h : A \vee B$, case-analyze both branches. For $h : A \to B$, synthesize an A and apply h to obtain B. A focused or contraction-free calculus controls the order so proof search terminates and avoids redundant duplication.

This is the logical heart of Djinn. Exference reaches similar terms by a different, heuristic best-first exploration over typed expression states.

Source trail

Djinn's object language and proof terms are defined in https://github.com/VladimirReshetnikov/Djex/blob/0cf82f2aa22debb1de2c7aaa13a2abbe190550d8/djinn/src-core/Djinn/Internal/LJTFormula.hs. The contraction-free LJT search and its explicit completeness/budget distinction are in `djinn/src-core/Djinn/Internal/LJT.hs`. Exference's priority-queue-driven engine is in `exference/src-core/Language/Haskell/Exference/Core/Internal/Exference.hs`.

Chapter checkpoint

Inhabitation unifies theorem proving and program synthesis. Types strongly constrain structure but often underdetermine behavior. Positive soundness can be recovered by checking candidates in Lean; negative completeness requires an exact fragment and exhaustive search.

Part II

Dependent Type Theory

Judgments, Contexts, and Substitution

Dependent type theory is easiest to understand as a small collection of judgments and rules. The notation may look formal, but it merely makes explicit what Lean's elaborator and kernel are tracking.

5.1 The typing judgment

The central judgment is

$$\Gamma \vdash t : A,$$

read "under assumptions Γ, term t has type A." The context is an ordered list of declarations such as

$$\Gamma = (A : \mathsf{Type},\ B : \mathsf{Type},\ f : A \to B,\ x : A).$$

From this context we can derive $\Gamma \vdash f\,x : B$.

Order matters because later types may mention earlier variables. This context is valid:

$$(A : \mathsf{Type},\ x : A,\ P : A \to \mathsf{Prop},\ h : P\,x).$$

The type of x mentions A; the type of P mentions A; the type of h mentions both P and x. Reordering the entries arbitrarily would make names appear before their declarations.

5.2 Variable and weakening rules

If a context contains $x : A$, then x may be used at type A:

Variable

$$\frac{}{\Gamma, x : A, \Delta \vdash x : A}$$

provided the context is well formed.

A derivation that does not use a new assumption remains valid after that assumption is added. This is weakening. In code, adding a local variable does not invalidate earlier expressions, though dependent references require the context to remain ordered.

5.3 Substitution is the engine of dependency

Suppose a body has a free variable x:

$$\Gamma, x : A \vdash t : B(x).$$

If another term a has type A, substitution replaces x by a everywhere:

$$\Gamma \vdash t[a/x] : B(a).$$

The crucial point is that substitution occurs not only in the term but also in its type. This is what distinguishes dependent application from ordinary function application.

In Lean, if

A dependent function

```
f : (n : Nat) → Vector α n
k : Nat
```

then `f k` has type `Vector α k`. The argument k is substituted into the codomain.

5.4 Binding, alpha-equivalence, and capture avoidance

The terms

$$\lambda x : A.\, x \qquad \text{and} \qquad \lambda y : A.\, y$$

mean the same thing. Bound variable names are presentation aids; renaming them consistently does not change the term. This is alpha-equivalence.

A substitution implementation must avoid accidentally capturing free variables. If one substitutes y for x in $\lambda y.\, x$, naively obtaining $\lambda y.\, y$, the formerly free y has been captured. The binder must first be renamed, for example:

$$(\lambda z.\, x)[y/x] = \lambda z.\, y.$$

Lean's kernel representation avoids many name-management problems by using de Bruijn indices for bound variables. Djex's shared source types and proof terms instead carry explicit identities and therefore contain binder freshening, alpha-normalization, rigid/flexible variable distinctions, and capture-avoiding substitution machinery.

Locally nameless Lean expressions

A kernel lambda body represents its bound variable with `Expr.bvar 0`; an outer binder is `bvar 1`, and so on. During traversals, Lean commonly instantiates bound variables with fresh free-variable identifiers in a local context. The names printed to users do not determine binding identity.

5.5 Metavariables are temporary unknowns

During elaboration, Lean may know that a term must exist before it knows the term. It represents such holes with metavariables. A metavariable has a local context and an expected type; tactics and unification assign it later.

For example, elaborating

An inferred argument

```
List.nil
```

creates an unknown element type α until the surrounding expected type or later constraints determine it. Similarly, the placeholder _ asks the elaborator to create and solve a metavariable.

Metavariables are not accepted by the kernel. The source definition of `Lean.Expr` explicitly permits `mvar` because the same expression datatype is used by the elaborator, but the kernel type checker rejects any expression that still contains one:

the pinned `src/kernel/type_checker.cpp` throws "`kernel type checker does not support meta variables`" when `infer_type` meets an `mvar`.

Why this matters for Leant

Leant's serializer runs inside Lean's metaprogramming layer, where it can instantiate metavariables, weak-head normalize expressions, inspect binder information, ask whether a term is a class application, and query inductive declarations. Its output must describe a closed enough fragment for Haskell to consume. Later, rendered candidates deliberately reintroduce _ only as surface syntax so Lean can perform a fresh, exact elaboration.

Chapter checkpoint

A typing judgment is always relative to an ordered context. Dependency means substitution changes types as well as terms. Bound names are not identities. Metavariables are elaboration-time obligations, never kernel-level proof holes.

Dependent Function Types: The Pi-Type

6.1 From arrows to dependent arrows

An ordinary function type $A \to B$ says that every input has the same result type B. A dependent function type permits the result type to mention the input:

$$\Pi(x : A),\ B(x).$$

Lean uses several surface forms for this one core construction:

Pi-type notation

```
(x : A) → B x
∀ x : A, B x
{x : A} → B x
[x : C] → B x
```

When x does not occur in B, the dependent product degenerates to an ordinary arrow:

$$A \to B \quad \text{is notation for} \quad \Pi(\_ : A),\ B.$$

This is why the kernel needs only one constructor, `Expr.forallE`, for both function types and universal quantification.

6.2 Formation, introduction, and elimination

The formation rule says that if A is a type and $B(x)$ is a type for each $x : A$, then the dependent function type is a type:

Π formation

$$\frac{\Gamma \vdash A : \mathrm{Sort}\ u \qquad \Gamma, x : A \vdash B : \mathrm{Sort}\ v}{\Gamma \vdash \Pi(x : A), B : \mathrm{Sort}(\mathrm{imax}(u, v))}$$

Introduction is lambda abstraction:

Π introduction

$$\frac{\Gamma, x : A \vdash t : B}{\Gamma \vdash (\lambda x : A.\, t) : \Pi(x : A), B}$$

Elimination is application, with substitution in the result type:

Π elimination

$$\frac{\Gamma \vdash f : \Pi(x : A), B(x) \qquad \Gamma \vdash a : A}{\Gamma \vdash f\ a : B(a)}$$

These three rules simultaneously explain ordinary functions, polymorphism, universal quantification, and type constructors.

6.3 Four uses of the same constructor

6.3.1 Functions from values to values

Ordinary function

```
def successor : Nat → Nat := fun n => n + 1
```

The codomain does not depend on n.

6.3.2 Functions from types to values

Polymorphic identity

```
def ident {α : Sort u} (x : α) : α := x
```

The function quantifies over a type α and then over a value of that type. Its full type is approximately

$$\Pi(\alpha : \text{Sort } u),\ \alpha \to \alpha.$$

6.3.3 Functions from values to types

`Vector α` maps a natural number to a type:

A type family

```
#check Vector
-- Vector.{u} (α : Type u) (n : Nat) : Type u

#check (Vector)
-- Vector : Type u_1 → Nat → Type u_1
```

(`#check` on a bare constant prints its *signature*, with named binders and the universe parameter `.{u}`; wrapping the name in parentheses makes Lean elaborate it as a term and print the arrow form.) For fixed α, `Vector α 3` and `Vector α 4` are different types. A type family is simply a function whose codomain is a universe.

6.3.4 Functions from types to types

`List` is a type constructor:

A polymorphic type constructor

```
#check List
-- List.{u} (α : Type u) : Type u

#check (List)
-- List : Type u_1 → Type u_1
```

At the kernel level, this is again an ordinary constant whose type is a Π-type.

6.4 Universal propositions are dependent functions

The proposition

$$\forall n : \mathbb{N},\ n = n$$

is represented by a dependent function type whose codomain lies in `Prop`. Its proof is a function returning a proof for each input:

A universal proof

```
theorem reflAll : ∀ n : Nat, n = n :=
  fun n => rfl
```

The notation `∀` changes how humans read the type, not the kernel constructor.

6.5 Dependency can be computationally relevant

Consider a function that returns a vector whose length is its input:

Result type depends on the argument

```
def replicateVec (α : Type u) (a : α) :
    (n : Nat) → Vector α n
  | 0 => #v[]
  | n + 1 => (replicateVec α a n).push a

#eval replicateVec Nat 7 3
-- { toArray := #[7, 7, 7], size_toArray := _ }
```

For each *n*, the result has a different type. The recursive branch must return exactly `Vector α (n + 1)`, not merely "some vector": `#v[]` is the empty vector of type `Vector α 0`, and `Vector.push : Vector α n → α → Vector α (n + 1)` lengthens the type by one along with the data. The type checker tracks the index through the computation.

(In core Lean, `Vector α n` is a *structure* wrapping an `Array α` together with a proof `size_toArray : toArray.size = n`; there is no `Vector.nil`/`Vector.cons`. Chapter 10 defines an inductive family `Vec` with exactly those two constructors, which is the textbook shape most type-theory expositions have in mind.)

This expressive power is precisely what a simply typed search calculus lacks. Djinn can treat an entire formula such as `∀ n, P n` as an opaque atom and move it around, but it cannot generally synthesize a term by inspecting *n*, changing a result index, or transporting across an equality.

6.6 Rank and prenex quantification

A type is rank-1 when universal quantifiers appear only at the outermost level. A higher-rank type places a polymorphic function inside an argument:

A rank-2 shape

```
((∀ α : Type, α → α) → Nat)
```

Djex's shared Haskell-like type representation supports explicit `ForallType` layers and bounded handling of certain higher-rank uses. But the quantifier is still a type-scheme mechanism, not full term dependency. Leant must preserve Lean binder visibility and, for some instantiations, enumerate reconstruction variants.

Do not confuse two meanings of "forall"

In Lean, every function binder is represented by the same dependent Π constructor. In Djex's source type language, `ForallType` means Haskell-style type-variable quantification, while `FunctionType` means a value-level arrow. Leant's translation decides how a Lean `forallE` maps across that split.

Chapter checkpoint

The Π-type unifies functions, polymorphism, universal proofs, and type families. Application substitutes the argument into the result type. Lean's single kernel constructor is richer than Djex's separate Haskell-style forall and arrow constructors, so translation is semantic rather than syntactic.

Dependent Pairs, Subtypes, and Existence

7.1 The Sigma-Type

A dependent pair packages a first component $x : A$ and a second component whose type may depend on x:

$$\Sigma(x : A),\ B(x).$$

Lean's general dependent pair is `Sigma`:

A length-indexed package

```
def SomeVector (α : Type u) :=
  Sigma fun n : Nat => Vector α n
```

A value of `SomeVector α` contains a natural number n and a vector of exactly that length. The second projection's type depends on the first projection.

Constructing and consuming a Sigma

```
def oneVector (a : α) : SomeVector α :=
  ⟨1, #v[a]⟩

def packedLength (v : SomeVector α) : Nat :=
  v.1

#eval packedLength (oneVector "x")
-- 1
```

The ordinary product $A \times B$ is the non-dependent special case where B ignores the first component.

7.2 Why dependent pairs are harder to translate

A structural synthesis engine can represent a pair as "produce an A and a B." For a dependent pair it must instead produce an $a : A$ and then produce a term of $B(a)$. The second search problem is not even known until the first term is chosen. If $B(a)$ contains computation, equality, or indexed inductives, simple propositional search is insufficient.

Leant therefore expands only carefully characterized product-like inductives with non-dependent fields. A genuinely dependent structure, such as a constructor whose later field type mentions an earlier field, remains opaque to Djinn's structural logic.

7.3 Subtypes

A subtype packages data together with a proposition about it:

A bounded natural number

```
def SmallNat := {n : Nat // n < 10}

def three : SmallNat :=
  ⟨3, by decide⟩
```

The first component is computational data. The second is a proof in `Prop`. Because proofs are irrelevant for computation, Lean can erase the proof when generating executable code, while still using it during type checking.

A value of `Fin n` is essentially a natural number together with a proof that it is below n. Its type depends on the bound:

An index carrying a bound

```
#check Fin
-- Fin (n : Nat) : Type
```

This lets an array access API require evidence that an index is in range. The caller cannot pass an arbitrary natural number without also discharging the bound.

7.4 Existence in `Prop` is not data-level `Sigma`

Lean's `Exists` is an inductive proposition:

Existential proposition

```
∃ x : A, P x
```

It resembles a dependent pair, but it lives in `Prop`. Lean's elimination restrictions and proof irrelevance prevent arbitrary extraction of computational data from a proof of existence into a data universe. This preserves the distinction between merely proving that a witness exists and carrying a witness as executable data.

Use `Sigma` when the witness is data to be computed with. Use `Exists` when the package is a proposition and only logical consequences are needed.

Similar notation does not imply interchangeable elimination

Both `⟨w, h⟩ : ∃ x, P x` and `⟨w, v⟩ : Sigma B` look like pairs. Their universe and elimination behavior differ. Leant's fragment classifier must retain those nominal and sort distinctions even when the search logic sees a similar product shape.

7.5 Structures are one-constructor inductives

A Lean structure is a convenient declaration of an inductive type with one constructor and named projections:

A dependent structure

```
structure SizedBuffer (α : Type u) where
  size : Nat
  data : Vector α size
```

The field `data` depends on `size`. Lean generates a constructor and projections; the kernel checks the inductive declaration and associated recursor. The compact `Expr.proj` node represents projections in elaborated expressions.

Non-dependent records can often be treated as products. Dependent records need transport-aware elimination and remain outside Leant's ordinary structural expansion.

Chapter checkpoint

A Σ-type is a pair whose second component depends on the first. Ordinary products, subtypes, existentials, and structures are related but not identical. Dependent fields are a sharp boundary for a propositional synthesis engine.

Universes, Sort, Type, and Prop

8.1 Why types need types

If every term has a type, what is the type of `Nat`? It is `Type`. What is the type of `Type`? It is `Type 1`. Continuing prevents the inconsistent self-typing rule `Type : Type`.

Lean's primitive notation is `Sort u`. Two common abbreviations are

$$\texttt{Prop} = \texttt{Sort } 0, \qquad \texttt{Type } u = \texttt{Sort}(u+1).$$

Thus `Type` is `Type 0`, or `Sort 1`.

Sort

$$\frac{}{\Gamma \vdash \texttt{Sort } u \,:\, \texttt{Sort } (u+1)}$$

Instances: `Prop : Type` (that is, `Sort 0 : Sort 1`), `Type : Type 1`, and for parameters, `#check Sort (imax u v)` prints `Sort (imax u v) : Type (imax u v)`, i.e. Sort $(\mathrm{imax}(u,v)+1)$.

8.2 Lean's universes are non-cumulative

Lean uses a countable hierarchy of *non-cumulative* universes. A type that inhabits `Type` is not automatically treated as an inhabitant of `Type 1`. Universe-polymorphic definitions are instead instantiated at the required level.

This often surprises users familiar with cumulative systems. The hierarchy still satisfies

Universes themselves inhabit the next universe

```
#check Type
-- Type : Type 1

#check Type 1
-- Type 1 : Type 2
```

but a small data type is not silently lifted into every larger universe. Lean's universe parameters and `max`/`imax` expressions keep definitions reusable without cumulative coercions.

8.3 Universe polymorphism

A polymorphic identity should work for propositions, ordinary data, and higher-universe objects:

Sort-polymorphic identity

```
def sid.{u} {α : Sort u} (x : α) : α := x
```

The universe parameter `u` is instantiated at each use. Constants store a list of universe parameters in their declaration and a list of universe arguments in each `Expr.const` occurrence.

Lean's internal universe levels are built from:

- `zero`;
- `succ u`;
- `max u v`;
- `imax u v`;
- named parameters;
- elaboration metavariables.

Kernel expressions cannot retain level metavariables, just as they cannot retain term metavariables.

8.4 The meaning of `imax`

The universe of a dependent function type is

$$\operatorname{imax}(u, v) = \begin{cases} 0, & v = 0, \\ \max(u, v), & v > 0. \end{cases}$$

This special operation encodes the impredicativity of `Prop`. If the codomain of a Π-type is a proposition, the whole dependent function type remains a proposition no matter how large the domain is:

$$\Pi(A : \text{Type } 3),\ P(A) : \text{Prop}.$$

Lean confirms this directly: `#check ∀ (T : Type 3), T = T` prints `∀ (T : Type 3), T = T : Prop`, whereas the data-valued `#check (T : Type 3) → T` prints `(T : Type 3) → T : Type 4`. (Do not try this with a very large literal: Lean's `maxUniverseOffset` option rejects level offsets above 32 by default.)

A proposition may quantify over objects from arbitrary universes without leaving `Prop`. By contrast, a data-valued dependent function must live high enough to contain both its domain and codomain structure.

The pinned kernel implementation computes this literally: `infer_pi` collects domain sort levels and folds `mk_imax` from the final codomain level.

8.5 Proof irrelevance

All proofs of the same proposition are treated as definitionally equal by Lean's kernel. If $p, q : P$ with $P : \text{Prop}$, then the kernel's definitional equality can regard p and q as equal without comparing their proof structure.

Proof irrelevance supports efficient erasure and prevents programs from branching on which proof of a proposition was supplied. It also explains why proposition-valued structures are not interchangeable with data-valued structures even when their constructor shapes match.

8.6 Large elimination and restrictions

Because Prop is impredicative and proof-irrelevant, arbitrary elimination from proposition-valued inductives into data would allow computational results to depend on erased proofs. Lean therefore restricts such elimination, with special cases for subsingleton-like propositions. One may eliminate False into any sort, but one may not generally turn an arbitrary existential proof into a data-level witness.

The restriction is enforced by the elaborator's pattern compiler and, ultimately, by the type of the recursor Lean generated for the inductive: for Exists, Or, and most other propositions the motive of .rec/.casesOn ranges over Prop only.

Extracting a witness: rejected into Nat, accepted into Prop

```
def getWitness (h : ∃ n : Nat, n > 0) : Nat :=
  match h with
  | ⟨n, _⟩ => n
-- error: ... recursor `Exists.casesOn` can only eliminate into `Prop`

theorem useWitness (h : ∃ n : Nat, n > 0) : ∃ m : Nat, m > 0 :=
  match h with
  | ⟨n, hn⟩ => ⟨n, hn⟩
-- accepted: the result type is a proposition

def fromFalse (h : False) : Nat := False.elim h
-- accepted: False has no constructors, so eliminating it into any Sort is safe
```

Compare @False.elim : {C : Sort u_1} → False → C with Exists.rec, whose motive is Exists p → Prop: the universe of the motive is where the rule lives.

Why this matters for Leant

The Leant serializer records whether the goal itself is proposition-valued or data-valued and preserves distinctions among And/Prod/PProd, Or/Sum/PSum, and empty/unit families. Djinn may search their common logical shape, but the renderer relies on Lean's expected type to recover the correct universe-specific constructor.

Source trail

Universe syntax is defined in https://github.com/leanprover/lean4/blob/79c4cd09fdab845aa6066bcbb09a663876601027/src/Lean/Level.lean. The kernel rules for sorts, applications, and Π universes are implemented in src/kernel/type_checker.cpp. The official reference explicitly describes Lean's universes as non-cumulative and Type u as Sort (u+1).

Chapter checkpoint

Lean avoids Type : Type with a non-cumulative universe hierarchy. Prop is Sort 0; data universes start at Sort 1. The Π universe uses imax, making Prop impredicative. Proof irrelevance is a kernel conversion principle, not an optional theorem.

Computation and Definitional Equality

9.1 Two ways terms can be equal

Lean distinguishes:

Definitional equality
: $t \equiv u$: the type checker recognizes the terms as equal by built-in computation and conversion. No proof object is required.

Propositional equality
: $h : t = u$: an explicit term inhabits the inductive equality proposition.

For example,

$$(\lambda x.\, x + 1)\ 4 \equiv 5$$

by beta reduction and natural-number computation. Therefore `rfl` can prove an equality whose two sides reduce to the same normal form.

Computation makes reflexivity sufficient

```
example : (fun x : Nat => x + 1) 4 = 5 := rfl
```

9.2 The main reduction rules

9.2.1 Beta reduction

Applying a lambda substitutes the argument into the body:

$$(\lambda x : A.\, t)\ a \longrightarrow t[a/x].$$

9.2.2 Delta reduction

A transparent definition may unfold to its body:

$$\text{twice}\ f\ x \longrightarrow f(f x).$$

The kernel uses lazy delta reduction and reducibility hints to decide an efficient unfolding order. The hints affect performance, not the logical meaning of transparent definitions.

9.2.3 Iota reduction

A recursor or pattern match applied to a constructor reduces to the matching branch. Informally:

$$\text{match inl}\ a\ \text{with}\ \cdots \longrightarrow \text{left branch with}\ a.$$

9.2.4 Zeta reduction

A `let` expression substitutes its value into the body:

$$\text{let } x := a;\ t \longrightarrow t[a/x].$$

9.2.5 Projection reduction

Projecting a field from a constructor reduces to that field. Lean also supports eta-like conversion for functions and nonrecursive structures in the kernel's definitional equality procedure.

9.2.6 A reduction trace

Putting the rules together on the very first example of Chapter 1 (`#print twice` shows the stored body `fun f x => f (f x)`):

$$\begin{aligned}
\text{twice}\ (\lambda n.\, n+1)\ 40 &\longrightarrow_\delta (\lambda f.\lambda x.\, f\,(f\,x))\ (\lambda n.\, n+1)\ 40 \\
&\longrightarrow_\beta (\lambda x.\, (\lambda n.\, n+1)\,((\lambda n.\, n+1)\,x))\ 40 \\
&\longrightarrow_\beta (\lambda n.\, n+1)\,((\lambda n.\, n+1)\,40) \\
&\longrightarrow_\beta (\lambda n.\, n+1)\,(40+1) \longrightarrow_\beta (40+1)+1 \\
&\longrightarrow 42 \qquad \text{(literal arithmetic: the kernel evaluates } \texttt{Nat.add} \text{ on numerals natively)}
\end{aligned}$$

The order shown is one of several; confluence guarantees the same result. Lean agrees on both sides of the interface: `#reduce twice (fun n => n + 1) 40` prints `42`, and `example : twice (fun n => n + 1) 40 = 42 := rfl` is accepted, because `rfl` succeeds exactly when the two sides are definitionally equal.

9.3 Weak-head normalization

The kernel rarely normalizes every subterm eagerly. It computes to *weak-head normal form*: enough to expose the outer constructor. If a purported function type is hidden behind a definition, `ensure_pi` weak-head normalizes until it sees a Π or reports an error. If a purported sort is hidden, `ensure_sort` does the analogous work.

This strategy matters operationally. Full normalization can be expensive; type checking usually needs only the head shape and selective definitional equality.

9.4 Conversion in typing

A conversion rule permits a term inferred at type A to be used where a definitionally equal type B is expected:

Conversion

$$\frac{\Gamma \vdash t : A \qquad A \equiv B \qquad \Gamma \vdash B : s}{\Gamma \vdash t : B}$$

Dependency makes this indispensable. A recursive function may produce `Vector α (Nat.succ n)` while the expected type prints as `Vector α (n + 1)`; whether the terms align depends on reduction and the definitions involved.

9.5 Opaque declarations and theorems

Not every named constant unfolds. An `opaque` constant and an `axiom` never unfold in the kernel: the pinned kernel's `constant_info::has_value()` answers false for both, so `is_delta` never selects them. Theorems sit in between. The kernel *may* unfold a theorem body if a definitional-equality check comes down to it (`has_value()` is true for theorems), but it does so only as a last resort, and Lean's elaborator never does so at all: `Lean.Meta.getUnfoldableConst?` in `src/Lean/Meta/GetUnfoldableConst.lean` returns `none` for every `thmInfo`, so `whnf`, `simp`, and `rw` treat a theorem as a black box. Because any two proofs of the same proposition are definitionally equal anyway (proof irrelevance), nothing is lost. This modularity keeps checking stable and prevents proof terms from expanding uncontrollably.

A term can therefore use a theorem or an `opaque` constant freely as a name with a type. What it should not do is rely on computing through the hidden body.

9.6 Why a renderer cannot implement definitional equality itself

Leant's Haskell side sees strings, fragment trees, Djex types, and candidate graphs. Reimplementing Lean's full conversion relation there would require reproducing environment lookup, universe instantiation, recursive recursors, projections, proof irrelevance, native reductions, and transparency behavior. That would create a second, drifting authority.

Instead, Leant uses Lean's own metaprogramming operations during translation and Lean's own elaborator/kernel during verification. Haskell-side normalization is limited to its own representations and never grants Lean type authority.

Expected-type elaboration is a semantic service

A candidate such as `⟨x, y⟩` is intentionally incomplete as a standalone string. Against an exact expected type, Lean selects a constructor, inserts universe parameters and implicit arguments, resolves notation, checks conversion, and produces a kernel expression. Leant treats that elaboration as the authoritative final lowering pass.

Chapter checkpoint

Definitional equality is computation built into type checking. It includes beta, delta, iota, zeta, projections, proof irrelevance, and eta behavior implemented by the kernel's conversion algorithm. Leant delegates this entire relation back to Lean instead of approximating it in Haskell.

Chapter 10

Inductive Types, Recursors, and Termination

10.1 Data by constructors

An inductive declaration introduces a type as the least collection closed under specified constructors:

Natural numbers

```
inductive MyNat where
  | zero : MyNat
  | succ : MyNat → MyNat
```

Every value of `MyNat` is built by a finite constructor tree. This gives both a way to construct values and a complete way to consume them: handle `zero` and handle `succ n` assuming the recursive result for n.

10.2 Parameters and indices

A parameter is uniform across a whole inductive family. An index may vary by constructor:

An indexed vector family

```
inductive Vec (α : Type u) : Nat → Type u where
  | nil : Vec α 0
  | cons {n : Nat} : α → Vec α n → Vec α (n + 1)
```

Here α is a parameter and the length is an index. The constructors return different members of the family. Pattern matching on a vector therefore refines what is known about its length.

The Lean kernel's `InductiveVal` explicitly records the number of parameters and indices, constructor names, universe parameters, whether the family is recursive, and other metadata used to construct and check recursors.

10.3 Recursors are the primitive eliminators

For each accepted inductive declaration, Lean generates a recursor. For natural numbers, its shape is conceptually:

$$\text{Nat.rec} : \Pi(\mathit{motive} : \text{Nat} \to \text{Sort } u),\ \mathit{motive}\ 0 \to (\Pi n, \mathit{motive}\ n \to \mathit{motive}\ (n+1)) \to \Pi n, \mathit{motive}\ n.$$

The *motive* is the result type family. If it is constant, recursion returns ordinary data. If it depends on the input, the same recursor performs induction or dependent pattern matching.

The recursor can be called directly. Here the motive is the constant family `fun _ => Nat`, so `Nat.rec` is just primitive recursion; the accepted `rfl` proofs show that iota reduction fires even on the open term `n + 1`:

Nat.rec used by hand

```
def double (n : Nat) : Nat :=
  Nat.rec (motive := fun _ => Nat) 0 (fun _ ih => ih + 2) n

#eval double 5
-- 10
example : double 3 = 6 := rfl
theorem double_succ (n : Nat) : double (n + 1) = double n + 2 := rfl
```

Ordinary recursive definitions such as `append` in Chapter 2 are compiled to this primitive (`#print append` shows a body built from `List.brecOn`, a course-of-values wrapper around `List.rec`), which is how the kernel can accept them without any notion of "recursive call."

This one principle unifies:

- defining a numeric function by recursion;
- proving a proposition by induction;
- constructing an indexed result whose type tracks the input;
- eliminating a sum or product by cases.

10.4 Pattern matching is elaborated

User-friendly equations and `match` expressions are compiled into recursors, auxiliary definitions, and equality transports. The equation compiler is not trusted. Its generated term is checked by the kernel against the declared type.

For a dependent match, each branch may have a refined expected type. Consider a vector head operation that requires nonempty length:

Index refinement by pattern matching

```
def Vec.head : Vec α (n + 1) → α
  | .cons a _ => a
```

The impossible `nil` branch is absent because `Vec α 0` cannot unify with `Vec α (n + 1)`.

10.5 Strict positivity

Inductive definitions must satisfy positivity conditions. Roughly, the type being defined may occur recursively only in positions that preserve monotonicity. A forbidden negative occurrence would allow paradoxical self-application and destroy consistency.

A negative occurrence, rejected by the kernel

```
inductive Bad where
  | mk : (Bad → Nat) → Bad
-- error: (kernel) arg #1 of 'Bad.mk' has a non positive occurrence
--        of the datatypes being declared
```

Note the `(kernel)` prefix: this check is performed by the trusted checker itself when the inductive declaration is added, not merely by the elaborator.

Lean's inductive checker validates positivity, universe constraints, constructor result heads, and elimination restrictions before adding the compiled declarations to the environment.

10.6 Termination and totality

Lean's logical definitions must terminate. Structural recursion is accepted when recursive calls are made on syntactically smaller constructor fields. More general recursion can be compiled through well-founded recursion, requiring a proof that calls decrease according to a well-founded relation.

The kernel itself does not have a primitive "general recursive definition" constructor. The source comments in `src/Lean/Declaration.lean` state that recursive definitions are compiled using recursors and `WellFounded.fix`. Unsafe or partial definitions belong to separate safety classes and cannot be used to prove safe logical declarations as though they were total.

10.7 Why recursive synthesis is a separate problem

A type such as `List α → List α` does not say whether the result should recurse over the list, preserve length, reverse order, or ignore input. Inventing a recursive program requires choosing:

- an eliminator or recursion scheme;
- the argument on which to recurse;
- branch bodies;
- recursive-call arguments;
- termination evidence;
- possibly a dependent motive.

Leant therefore does not pretend that general recursive synthesis follows from propositional inhabitation. It supports bounded constructor introduction for certain recursive types and can expose curated library functions as provider premises. Recursive elimination is restricted and engine-specific; arbitrary recursor invention remains out of scope for ordinary `:synth`.

Constructor inventory is evidence

For a nonrecursive, nonindexed inductive with explicit non-dependent fields, Leant may translate the declaration into a finite sum of products. For a recursive type it records whether the constructor inventory is complete and how recursive occurrences appear. An incomplete or ambiguous inventory may still help produce positive candidates, but it cannot justify a conclusive negative result.

Chapter checkpoint

Inductive declarations generate constructors and recursors. Parameters are uniform; indices vary and enable type refinement. Pattern matching is compiled and checked. Total recursion is reduced to structurally or well-foundedly justified terms. General recursive synthesis is therefore much richer than propositional proof search.

Equality, Rewriting, and Transport

11.1 Equality is an indexed inductive family

Lean's propositional equality has one constructor:

Conceptual shape of equality

```
Eq.refl : (a : α) → a = a
```

In `Eq a b`, the type `α` and the left endpoint `a` are *parameters* and the right endpoint `b` is the *index* (`#print Eq` reports "`number of parameters: 2`"); the constructor `Eq.refl a` fixes that index to `a`. A proof of `a = b` can therefore be built directly only when the two sides are the same term, modulo definitional equality.

An equality proof can therefore refine types when eliminated. This is much stronger than a Boolean comparison.

11.2 Reflexivity and computation

The tactic or term `rfl` constructs `Eq.refl _`. It succeeds when both sides are definitionally equal:

Reflexivity after reduction

```
example (x : Nat) : (fun y => y) x = x := rfl
example : List.length [10, 20] = 2 := rfl
```

When equality is not definitional, one needs a theorem or a derivation using induction, congruence, rewriting, arithmetic, or another proof method.

All six lines below look equally trivial; Lean 4.33.0 accepts four and rejects two.

rfl versus decide

```
example : 2 + 2 = 4 := rfl                -- accepted
example : 100 * 100 = 10000 := rfl        -- accepted: literal arithmetic
example (n : Nat) : n + 0 = n := rfl      -- accepted
example (n : Nat) : 0 + n = n := rfl      -- rejected
example : 10 < 20 := rfl                  -- rejected
example : 10 < 20 := by decide            -- accepted
```

`rfl` is not "obviously true"; it is "the same after computation"

`Nat.add` is defined by recursion on its *second* argument, so `n + 0` computes to `n` while `0 + n` is stuck on the variable `n`; the latter needs induction (`Nat.zero_add`). For `10 < 20` Lean reports "`Type mismatch: rfl has type ?m = ?m but is expected to have type 10 < 20`": `<` on `Nat` unfolds to the inductive predicate `Nat.le 11 20`, which is not an equality at all, so `Eq.refl` cannot even have the right head. `decide` instead runs the `Decidable` instance for `10 < 20` and checks that it evaluates to `isTrue`. For a synthesis tool this is the difference between "the checker sees these as the same" and "a decision procedure was invoked".

11.3 Transport

If $h : a = b$ and $P : A \to \text{Sort } u$, a value of $P(a)$ can be transported to $P(b)$:

$$\text{Eq.rec} : P(a) \to a = b \to P(b).$$

Lean's rewrite notation hides this eliminator:

Transport across type equality

```
def castValue {α β : Sort u} (h : α = β) (x : α) : β :=
  h ▸ x
```

The term x cannot be returned directly because its type is α, not β. The equality proof changes the expected type.

11.4 Transport in indexed families

Suppose v : Vector α n and $h : n = m$. Transport yields a vector at index m:

Changing an index with an equality

```
def changeLength {n m : Nat} (h : n = m)
    (v : Vector α n) : Vector α m :=
  h ▸ v
```

A synthesis engine that treats `Vector α n` and `Vector α m` as unrelated opaque atoms will not discover this term. A synthesis engine that simply erases the indices and treats both as `Vector α` may generate unsound terms. Correct dependent synthesis must reason about the equality and insert transport.

11.5 Dependent pattern matching creates equalities implicitly

When matching an indexed constructor, Lean refines the index and may insert transports behind the scenes. The surface branch looks simple because the elaborator constructs the appropriate motive and equality reasoning.

This is a major line between Leant's `:synth` and `:prove` modes. The former can transport an opaque dependent proposition as a whole if the surrounding logic only rearranges it. It does not generally invent index equalities, dependent motives, or transport chains. In proof mode, standard Lean tactics and user guidance can expose and solve those obligations.

11.6 Homogeneous and heterogeneous equality

Ordinary `Eq` compares terms of one common type. Dependent developments sometimes need heterogeneous equality, where the endpoint types differ. Lean provides tools such as `HEq`, but most practical reasoning reduces heterogeneous equalities to ordinary equality plus transport.

The presence of such relations is another warning against flattening dependent applications into nominal strings. Their argument structure and universe instantiations determine what can be rewritten.

Dependency is not decoration

An index in `Vector α n` can govern which constructors exist, which patterns are possible, and what result type a branch must produce. Erasing the index changes the typing problem. Keeping it only as an opaque identity preserves sound transport of whole values but sacrifices the ability to synthesize index-sensitive computations.

Chapter checkpoint

Propositional equality is an indexed family whose eliminator performs transport. `rfl` uses definitional equality; rewriting uses an explicit proof. Indexed synthesis often requires equality generation and transport, which Leant intentionally does not model in its ordinary structural fragment.

The Calculus of Constructions and Its Inductive Extension

12.1 Why the name matters

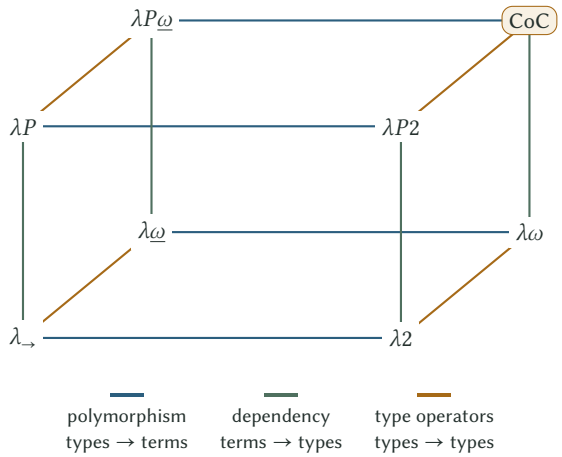
The Calculus of Constructions (CoC) is a compact dependent lambda calculus in which terms, types, and type constructors inhabit one stratified language. It combines:

- ordinary functions from terms to terms;
- polymorphic functions from types to terms;
- type operators from types to types;
- dependent type families from terms to types.

Lean is based on a version of this tradition with a countable hierarchy of non-cumulative universes, an impredicative proof-irrelevant `Prop`, inductive types and recursors, quotient support, and a modern elaboration/programming infrastructure. It is therefore better to say "Lean's dependent type theory is in the CoC/CIC family" than to identify it mechanically with one historical presentation.

12.2 The lambda cube

The lambda cube classifies typed lambda calculi by three independent forms of dependency added to the simply typed lambda calculus. Let $*$ stand informally for ordinary types and $\Box$ for kinds or universes.



The top corner supports all three axes. The ordinary Π-type constructor is expressive enough to encode them uniformly, with universes preventing paradox.

12.3 What inductive types add

Pure CoC has dependent functions but does not by itself provide the practical repertoire of freely generated datatypes and induction principles used in Lean. The Calculus of Inductive Constructions (CIC) adds inductive families together with checked eliminators.

From the user's perspective, this supplies `Nat`, `List`, equality, conjunction, disjunction, existential quantification, structures, well-founded recursion infrastructure, and essentially every concrete datatype.

From the kernel's perspective, an inductive block is checked once and compiled into declarations for:

- the type former;
- each constructor;
- recursors and related eliminators;
- no-confusion and projection machinery in higher layers as appropriate.

12.4 A compact core grammar

Ignoring literals, metadata, and projections, the dependent lambda-calculus core can be sketched as

$$\begin{aligned} t ::= x \mid \mathrm{Sort}\ u \mid c.\{\vec{u}\} \mid t\ t \mid \lambda(x : A), t \\ \mid \Pi(x : A), B \mid \mathrm{let}\ x : A := t;\ u. \end{aligned}$$

Lean's actual `Expr` datatype closely mirrors this grammar, plus free and meta variables, literals, metadata, and compact projections. Inductive types do not require a new general expression constructor: they enter as constants and recursors stored in the environment.

12.5 Normalization, consistency, and total programs

The logical use of Curry–Howard depends on terms not conjuring inhabitants through nontermination. In a language where an infinite loop can be assigned every type, every proposition would become "provable" by divergence. Lean therefore separates safe total definitions from unsafe or partial computation.

Strong normalization is a central metatheoretic property of the pure calculi; Lean's implemented theory extends the core while preserving a small checkable proof language. The practical guarantee is enforced syntactically and semantically through checked declarations, positivity, universe consistency, and termination compilation.

12.6 Why Djinn does not search CoC directly

Full proof search for arbitrary dependent type theory is qualitatively different from propositional inhabitation. A searcher may need to invent:

- terms that appear inside types;
- motives for dependent eliminators;
- equality proofs and transports;
- universe instantiations;
- recursive definitions and termination proofs;
- instances, coercions, overloaded notation, and environment-dependent constants.

There is no simple finite LJT-style decision procedure for all of that. Leant's architecture is therefore a principled compromise: use Lean to understand and check the rich language; project an admitted structural fragment to search calculi with known operational properties; preserve unsupported dependent subterms as opaque, alpha-aware atoms; and fail closed on negative evidence.

The architecture follows the theory

Leant is not missing a magical parser flag that would make Djinn "support dependent types." The search calculus itself is propositional and its proof terms are untyped lambdas with tuple/sum operations. Supporting dependency analytically would require a different proof calculus, candidate representation, unifier, eliminator search, and completeness argument. Opaque cargo is a deliberate semantic boundary, not merely an unfinished pretty printer.

Chapter checkpoint

CoC unifies ordinary functions, polymorphism, type operators, and dependent families. CIC adds inductive families and recursors. Lean belongs to this family but has its own precise universe, proof-irrelevance, quotient, safety, and implementation choices. Djinn searches a much smaller propositional calculus, so Leant must mediate between two genuinely different logics.

Part III

How Lean Implements the Theory

From Surface Syntax to a Kernel Term

13.1 Lean source is intentionally incomplete

Human-friendly Lean code omits information that the kernel requires. Consider:

Compact surface syntax

```
fun x => (x, x)
```

The source does not state the type of x, the universe containing that type, the exact product constructor, or its implicit parameters. In a suitable expected context, Lean can infer all of them. Without an expected type it cannot, and it does not guess: `#check fun x => (x, x)` answers `fun x => (x, x) : ?m.1 → ?m.1 × ?m.1`, leaving the type of `x` as an unsolved metavariable `?m.1`, and turning the same text into a declaration is an error:

No expected type, no elaboration

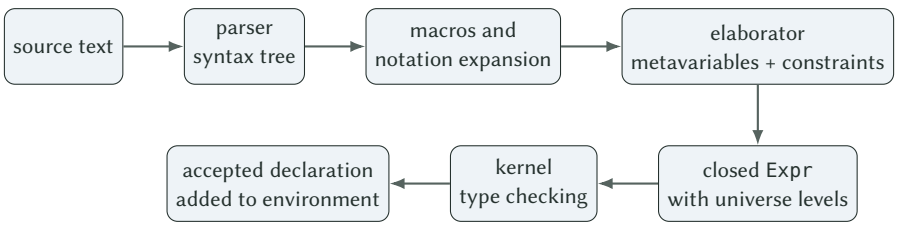
```
def pair := fun x => (x, x)
-- error: don't know how to synthesize implicit argument `β`
--   @Prod.mk ?m.5 ?m.5 x x
-- error: don't know how to synthesize implicit argument `α`
--   @Prod.mk ?m.5 ?m.5 x x
-- error: Failed to infer type of binder `x`
-- error: Failed to infer type of definition `pair`
```

(The numbering of `?m.5` varies between Lean versions.) The message already reveals the term Lean was building: an application of the constant `Prod.mk` to two type arguments, both still holes, and two copies of `x`. With an expected type every hole is filled: `#check (fun x => (x, x) : Nat → Nat × Nat)` under `set_option pp.all true` prints `fun (x : Nat) => @Prod.mk.{0, 0} Nat Nat x x`.

This gap is filled by *elaboration*: a constraint-generating and constraint-solving translation from syntax to fully explicit core expressions.

13.2 The pipeline

A simplified Lean command pipeline is:



Each box has a concrete output:

- *Parser*: a `Syntax` tree. The parser knows nothing about types, names, or scopes; `⟨p, q⟩` is just an "anonymous constructor" node.

- *Macros and notation*: another `Syntax` tree. Notations such as `a + b` or `[1, 2]` and user `macros` rewrite syntax to syntax. In the real implementation this is not a separate pass: the term elaborator's main loop (`elabTermAux` in `src/Lean/Elab/Term/TermElabM.lean`) first asks whether the current node has a macro, expands it if so, and otherwise dispatches to the elaborator registered for that node kind; expansion and elaboration are interleaved node by node.
- *Elaborator*: an `Expr` that may still mention metavariables, together with a metavariable context recording their assignments. `instantiateMVars` substitutes every assigned metavariable; any that remain unassigned are reported ("don't know how to synthesize …") and the declaration is rejected before the kernel ever sees it.
- *Between elaborator and kernel*: for a definition, recursion is compiled away, because the kernel has no notion of a recursive definition. Structural recursion becomes an application of the type's recursor (in practice through `brecOn`; `#print` of a structurally recursive `sumTo : Nat → Nat` shows `fun x => Nat.brecOn x sumTo._f` in Lean 4.33), well-founded recursion becomes `WellFounded.fix`, and each `match` becomes a call to an auxiliary matcher constant such as `isZero.match_1`, itself a separately checked declaration.
- *Kernel*: `addDecl` sends a `Declaration` to the kernel and receives either a new `Environment` or an error, printed with the prefix `(kernel)`. If the definition is computable, the code generator runs afterwards on its own copy of the term (Section 15.6).

Tactics, type-class resolution, coercion insertion, notation elaborators, and user metaprograms all operate before the final kernel check. They may be complicated and occasionally buggy without becoming proof authorities: their output is rejected unless it forms a valid kernel term.

13.3 Expected-type-directed elaboration

Elaboration often proceeds bidirectionally:

- *Synthesis mode*: infer a term's type from its syntax.
- *Checking mode*: elaborate a term against a known expected type.

A lambda with an omitted binder type is easy in checking mode:

Expected type supplies the binder

```
example : Nat → Nat := fun x => x
```

The expected Nat → Nat tells the elaborator that x : Nat and that the body must produce a natural number.

Anonymous constructor syntax is similarly expected-type dependent:

One syntax, several possible constructors

```
example (p : P) (q : Q) : P ∧ Q := ⟨p, q⟩
example (a : A) (b : B) : A × B := ⟨a, b⟩
```

The string `⟨p, q⟩` does not name `And.intro` or `Prod.mk`. The expected type decides. Under `set_option pp.all true`, `#check fun (P Q : Prop) (p : P) (q : Q) => (⟨p, q⟩ : P ∧ Q)` prints

The chosen constructor, fully explicit

```
fun (P Q : Prop) (p : P) (q : Q) => @And.intro P Q p q
  : ∀ (P Q : Prop) (p : P) (q : Q), And P Q
```

The angle brackets are gone; the kernel term is an application of the constant `And.intro` to two *explicit* propositions and two proofs. Even the notation `P ∧ Q` has become the application `And P Q`.

13.4 Unification and metavariable assignment

Suppose `id` has type `{α : Sort u} → α → α`. When elaborating `id true`, Lean creates a universe metavariable `?u` and a term metavariable `?α : Sort ?u`, elaborates the argument `true : Bool`, and unifies `?α` with `Bool`; because `Bool : Type = Sort 1`, the universe is solved as `?u := 1`. With `set_option pp.all true`, `#check id true` prints the fully solved term:

The solved application

```
@id.{1} Bool Bool.true : Bool
```

A numeral is a less direct example than it looks. In `id 5` the literal `5` is elaborated against the expected type `?α`, which produces a pending instance problem `OfNat ?α 5`; only the *default instance* for numerals resolves `?α := Nat`, and the elaborated term is `@id.{1} Nat (@OfNat.ofNat.{0} Nat (nat_lit 5) (instOfNatNat (nat_lit 5)))`. Both mechanisms—unification and default instances—run inside the elaborator; the kernel receives only the final term.

Dependent unification is more subtle because metavariables may occur inside types. Lean combines higher-order pattern unification, reduction, postponement, and specialized elaboration procedures rather than solving arbitrary higher-order unification, which is undecidable.

13.5 Implicit arguments and placeholders

Implicit parameters are inserted automatically when a constant is applied. A placeholder `_` asks Lean to create a metavariable explicitly. This is useful to Leant's renderer:

A reconstruction placeholder

```
provider _ x
```

The search engine may know that a provider should be applied but may not have retained a surface-level explicit type argument. Emitting `_` delegates that choice to the exact Lean environment. If inference is ambiguous or inconsistent, candidate verification fails.

13.6 Type classes are ordinary dependent parameters plus search

A class declaration defines a structure. An instance is a term of that structure type. A binder `[C α]` is an instance-implicit dependent function argument. The elaborator invokes type-class resolution to synthesize it.

Dictionary passing beneath surface syntax

```
class Render (α : Type u) where
  render : α → String

def showOne {α : Type u} [Render α] (x : α) : String :=
  Render.render x
```

Conceptually, `showOne` receives a dictionary carrying the `render` operation. That dictionary is a real value and may contain computational content. Erasing it from a synthesis problem is therefore harmless only under carefully restricted conditions. The dictionary is visible as soon as the pretty printer stops hiding it. `#print showOne` under `set_option pp.all true` gives

The instance is an ordinary argument

```
def showOne.{u} : {α : Type u} → [Render.{u} α] → (x : α) → String :=
fun {α : Type u} [inst : Render.{u} α] (x : α) => @Render.render.{u} α inst x
```

`Render.render` is a projection out of the structure `Render α`, and `inst` is passed to it exactly like `α` and `x`. Type-class resolution is the elaborator procedure that finds a value for that binder; the kernel sees only the resulting term.

Hidden dictionaries poison negative evidence

A generated candidate can often leave instance arguments implicit and let Lean resolve them. But a claim that no inhabitant exists cannot ignore a hidden instance premise: the dictionary may provide exactly the operation or value needed to construct the result. Leant records instance binders and refuses strong refutation when their evidence was not represented exhaustively.

13.7 Coercions and overloaded notation

Numerals, arithmetic operators, field notation, sequence literals, and many other surface forms are overloaded. Their elaboration depends on expected types and instances. A numeral `0` may elaborate as a natural number, integer, rational, finite-ring element, or user-defined type through `OfNat`.

This is another reason candidate verification must compile the exact source text in context. A Haskell renderer cannot decide the meaning of every Lean notation independently of imports and instances.

13.8 Auto-implicit names

With `autoImplicit true`, an undeclared identifier in a declaration header may be introduced as an implicit variable when Lean can infer an appropriate kind. The option defaults to `true` in core Lean (`src/Lean/Elab/AutoBound.lean`), but large projects such as Mathlib turn it off, so a session cannot assume either setting. Leant's verification command therefore sets it explicitly; the exact text sent to Lean is

Leant's candidate-verification command

```
set_option autoImplicit true in noncomputable example : (GOAL) := (TERM)
```

so that a goal such as `List α → List α` elaborates with `α` auto-bound whether or not the user's file enables the option.

The option affects elaboration, not kernel semantics. The resulting declaration still contains explicit binders and universe parameters in its core type.

Elaboration success is stronger than parsing, weaker than behavioral correctness

A candidate that parses may still fail to resolve names, infer parameters, solve instances, or satisfy the expected type. A candidate that elaborates and kernel-checks is a genuine inhabitant of the goal. It may nevertheless be the wrong inhabitant for the user's unstated behavioral intent.

Chapter checkpoint

Lean source relies on expected types, metavariables, unification, implicit arguments, type-class search, coercions, and notation elaborators. The output is a fully explicit kernel expression. Leant intentionally emits syntax that Lean must elaborate, because that is the only reliable way to recover exact environment-dependent details.

Chapter 14

The Kernel Representation: `Level`, `Expr`, and `Declaration`

14.1 Universe levels

The datatype `Lean.Level` represents universe expressions:

Constructor	Meaning	Typical source form
`zero`	level 0	`Prop`
`succ u`	successor	`Type u` uses `succ u`
`max u v`	maximum	universe of data-valued combinations
`imax u v`	impredicative maximum	universe of a dependent product
`param n`	declaration universe parameter	`.{u}`
`mvar id`	unsolved elaboration level	temporary only

Level expressions are normalized and compared modulo their algebraic meaning. A closed kernel declaration may mention named parameters but not unresolved level metavariables.

14.2 The expression datatype

At the pinned Lean revision, the core expression datatype has the following constructors:

Constructor	Role	Important detail
`bvar`	bound variable	de Bruijn index into surrounding binders
`fvar`	free variable	stable identifier backed by a local-context declaration
`mvar`	metavariable	elaboration hole; a declaration containing one is rejected by the kernel before checking (`(kernel) declaration has metavariables`)
`sort`	universe term	`Prop`, `Type u`, and general `Sort u`
`const`	global constant	exact name plus universe arguments
`app`	application	applications are left-nested binary nodes
`lam`	lambda abstraction	name hint, domain, body, binder information
`forallE`	dependent product	represents both Π and non-dependent arrow
`letE`	local definition	declared type, value, body, and nondependency flag
`lit`	literal	natural or string literal with kernel reduction support
`mdata`	metadata wrapper	definitionally equal to the wrapped expression
`proj`	structure projection	type name, field index, and structure expression

This is strikingly small. There is no kernel node for `if`, `match`, `structure`, theorem tactics, notation, list syntax, or type classes. Those are elaborated into applications of constants, lambdas, Π-types, recursors, and projections.

A worked example: from surface text to nodes

The definition `def pairNat : Nat → Nat × Nat := fun x => (x, x)` is stored in the environment as a value that can be printed with `repr`:

Printing the stored kernel term

```
import Lean
open Lean Meta

def pairNat : Nat → Nat × Nat := fun x => (x, x)

#eval show MetaM Unit from do
  IO.println (repr (← getConstInfo ``pairNat).value!)
-- Lean.Expr.lam
--   `x
--   (Lean.Expr.const `Nat [])
--   (Lean.Expr.app
--     (Lean.Expr.app
--       (Lean.Expr.app
--         (Lean.Expr.app (Lean.Expr.const `Prod.mk [Lean.Level.zero, Lean.Level.zero])
--                        (Lean.Expr.const `Nat []))
--         (Lean.Expr.const `Nat []))
--       (Lean.Expr.bvar 0))
--     (Lean.Expr.bvar 0))
--   (Lean.BinderInfo.default)
```

Every fact the surface text omitted is now explicit: the binder type `Nat`, the constant `Prod.mk` with its two universe arguments (both `zero`, because `Nat : Type = Sort 1 = Sort (succ zero)` and `Prod` takes the levels below the `succ`), the two implicit type arguments, and the bound variable `x` as `bvar 0` twice. The name `‘x` on the `lam` node is only a hint for printing.

The following table lists where the common surface forms end up (all entries checked with `pp.all` or `repr` in Lean 4.33):

Surface form	Kernel form
`fun x => e`	`lam` with the binder type filled in
`(x : A) → B`, `∀ x, B`, `A → B`	`forallE`; the arrow is the case where the body has no `bvar 0`
`let x := v; e`	`letE`
`x` (bound)	`bvar` k, k = number of binders between use and binding
`Nat`, `Prod.mk`	`const` with an explicit universe list
`Prop`, `Type`, `Type u`	`sort zero`, `sort (succ zero)`, `sort (succ (param u))`
`f a b`	`app (app f a) b`
`5`	`apps` of `OfNat.ofNat` to `Nat`, `lit (natVal 5)`, and an instance
`if c then a else b`	application of the constant `ite` to the proposition, a `Decidable` instance, and both branches
`p.1`	application of the projection function `Prod.fst`, whose own body is the node `proj ‘Prod 0`
`match x with …`	application of an auxiliary matcher constant (`isZero.match_1`)
`recursive def`	`brecOn`/recursor or `WellFounded.fix` application; no self-reference remains
`by tac`	whatever term the tactic assigned to the goal metavariable
`[C α] binder`	`forallE` with `BinderInfo.instImplicit`; the instance is an ordinary argument

14.3 One constructor for arrows and quantifiers

The expression

A non-dependent arrow

```
Nat → Bool
```

and the expression

A dependent universal

```
(n : Nat) → Fin (n + 1)
```

both elaborate to `forallE`. The body of the first does not contain the bound variable; the body of the second does. Surface `∀` is also the same node. The stored terms confirm this. For `Nat → Bool` the value is

A non-dependent forallE

```
Lean.Expr.forallE (Lean.Name.mkNum `a._@._internal._hyg 0)
  (Lean.Expr.const `Nat []) (Lean.Expr.const `Bool []) (Lean.BinderInfo.default)
```

with an automatically generated binder name and no `bvar` in the body (`Expr.isArrow` returns `true` for it), while `(n : Nat) → Fin (n + 1)` is a `forallE` named `‘n` whose body is `Fin` applied to an `HAdd.hAdd` application containing `bvar 0`. There is no separate arrow node to inspect; dependency is a property of the body.

This single fact explains much of Leant's serializer complexity: it must inspect domain sorts, binder visibility, body dependency, application heads, and local variable roles to classify a Lean Π into Djex-compatible type quantification, value implication, opaque dependency, or instance evidence.

14.4 Applications are spines

The term `f a b c` is represented as

$$(((f\ a)\ b)\ c).$$

The utilities `Expr.getAppFn` and `Expr.getAppArgs` recover the head and the argument array. For the elaborated numeral `(5 : Nat)`, whose kernel form is `@OfNat.ofNat.{0} Nat (nat_lit 5) (instOfNatNat (nat_lit 5))`, they return the head `Lean.Expr.const ‘OfNat.ofNat [Lean.Level.zero]` and the three arguments `#[Nat, lit (natVal 5), instOfNatNat (lit (natVal 5))]`: the numeral *itself* is a raw `lit` buried two levels down. Leant uses this view to recognize named logical families such as `And P Q`, nominal type-constructor applications, class heads, and inductive family applications.

A dependent application cannot be classified solely by its printed head. Its arguments may be types, values, indices, proofs, or universe-polymorphic terms. The serializer asks Lean for each argument's type and whether the whole application is itself type-valued.

14.5 Bound and free variables

Inside stored declarations, lambda and Π bodies use de Bruijn bound variables. Metaprograms often open a binder by creating a fresh free variable, adding its declaration to the local context, and instantiating the body. This makes its type and binder metadata available through ordinary APIs.

Leant's generated serializer follows that pattern when traversing dependent foralls and provider schemes. The emitted fragment uses explicit string identities, but those strings are allocated under alpha-aware discipline rather than copied naïvely from pretty-printed binder names.

14.6 Declarations and environments

The global environment maps names to checked declaration records. What is *sent* to the kernel is a `Lean.Declaration`; what is *stored* afterwards is a `Lean.ConstantInfo`. The seven `Declaration` constructors are:

- `axiomDecl`: a name, universe parameters, and a type; no value is checked;
- `defnDecl`: additionally a value, `ReducibilityHints` (`opaque`, `abbrev`, or `regular h` with a definitional height `h`), and a `DefinitionSafety` (`safe`, `unsafe`, or `partial`);
- `thmDecl`: a value whose type must be a proposition (the kernel error `thmTypeIsNotProp` exists for exactly this);
- `opaqueDecl`: a value that is checked once and never unfolded again by the kernel;
- `quotDecl`: installs the quotient primitives `Quot`, `Quot.mk`, `Quot.lift`, `Quot.ind`;
- `mutualDefnDecl`: a block of definitions that must all be `unsafe` or `partial`;
- `inductDecl`: universe parameters, the number of parameters, and a list of inductive types with their constructors; the kernel generates the recursors.

After checking, the environment holds a `ConstantInfo`: `axiomInfo`, `defnInfo`, `thmInfo`, `opaqueInfo`, `quotInfo`, `inductInfo`, `ctorInfo`, or `recInfo`. A constant occurrence in a term (`Expr.const`) stores only its name and universe arguments. The environment supplies everything else: the type; the value for definitions, theorems, and opaque constants; the reducibility hints and safety flag; and, for inductive types, constructors, and recursors, their structural metadata. Note that `ReducibilityHints` are a kernel-side ordering heuristic for lazy unfolding and are unrelated to the `@[reducible]`/`@[irreducible]` attributes, which only guide the elaborator and tactics (docstring in `src/Lean/Declaration.lean`).

This environment dependence is why a bare goal string is not a complete synthesis problem. The same spelling can resolve differently under namespaces and imports; constructors and providers available in one session may not exist in another; instances alter elaboration; reducible definitions change visible structure.

14.7 Inductive metadata

For an inductive family, the environment records (in `InductiveVal`) the number of parameters and indices, the constructor names, whether the type is recursive, reflexive, or nested, and the other types of its mutual block; each constructor (`ConstructorVal`) records its index, its parameter count, and its field count; each recursor (`RecursorVal`) records the numbers of motives and minor premises, its computation rules, and whether it supports K-like reduction. A short `MetaM` query shows the values:

Inductive metadata read from the live environment

```
open Lean Meta in
#eval show MetaM Unit from do
  let .inductInfo v ← getConstInfo ``Eq | throwError "not inductive"
  IO.println s!"numParams={v.numParams} numIndices={v.numIndices} ctors={v.ctors} isRec={v.isRec}"
-- numParams=2 numIndices=1 ctors=[Eq.refl] isRec=false
--   (List: numParams=1 numIndices=0 ctors=[List.nil, List.cons] isRec=true)
--   (Nat.rec: numParams=0 numMotives=1 numMinors=2 k=false)
```

Leant queries this live metadata instead of trying to infer constructors from names or printed declarations.

The distinction among parameter, index, and dependent constructor field is semantic:

- parameters may be carried uniformly into an expanded nominal family;
- indices generally force the family to remain opaque;
- non-dependent explicit fields may become product components;

- recursive occurrences require bounded and inventory-aware handling;
- hidden or dependent fields prevent an exact first-order sum-of-products translation.

Source trail

The exact expression constructors and locally nameless design are documented in https://github.com/leanprover/lean4/blob/79c4cd09fdab845aa6066bcbb09a663876601027/src/Lean/Expr.lean. Universe levels are in `src/Lean/Level.lean`; declaration records and inductive metadata are in `src/Lean/Declaration.lean`. Kernel inference and definitional equality are in `src/kernel/type_checker.cpp`.

Chapter checkpoint

Lean's trusted term language is small: levels, variables, sorts, constants, applications, lambdas, dependent products, lets, literals, metadata, and projections. Rich surface constructs become these nodes plus environment declarations. Leant translates from this exact semantic representation, not from pretty-printed Lean text.

Chapter 15

Trust, Tactics, Metaprograms, and Execution

15.1 The small-kernel architecture

Lean's trust story separates *finding* a proof from *checking* it. The kernel is responsible for:

- checking universe levels and declaration parameters;
- inferring expression types;
- verifying applications and dependent substitution;
- checking definitional equality;
- validating inductive declarations and recursors;
- enforcing safety boundaries on declarations.

Equally important is what the kernel does *not* do: it does not parse, expand macros, unify, insert implicit arguments or coercions, search for instances, run tactics, or generate code. It receives a closed `Declaration` and either accepts it or reports an error prefixed with `(kernel)`.

The parser, macro system, elaborator, tactic framework, simplifier, compiler, and external automation may all be larger. They are proof-producing or term-producing infrastructure. A bug can cause failure, poor diagnostics, or a rejected term; it should not make an invalid term kernel-valid. One can watch the boundary directly by bypassing the elaborator and handing the kernel a hand-built declaration through `Lean.addDecl`:

The kernel checks what it is given, not what was intended

```
import Lean
open Lean

-- A ”theorem” whose body proves 0 = 0 but whose stated type is 1 = 0.
#eval show CoreM Unit from do
  let natTy := mkConst ``Nat
  addDecl <| .thmDecl {
    name := `bogus, levelParams := [],
    type  := mkApp3 (mkConst ``Eq [.succ .zero]) natTy (mkNatLit 1) (mkNatLit 0),
    value := mkApp2 (mkConst ``Eq.refl [.succ .zero]) natTy (mkNatLit 0) }
-- error: (kernel) declaration type mismatch, 'bogus' has type
--   0 = 0
-- but it is expected to have type
--   1 = 0

-- A declaration containing a metavariable.
#eval show CoreM Unit from do
  addDecl <| .defnDecl {
    name := `hole, levelParams := [], type := mkConst ``Nat,
    value := mkMVar { name := `m }, hints := .abbrev, safety := .safe }
-- error: (kernel) declaration has metavariables 'hole'

-- The same route with a correct body succeeds; `fine` is now a constant.
#eval show CoreM Unit from do
  let natTy := mkConst ``Nat
  addDecl <| .thmDecl {
    name := `fine, levelParams := [],
    type  := mkApp3 (mkConst ``Eq [.succ .zero]) natTy (mkNatLit 0) (mkNatLit 0),
    value := mkApp2 (mkConst ``Eq.refl [.succ .zero]) natTy (mkNatLit 0) }
#print fine
-- theorem fine : 0 = 0 :=
-- Eq.refl 0
```

Nothing in the elaborator was consulted for these three declarations; the verdicts came from the kernel alone.

15.2 Tactics are metaprograms over goals

A tactic state contains metavariable goals. A tactic may:

- introduce a binder;
- apply a theorem and create subgoals;
- refine a goal with a partially specified expression;
- normalize or rewrite its target;
- invoke search or decision procedures;
- assign solved metavariables.

When no goals remain, the assigned expression is submitted through the ordinary checking path. Tactics do not add a second notion of proof; `#print` shows the term the tactic block left behind, and that term—not the script—is what the kernel checked:

A tactic script and the term it produced

```
theorem both : True ∧ (∀ n : Nat, n = n) := by
  constructor
  · trivial
  · intro n
    rfl
#print both
-- theorem both : True ∧ ∀ (n : Nat), n = n :=
-- (True.intro, fun n => Eq.refl n)

theorem sq (n : Nat) : n * n = n * n := by simp
#print sq
-- theorem sq : ∀ (n : Nat), n * n = n * n :=
-- fun n => of_eq_true (eq_self (n * n))
```

The second example is typical of automation: `simp` performed a rewriting search, but what it recorded is an ordinary application of two library lemmas.

Leant's `:prove` command can be understood at this level: it interacts with a live proof state, asks its synthesis machinery for suitable terms at selected goals, and returns suggestions that Lean checks in context.

15.3 Metaprogramming inside Lean

Lean exposes APIs for expressions, local contexts, environments, declaration lookup, weak-head normalization, unification, instance synthesis, and syntax construction. Leant uses generated Lean metaprograms to perform tasks that must respect the exact Lean semantics:

- elaborate the raw goal;
- inspect the resulting `Expr`;
- classify logical connectives and Π binders;
- inspect inductive declarations and constructor fields;
- discover provider constants from the live environment;
- serialize a bounded semantic fragment to an S-expression.

The Haskell process does not link against Lean's internal objects directly. It sends commands to a persistent Lean REPL worker and receives structured JSON responses containing messages, environment identifiers, and results.

15.4 Axioms and opaque constants

An axiom is a trusted assumption because no body is checked. Core Lean declares exactly three:

The three standard axioms, as #print shows them

```
axiom propext : ∀ {a b : Prop}, (a ↔ b) → a = b
axiom Classical.choice.{u} : {α : Sort u} → Nonempty α → α
axiom Quot.sound.{u} : ∀ {α : Sort u} {r : α → α → Prop} {a b : α},
  r a b → Quot.mk r a = Quot.mk r b
```

The quotient type former itself is not an axiom but a kernel primitive: `#print Quot` answers `Quotient primitive Quot.{u} : {α : Sort u} → (α → α → Prop) → Sort u`, and `Quot.mk`, `Quot.lift`, and `Quot.ind` are installed by `quotDecl` with their computation rule built into the kernel; only `Quot.sound` is an assumption. A fourth constant, `sorryAx : (α : Sort u) → Bool → α`, is the axiom behind every `sorry`. Imported projects may add their own axioms.

An `opaque` constant is different: its body was checked when it was declared, but the kernel never unfolds it afterwards. A theorem is different again: its body is stored (`TheoremVal.value`) and the kernel may unfold it, but the elaborator's `whnf` and `isDefEq` do not unfold theorems except at transparency `all`, so in practice a theorem behaves as an opaque proof term. Axioms, opaque constants, and theorems all appear as `Expr.const` nodes downstream; only their provenance differs, and `#print axioms` follows the dependency graph to report which assumptions a constant ultimately rests on:

When each axiom appears

```
theorem twoTwo : 2 + 2 = 4 := rfl
#print axioms twoTwo
-- 'twoTwo' does not depend on any axioms

theorem iffEq (p q : Prop) (h : p ↔ q) : p = q := propext h
#print axioms iffEq
-- 'iffEq' depends on axioms: [propext]

theorem addZeroFun : (fun n : Nat => n + 0) = (fun n => n) := funext fun _ => rfl
#print axioms addZeroFun
-- 'addZeroFun' depends on axioms: [Quot.sound]

theorem lem (p : Prop) : p ∨ ¬p := Classical.em p
#print axioms lem
-- 'lem' depends on axioms: [propext, Classical.choice, Quot.sound]

theorem later (n : Nat) : n + 0 = n := sorry
-- warning: declaration uses `sorry`
#print axioms later
-- 'later' depends on axioms: [sorryAx]

theorem byOmega (a b : Nat) (h : a < b) : a + 1 ≤ b := by omega
#print axioms byOmega
-- 'byOmega' depends on axioms: [propext, Quot.sound]

theorem byDecide : 10 < 20 := by decide
#print axioms byDecide
-- 'byDecide' does not depend on any axioms
```

Function extensionality (`funext`) is proved from `Quot.sound`, which is why an extensionality proof pulls in the quotient axiom, and `Classical.em` is proved from all three (Diaconescu's argument). Automation such as `omega` routinely uses `propext`; `decide` produces a proof by evaluation that needs none. A `sorry` is not silent: it is a warning at declaration time and an axiom in `#print axioms`. Leant's REPL protocol reports the `sorries` of each command, and a candidate whose verification produced any is rejected rather than counted as a success.

15.5 Proof irrelevance and erasure

Terms whose types lie in `Prop` are computationally irrelevant, and the compiler erases them (together with types and type-class dictionaries that are only used at the type level). This permits specifications to carry proofs without runtime cost. The restriction on eliminating from `Prop` into data (Chapter 12) is not a consequence of erasure but a rule of the type theory: it is what makes erasure sound, since a program's result cannot depend on which proof was supplied.

The kernel still type checks proof arguments before erasure. A generated program cannot bypass a required proof merely because it will not exist at runtime.

15.6 Safe, unsafe, and partial definitions

Lean is also a practical programming language. It allows `unsafe` and `partial` definitions for executable code, but logical declarations are prevented from depending on unsafe computation as if it were a total proof-producing function. Each

definition carries a `DefinitionSafety` (`safe`, `unsafe`, or `partial`), and the kernel refuses to let a safe declaration mention an unsafe constant:

Safety is checked by the kernel

```
unsafe def peek (α : Type) (x : α) : α := x
def usePeek : Nat := peek Nat 3
-- error: (kernel) invalid declaration, it uses unsafe declaration 'peek'
```

A `partial` definition never reaches the kernel as a recursive definition at all. Lean elaborates the recursive body as an `unsafe` auxiliary and then declares the visible name as an `opaque` constant of the stated type, whose only logical content is that the type is inhabited (which is why `partial` requires an `Inhabited` or `Nonempty` instance for the result type):

What partial really declares

```
partial def spin (n : Nat) : Nat := spin (n + 1)
#print spin
-- opaque spin : Nat → Nat
#print spin._unsafe_rec
-- partial def spin._unsafe_rec : Nat → Nat :=
-- fun n => spin._unsafe_rec (n + 1)
#print axioms spin
-- 'spin' does not depend on any axioms

partial def spinE (n : Nat) : Empty := spinE n
-- error: failed to compile 'partial' definition `spinE`, could not prove that the type
--   Nat → Empty
-- is nonempty.
```

Consequently no theorem can be proved about `spin` by unfolding it; the kernel sees only an opaque constant. The compiled program, however, runs `spin._unsafe_rec`. The same split between "what the kernel knows" and "what the compiler runs" is exposed directly by `@[implemented_by]`, which tells the code generator to compile one definition using another's code while the kernel continues to reason about the original body:

implemented_by is trusted by the compiler, ignored by the kernel

```
def cheat (n : Nat) : Nat := n + 1000

@[implemented_by cheat]
def honest (n : Nat) : Nat := n

theorem honest_id (n : Nat) : honest n = n := rfl   -- accepted: kernel unfolds `honest`
#reduce honest 1
-- 1
#eval honest 1
-- 1001
```

An `implemented_by` or `extern` attribute is therefore part of the trusted base for *execution*, not for *proof*: the kernel's verdict about `honest` is unaffected by what runs at `#eval` time.

Finally, `noncomputable` concerns only the code generator. A definition that uses `Classical.choice` (or any other constant without executable code) type checks in the kernel but fails to compile:

noncomputable turns off compilation, nothing else

```
def choose (α : Type) [Nonempty α] : α := Classical.choice inferInstance
-- error: `Classical.choice` not supported by code generator;
--        consider marking definition as `noncomputable`

noncomputable def choose (α : Type) [Nonempty α] : α := Classical.choice inferInstance
#print axioms choose
-- 'choose' depends on axioms: [Classical.choice]
theorem choose_eq : choose Nat = choose Nat := rfl   -- fine: it is a kernel term
#eval choose Nat
-- error: failed to compile definition, consider marking it as 'noncomputable'
--        because it depends on 'choose', which is 'noncomputable'
```

Leant's candidate-verification command (`set_option autoImplicit true in noncomputable example : (GOAL) := (TERM)`) is marked `noncomputable` for exactly this reason: a candidate may use logically valid noncomputable constants without requesting executable code. This relaxes compilation, not elaboration, type checking, universe checking, or safety checking; the `peek` example above is still rejected by the kernel when it is wrapped in that command, and a candidate containing `sorry` still produces the `sorry` warning that Leant treats as rejection.

15.7 External solvers and replay

An external solver can be used in several trust arrangements:

1. trust the solver's yes/no answer directly;
2. demand a certificate and check it in Lean;
3. reconstruct a proof term and ask the kernel to check it;
4. use the solver only as a heuristic or counterexample finder, then independently replay the result.

Leant's ordinary Djex candidates use arrangement 3 at the final boundary: a Lean term is reconstructed and checked. The optional Length/Z3 layer uses arrangement 4 for its bounded behavioral claims: the solver status alone is not authority; models and positive results are interpreted under a sealed contract and independently replayed by Djex's semantic layer. Even then, the result is not automatically a general Lean theorem about extensional behavior.

Three different authorities

1. Lean's kernel authorizes "this term has this Lean type."
2. A complete Djinn search over an exact fragment can authorize "this translated formula has no proof."
3. A sealed behavioral checker can authorize only the specific bounded semantic verdict described by its contract and replay rules.

No one authority silently upgrades another.

Chapter checkpoint

Lean's automation constructs terms; the kernel checks them. Metaprograms can inspect the exact live environment without joining the trusted base. Noncomputability, opacity, axioms, unsafe code, and external solvers each have distinct trust meanings. Leant's design is strongest where it preserves those distinctions explicitly.

Part IV

Leant and Djex

Chapter 16

Djex's Deliberately Smaller Type World

16.1 Why an intermediate language exists

Lean's kernel language is compact, but its types are still fully dependent and environment-sensitive. Searching that language directly would require solving the general problems described in Chapter 12. Djex instead provides synthesis engines for a Haskell-like source type language and, beneath Djinn, a propositional proof calculus.

Leant therefore needs an intermediate semantic boundary. It cannot pass an arbitrary Lean `Expr` to Djex and hope that both sides mean the same thing. It first classifies the goal into a Leant-owned fragment, then the engine adapter lowers admitted pieces into Djex representations.

16.2 Djex's shared source type

The current shared datatype `Language.Haskell.Synthesis.Type.Type` has these principal forms:

Djex constructor	Meaning	Lean relationship
`TypeVariable`	type variable	a selected sort-bound Lean variable
`TypeConstructor`	nominal type constructor	a Lean constant retained by exact name mapping
`TypeApplication`	left-associated type application	only admitted type-level applications
`FunctionType`	non-dependent value arrow	a Lean Π whose body does not depend on the value binder
`TupleType`	structural boxed/unboxed tuple	product-like fragment
`ForallType`	type-variable binders, class context, body	selected Lean sort quantifiers and exact provider schemes

This language supports polymorphism, higher-kinded constructor applications, constraints, tuples, and ordinary functions. It has no constructor whose codomain is an arbitrary term-dependent type. A `FunctionType A B` cannot express $\Pi(x : A), B(x)$ when x appears in B.

The shared kind language is correspondingly Haskell-like:

$$\text{ProperTypeKind} \quad | \quad \text{KindVariable } k \quad | \quad \text{FunctionKind } k_1\ k_2.$$

A sealed checked environment uses ground kinds with no unresolved kind variables. Universe levels are not represented as Lean universes; the frontend must establish that the admitted source type has a compatible kind shape.

16.3 Flexible variables and rigid variables

Djex distinguishes inference variables that unification may instantiate from rigid skolems that must remain fixed:

- a *flexible variable* represents an unknown chosen by the search/unifier;
- a *rigid variable* represents a universally quantified type that search must treat parametrically.

Confusing the two would make polymorphic synthesis unsound. A target such as $\forall A\, B, A \to B$ must not be solved by unifying the rigid A and B.

Djex's shared layer also owns capture-avoiding substitution, binder freshening, alpha normalization, validation, constructor-application normalization, and explicit forall-spine processing. These are not cosmetic utilities: they enforce the source calculus on which both engines agree.

16.4 Djinn's propositional formula language

Djinn lowers types further into LJT formulas:

$$F ::= P \mid F \to F \mid \bigwedge_i F_i \mid \bigvee_i F_i \mid \bot_N.$$

The exact Haskell constructors are `PVar`, implication, n-ary `Conj`, constructor-tagged `Disj`, and nominal `Empty`. A formula atom can be an ordinary symbol or an `OpaqueTypeSymbol` carrying an exact shared source type plus an alpha-normal key.

The proof-term language contains:

- variables, lambdas, and applications;
- tuple construction and splitting;
- sum injection and case analysis;
- constructor descriptors carrying names and arities.

It is deliberately not a dependent proof-term language. Formula atoms have no eliminators except those supplied by assumptions whose formulas expose implications, products, or sums.

16.5 Opaque atoms preserve identity, not internal logic

Suppose Leant encounters the Lean proposition `∀ n : Nat, P n`. If the surrounding goal only needs to move this proposition from one product component to another, Djinn need not understand the quantifier. It can represent the whole type as one atom X and prove

$$X \wedge Q \to Q \wedge X.$$

The atom must still have stable semantic identity, and at the pinned revision that identity is established at two different levels. Leant's own atoms (`FAtom` in `src/Leant/Synth/Fragment.hs`) are keyed by the spelling that Lean's `Meta.ppExpr` produces for the sealed subterm; the serializer renames every sort binder it opens to `s0`, `s1`, … so that two occurrences of the same subformula under the same prefix print identically, and the engine adapter (`src/Leant/Synth/Engine.hs`, "Fragment -> Djinn type") turns each distinct key into one Djex type variable that search may transport but never analyze. Djex's own `OpaqueTypeSymbol` (in `djinn/src-core/Djinn/Internal/LJTFormula.hs`) is different: it is created when a *quantified Djex type* (a `ForallType` that Leant passed structurally, e.g. a rank-2 premise) must be treated as one LJT atom, and it stores the exact shared source tree together with an alpha-normal key (`alphaTypeKey`), so `forall a. a -> a` and `forall b. b -> b` are the same proposition regardless of binder spelling. Comparing arbitrary pretty-printed strings would be unsafe for the second kind of atom, which is why Djex does not do it; for the first kind the pretty-printed key is acceptable because it only ever has to recognize the same Lean subterm again inside one query.

Opaque means indivisible, not forgotten

An opaque atom retains enough identity to recognize the same type again and to reconstruct the original Lean spelling. Search may copy it, pass it to a premise, return it, or place it in a product. Search may not inspect its quantifiers, indices, equalities, or constructors.

16.6 The revision boundary

The Leant snapshot studied here vendors Djex commit `7d65eba6f753` as a submodule. The standalone Djex repository was also inspected at `0cf82f2aa22d` to understand the current shared architecture and documentation. Statements about Leant's actual executable behavior should be read against the vendored revision; later standalone changes do not automatically enter Leant until the submodule pointer advances.

Source trail

The shared type and kind representations are in https://github.com/VladimirReshetnikov/Djex/blob/0cf82f2aa22debb1de2c7aaa13a2abbe190550d8/synthesis/src/Language/Haskell/Synthesis/Type.hs and `synthesis/src/Language/Haskell/Synthesis/Kind.hs`. Djinn formulas and proof terms are in `djinn/src-core/Djinn/Internal/LJTFormula.hs`.

Chapter checkpoint

Djex does not contain Lean's dependent type theory. Its shared source language is polymorphic and Haskell-like; Djinn's core is propositional. Opaque atoms preserve exact alpha-aware identity while withholding internal structure. Leant must prove, query by query, which Lean structure can safely cross this boundary.

Chapter 17

The End-to-End Leant Pipeline

17.1 A GHCi-shaped frontend for a Lean worker

Leant is a Haskell executable that presents a persistent interactive session. It feels like a REPL, but the authoritative Lean state lives in a separate Lean process. Leant launches the community Lean REPL through a Lake environment and communicates over a blank-line-delimited JSON protocol.

The Haskell state tracks, among other things:

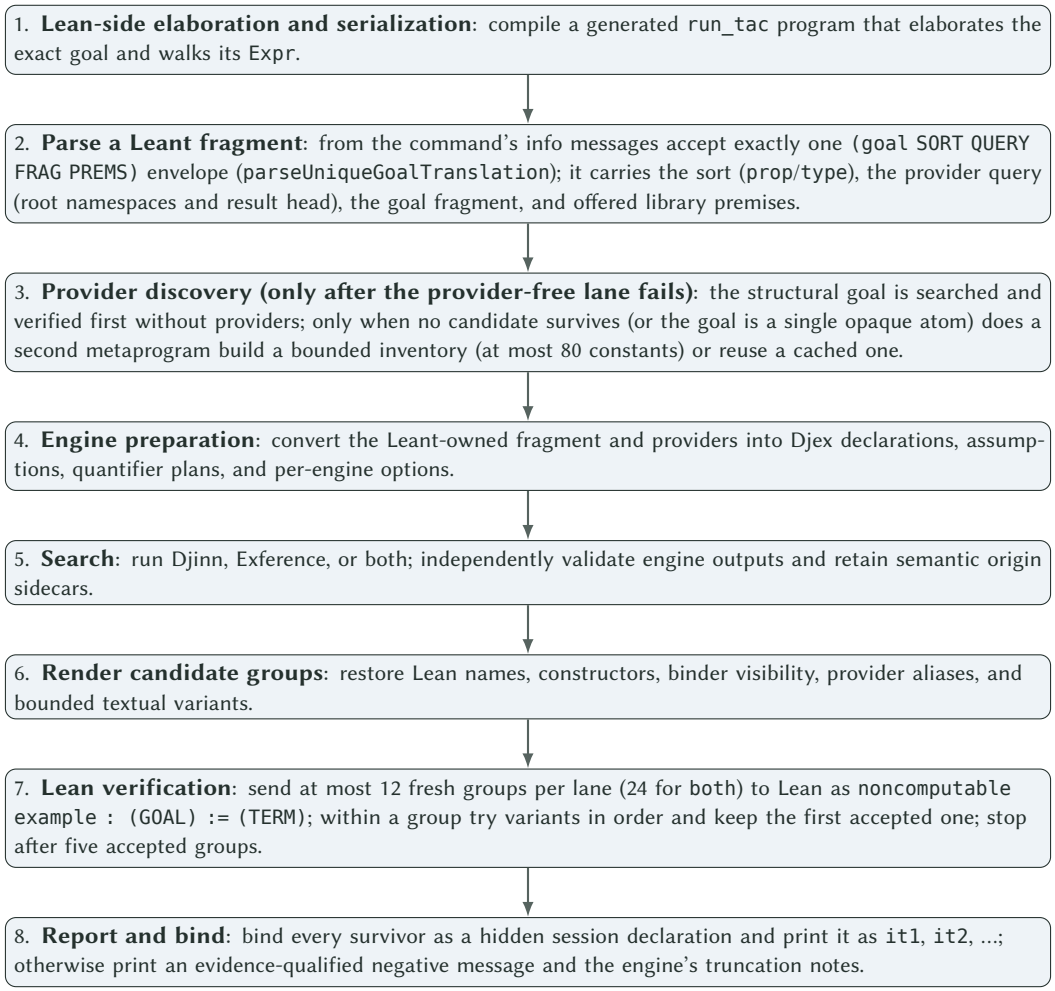
- the current Lean environment identifier and the undo stack of earlier identifiers (`rsEnvStack`);
- the replayable command history, the import list, and any snapshot base loaded by `:unpickle`;
- the synthesized `it`, `it1`, `it2`, … bindings (`rsItCounter`, `rsSynthIts`);
- the engine selection `rsSynthEngine` (`:set synth-engine`) and Exference's step budget `rsSynthSteps` (`:set synth-steps`); the queue bound, the candidate window, the verification frontier, and the five-result display quota are compiled-in constants, not settings;
- the cached synthesis side environment (`rsSynthBase`, `rsSynthEnv`), the provider world generation and the twelve-entry provider cache (`rsProviderWorld`, `rsProviderCache`);
- the `synth-library` and `synth-classical` toggles, the rated library inventory, and the startup-only Length assessment mode;
- proof-mode state (`rsProve`) and the last `sorry` goal that a bare `:synth` may target.

(All of these live in the `ReplState` record near the top of `src/Main.hs`.)

The persistent worker matters because a synthesis query is evaluated in the live environment created by previous commands and imports, not in a freshly guessed module.

17.2 The major stages

A representative `:synth GOAL` request follows this path:



17.3 Why translation happens inside Lean

The serializer is emitted as Lean source and executed in the live worker. It can therefore:

- elaborate the user's exact text with Lean's parser, macros, notation, imports, and options;
- instantiate metavariables and weak-head normalize under Lean's definitions;
- inspect `Expr.forallE` dependency and binder information;
- resolve constant names and universe instantiations;
- inspect inductive declarations and constructor types;
- distinguish classes, propositions, data sorts, local variables, and nominal applications;
- use Lean's definitional equality when comparing provider assignments.

Attempting these tasks by parsing pretty-printed goals in Haskell would immediately lose binding identity, namespace resolution, implicit arguments, reducible definitions, and universe information.

17.4 The serializer output is treated as an untrusted protocol

The generated Lean program emits a tagged S-expression inside an info message. User code can also emit messages, so Leant does not trust the first line with a familiar prefix. The parser requires exactly one well-formed goal envelope and

rejects missing, duplicate, malformed, or trailing data.

This is a small but representative security principle: a semantic stage hands structured data to the next stage through a narrow checked boundary. Messages, strings, and debug output never become authority merely because they resemble internal protocol text.

17.5 Semantic origin survives textual operations

A candidate is not only a string. Leant records the exact source goal, search goal, declarations, providers, constructor/name maps, premise layout, engine preparation, search policy, and typed candidate or proof provenance needed to interpret it.

Rendering can produce several strings from one candidate. Textual deduplication can merge equal display forms. Verification can accept only one variant. Through those operations, the semantic origin sidecar must remain attached to the group that owns it. The internal documentation calls this a semantic-origin record; typed candidates additionally carry graph fingerprints and sealed inventories.

Authority does not transfer by string equality

Two identical Lean strings generated from different environments or candidate graphs are not automatically the same semantic artifact. A copied string has no right to inherit a completeness proof, behavioral contract, provider inventory, or solver receipt from another candidate merely because the characters match.

17.6 Backend verification usually dominates latency

Structural search can be fast, especially for small formulas. Candidate verification invokes Lean elaboration in the live environment for each variant and can dominate wall-clock time. Leant therefore groups variants, uses lazy quota-aware verification, caches stable provider inventories, and avoids counting failed variants against the accepted-result quota.

17.7 One goal, every stage

The first query of the golden transcript `test/synth-basic.golden` is small enough to follow through every stage with the exact intermediate forms the pinned code produces.

Transcript

```
λ> :synth (A × (B ⊕ C) → (A × B) ⊕ (A × C))
preparing synthesis environment (session imports + Lean)...
  it1  fun ⟨x, y⟩ => match y with | .inl z => .inl ⟨x, z⟩ | .inr w => .inr ⟨x, w⟩
```

Stage 1, side environment (`src/Main.hs`, `ensureSynthEnv`). The first `:synth` of a session compiles `synthPrelude` (the `LeantSynth` namespace with `go`, `indOf`, `recOf`, `appOf`, the provider scanner, and the rated library table) on top of the session imports plus `import Lean`, then replays the session history onto it; that is the "preparing synthesis environment" line.

Stage 2, serialization (`serializerProgram`). Lean runs

Generated by serializerProgram

```
open Lean Meta Elab Tactic in
set_option autoImplicit true in
set_option linter.unusedVariables false in
example : (A × (B ⊕ C) → (A × B) ⊕ (A × C)) := by
  run_tac withMainContext do
    let tgt ← getMainTarget
    ...  -- LeantSynth.go false false 100 0 [] tgt, premSpine, roots, head
    logInfo ("(goal " ++ ... ++ ")")
  sorry
```

`A`, `B`, `C` are auto-bound locals. `go` sees a non-dependent explicit `forallE` and emits `(-> D R)`; the domain `Prod A (Sum B C)` matches the two-argument `Prod` case and becomes `(lean-prod (var "A") (sum (var "B") (var "C")))`; the codomain becomes `(sum (lean-prod (var "A") (var "B")) (lean-prod (var "A") (var "C")))`. `Meta.isProp` is false, the used constants are `Prod` and `Sum`, the peeled result head is `Sum`, and no recursive inductive occurs, so no library premise is offered. Up to whitespace and root order the info message is

Goal envelope

```
(goal type (query (roots "Prod" "Sum") (head "Sum"))
  (-> (lean-prod (var "A") (sum (var "B") (var "C")))
      (sum (lean-prod (var "A") (var "B")) (lean-prod (var "A") (var "C")))))
```

Stage 3, parsing (`parseUniqueGoalTranslation`, `parseGoalSexp`). Exactly one info message begins with `(goal` followed by a space; it parses to `GoalType`, the canonical query `ProviderQuery ["Prod","Sum"] (Just "Sum")`, and the fragment that `LEANT_SYNTH_DEBUG=1` would print as

debug fragment

```
FArr (FLeanProd (FVar "A") (FSum (FVar "B") (FVar "C")))
     (FSum (FLeanProd (FVar "A") (FVar "B")) (FLeanProd (FVar "A") (FVar "C")))
```

`fragRefusal` is `Nothing` (the root is an arrow, not an atom), `fragUnsafeAtoms` is empty, `fragHasDepth` is false: this goal is eligible for a sound refutation should search fail.

Stage 4, engine preparation (`src/Leant/Synth/Engine.hs`, `prepareSynthesis`/`fragToDjinn`). `FVar` becomes a Djex `TypeVariable`, `FLeanProd` a boxed `TupleType` of two components, `FSum` an application of the shared `Either` constructor, `FArr` a `FunctionType`; in Haskell notation the search goal is `(A, Either B C) -> Either (A, B) (A, C)`, with `A`, `B`, `C` as free, implicitly universally quantified type variables. No declarations, providers, or premises are added. The `PreparedSemanticOrigin` records the source goal, the search goal, the empty declaration list, the constructor/type maps, and the premise layout (one source arrow, no premises) for the renderer.

Stage 5, search (`djinnRun`). Djinn lowers the type to the LJT formula $(A \wedge (B \vee C)) \to ((A \wedge B) \vee (A \wedge C))$ over atoms `A`, `B`, `C`, proves it by a right implication rule, a left conjunction rule, and a left disjunction rule, checks the proof term, and returns `ValidatedCandidates` with one candidate; the run is unbudgeted with cutoff 60.

Stage 6, rendering (`src/Leant/Synth/Render.hs`). The proof term's split becomes an anonymous-constructor pattern `⟨x, y⟩`, the case analysis a `match` with dot-constructor alternatives, and the injections `.inl`/`.inr` with anonymous constructors for the pairs; because the goal has no explicit quantifier slots and no quantified hypotheses, the group has a single variant.

Stage 7, verification (`synthVerify`, `verifyCandidateGroups`). Lean receives

Generated by candidateVerificationProgram

```
set_option autoImplicit true in noncomputable example : (A × (B ⊕ C) → (A × B) ⊕ (A × C)) := (fun ⟨x, y⟩
 ↪  => match y with | .inl z => .inl ⟨x, z⟩ | .inr w => .inr ⟨x, w⟩)
```

Lean answers with no error and no `sorry`, so the variant is `VariantAccepted`; the group consumes one of the five success slots.

Stage 8, binding and display (`finalizeSynthLanePresentations`, `synthBind`). Lean receives

Generated by synthBind

```
set_option autoImplicit true in def «it!1» : (A × (B ⊕ C) → (A × B) ⊕ (A × C)) := (fun ⟨x, y⟩ => match y
 ↪  with | .inl z => .inl ⟨x, z⟩ | .inr w => .inr ⟨x, w⟩)
```

The environment identifier advances, and the row `it1 fun ⟨x, y⟩ => ...` is printed. Because the baseline lane produced a survivor, provider discovery never runs for this query.

17.8 Stage, owner, key type, failure mode

Stage	Module	Key function or type	What can fail, and how it surfaces

side environment	`src/Main.hs`	`ensureSynthBase`
		`ensureSynthEnv`
		`rsSynthBase`, `rsSynthEnv`
			prelude does not compile over the imports (snapshot without `Lean` metaprogramming): `:synth` reports the compile error

elaborate and serialize	`src/Leant/Synth/Fragment.hs`	`serializerProgram` generated Lean, `(goal ...)`	the goal does not elaborate: "cannot translate this goal:" with Lean's errors; stuck universes trigger the auto-`Type` retry
parse	`Fragment.hs`	`parseUniqueGoalTranslation` `ParsedGoal`, `Frag`	zero or several envelopes, malformed or trailing tokens: "goal translation ..." error
refusal check	`Fragment.hs`	`fragRefusal` `fragProviderMayOpen`	single opaque atom/application or depth marker: "out of fragment: ..." unless a provider may open it

provider discovery	`Fragment.hs`, `Main.hs`, `ProviderCache.hs`	`providerProgram`
		`loadSynthProviders`
		`ProviderQuery`, `[ProviderFrag]`
			discovery error degrades silently to an empty inventory (visible with `LEANT_SYNTH_DEBUG`)

translation	`src/Leant/Synth/Engine.hs`	`prepareSynthesis`
		`PreparedSemanticOrigin`
		Djex `Type`, `Declaration`
			inconsistent family schemas or provider kinds: "synthesis engine error: ..." or a dropped provider

search	`Engine.hs`; Djex `Djinn/Core.hs`, `Exference/Core`	`djinnRun`, `exferenceRun`
		`QueryEvidence`, `Progress`
		`DetailedSynthOutcome`
			truncation notes; timeout "the engine did not finish within Ns"

render	`src/Leant/Synth/Render.hs`	`renderLeanTerm`
		`renderLeanTermGraphProjection`
		`[String]` variants per group
			unsupported node or ambiguous instantiation: group dropped, counted in debug metrics

verify	`src/Leant/Synth/Verification.hs`, `Main.hs`	`verifyCandidateGroups`
		`synthVerify`
		`VariantVerdict`, `Verified`
			every variant rejected: "the engine proposed N candidate(s) but none survived Lean verification"

bind and report	`Main.hs`	`finalizeSynthLaneAccumulation`
		`synthBind`
		«`it!N`» declarations
			a survivor that verifies but will not bind is shown as its raw term

17.9 Fixed bounds at the pinned revision

Bound	Value	Where
serializer fuel (goal / provider scheme)	100 / 80	`serializerProgram`, `providerProgram`
providers serialized per query	80	`providerProgram`
Djinn provider prefixes	1, 4, 16, all	`providerStages`
provider cache entries	12	`providerCacheCapacity`
type binders / instance heads / assignments per provider	6 / 32 / 16	`providerInstantiationAssignments` Djex `maximumProviderInstantiationArguments`
kind arity of a provider argument	64	`maximumProviderArgumentKindArity`
Djinn candidate window	60	`candidateWindow`
Djinn choice-point budget (library, excluded-middle lanes)	100 000	`src/Main.hs`
library premises kept	8	`selectLibraryPremises`
excluded-middle atoms tolerated	5	`synthClassical`
Exference steps (default) / queue	4096 / 1024	`rsSynthSteps`, `exferenceRun`
fresh groups sent to Lean per lane	12 (24 for `both`)	`synthMaxTried`, `synthVerificationWindow`
accepted groups shown and bound	5	`synthMaxShown`
wall-clock deadline	20 s (`LEANT_SYNTH_TIMEOUT`)	`synthTimeoutSeconds`

Source trail

The process protocol and lifetime management are in https://github.com/VladimirReshetnikov/Leant/blob/49c8f9f100a644b5a32b3c942bc6565a3a3ba6a0/src/Leant/Backend.hs. The interactive orchestration is in `src/Main.hs`. Translation and verification command generation are in `src/Leant/Synth/Fragment.hs`; internal provenance invariants are summarized in `docs/synth-internals.md`.

Chapter checkpoint

Leant is a persistent Haskell controller around an authoritative Lean worker. The goal is elaborated and classified inside Lean, searched in Djex, reconstructed in Haskell, and checked again inside Lean. Structured origins and exact environment identities survive every intermediate transformation.

The Leant Synthesis Fragment

18.1 A frontend-owned semantic tree

Leant does not translate Lean expressions directly into Djex types. It first constructs its own `Frag` tree. This keeps Lean-specific policy out of Djex and makes the exact/opaque/completeness decisions inspectable before an engine is chosen.

The principal fragment forms at the pinned revision are summarized below.

Fragment	Meaning	Typical Lean source
`FArr`	implication/function arrow	non-dependent `A → B`
`FProd`	logical product	`And` and `PProd`, both serialized as `(prod ...)`
`FLeanProd`	Lean data product	a saturated canonical `Prod` (after reducible normalization), serialized as `(lean-prod ...)`; searched exactly like `FProd`, kept distinct only so a later behavioral adapter can tell data pairs from propositions
`FSum`	logical/data sum	`Or`, `Sum`, or `PSum` with constructor map
`FTop`	unit/truth	`True`, `Unit`, `PUnit`
`FBot`	empty/falsity	`False`, `Empty`, `PEmpty`
`FAll`	admitted sort quantifier	a dependent binder whose domain is a sort or a first-order kind arrow ending in a sort; carries an explicitness flag and the serializer's binder name `s0`, `s1`, ... `((all ...)` explicit, `(alli ...)` implicit)
`FInst`	instance binder wrapper	a non-dependent instance-implicit binder, keyed by its pretty-printed class type; erased before either engine searches, but the renderer must still bind a wildcard, and the key counts as an unsafe atom for refutations
`FExactContext`	structured class context	exact provider schemes/assignments only
`FVar`	universal type variable	selected sort-bound local variable
`FAtom`	opaque atom with refutation-safety bit and pretty-printed display key	any subterm the serializer cannot decompose (dependent forall, term-indexed application, a sort, a concrete constant such as `Nat` outside an inductive expansion)
`FApp`	nominal/variable type application	first-order type-valued application with retained arguments
`FParamInd`	parameterized nonrecursive inductive	complete sum-of-products family with parameters
`FInd`	occurrence-local nonrecursive inductive	the same expansion when some applied parameter is not a proper type (or there are none), keyed by the occurrence's spelling
`FParamRec`	parameterized recursive (or nested) family	`(param-rec complete|partial ...)`: exact family head, ordered proper-type parameters, and the constructors whose fields could be serialized; recursive self-fields appear as the occurrence's own atom key
`FRec`	recursive (or nested) family	`(rec complete|partial ...)`: the occurrence-local form; Djinn uses the constructors only as introduction premises, Exference may additionally case on one layer of a complete occurrence; the key always counts as unsafe for refutations
`FDepth`	bounded-depth sentinel	marks a traversal limit rather than logical structure

The exact constructor names matter less than the separation of concerns: arrows/products/sums are searchable logic; variables and applications preserve type structure; inductive records preserve declarations; atoms preserve identity; flags preserve whether absence of a proof can be trusted.

18.2 Recognized logical structure

The Lean-side traversal recognizes common families by exact constant identity, not by pretty-printed notation. It structurally exposes:

- non-dependent arrows;
- `And`, `Prod`, and `PProd`;
- `Or`, `Sum`, and `PSum`;
- `Iff`, lowered to the two implications;
- `Not`, lowered to implication into falsity;
- `False`/`Empty`/`PEmpty`;
- `True`/`Unit`/`PUnit`;
- selected sort quantifiers;
- qualifying inductive declarations.

The renderer later restores nominal distinctions and exact constructor names. The search engines can reason over a common product/sum structure without pretending the Lean families are definitionally identical.

18.3 Non-dependent versus dependent foralls

For a Lean `forallE`, the serializer asks whether the body contains the bound variable.

Dependent binder whose domain is a type kind
: If the body mentions the bound variable and the domain is a sort or a first-order kind arrow whose domains and result are sorts (the serializer's `isTypeKind`), the binder becomes `FAll`: the serializer opens it as a local named `s0`, `s1`, … (`s` followed by the current binder depth), emits `(all "s0" ...)` for an explicit binder and `(alli "s0" ...)` for an implicit one, and translates the body under it. `Nat → Type` and other term-indexed families are not type kinds.

Other dependent binder
: Cannot become an ordinary arrow. The whole `forallE` is sealed as one opaque atom (`(atom unsafe "∀ (n : Nat), P n")`, say); the traversal does not descend into it.

Non-dependent explicit binder
: Becomes an arrow `(-> D R)`: introduce an ordinary term and synthesize the codomain.

Non-dependent instance-implicit binder
: Becomes `(inst "C α" R)`, i.e. `FInst`, in ordinary goals; in provider schemes and exact assignment payloads it becomes structured `(exact-context ...)` evidence or fails closed.

Non-dependent ordinary implicit binder
: Still becomes an `(alli "i0" ...)` scheme binder (`i` plus the depth) in goal mode, so that Djex may choose a visible instantiation for an otherwise unused implicit type parameter.

Thus `∀ α : Type, α → α` can be represented as polymorphism, while `∀ n : Nat, P n` is not analyzed as a first-order type scheme. It may still be carried opaquely.

18.4 Type applications retain arguments when they are type-level

Earlier versions of many synthesis adapters flatten applications into strings. Leant instead retains the application head and argument fragments when Lean establishes that the application is type-valued and its arguments fit the admitted first-order kind discipline.

For example, `List α`, `Except ε α`, or a user-defined unary type constructor can preserve nominal family identity and parameter structure. This supports higher-kinded provider matching and prevents unrelated instantiations from collapsing into one atom.

A term-indexed application such as `Vector α n` is different: *n* is data, not a type parameter. Unless the whole family is handled through a qualifying inductive representation, the application remains opaque. The search engine does not erase *n* and pretend all vector lengths coincide.

18.5 Opaque dependent cargo

Consider:

A dependent proposition moved as a whole

```
opaque P : Nat → Prop
opaque Q : Prop

-- Synthesis target:
((∀ n : Nat, P n) ∧ Q) → Q ∧ (∀ n : Nat, P n)
```

Let X denote the exact opaque atom for `∀ n, P n`. The searchable formula is

$$(X \wedge Q) \to (Q \wedge X).$$

Djinn can synthesize the product swap. The renderer restores the original dependent formula as the type of the moved hypothesis; no term inspects *n* or proves any `P n`.

This technique is useful for arbitrarily complicated formulas when their role is purely structural. It is not dependent proof search. A target such as

Requires analysis, not transport

```
(∀ n : Nat, P n) → P 0
```

requires instantiating the dependent function at `0`. At the pinned revision Leant cannot do this: the premise is a dependent forall over a non-type domain, so the serializer seals it whole as `(atom unsafe "∀ (n : Nat), P n")`, and `P 0` (a proposition-valued application whose argument is a term) becomes the distinct atom `(atom unsafe "P 0")`. Djinn sees $X \to Y$ for two unrelated atoms and finds nothing; because both atoms are unsafe the outcome is reported as "no term found within bounds", never as a refutation. Leant's bounded quantifier instantiation (Chapter 20) concerns *type* quantifiers admitted as `FAll`, not term-level foralls such as this one.

18.6 Refutation-safety bits

An opaque atom can be safe for positive transport yet unsafe for negative reasoning. Leant attaches a Boolean safety classification to atoms and applications; the serializer's rule is one line, `safe := e.getUsedConstants.isEmpty && !e.isSort`, so a sealed subterm is safe exactly when it mentions no global constant and is not itself a universe.

A formula built only from universally quantified abstract variables can sometimes be treated parametrically: its hidden structure cannot smuggle in a concrete known inhabitant. An opaque subtree mentioning concrete constants, fixed indices,

hidden instances, incomplete declarations, or traversal limits may conceal exactly the proof that search failed to find. Such a subtree downgrades a negative outcome.

The flag is not Lean's `unsafe` declaration marker. It means "safe to rely on this opacity when interpreting exhaustive failure." On the Haskell side `fragUnsafeAtoms` collects everything that poisons a negative verdict: unsafe atoms, unsafe applications, every `FInst` key, every `FExactContext` class, and the keys of all recursive occurrences (`FRec`, `FParamRec`); `fragHasDepth` reports a truncated traversal. `Leant.Synth.Engine` accepts a Djinn refutation as sound only when both are clear and the search ran unbudgeted.

18.7 Depth and resource limits are represented, not forgotten

A bounded traversal that runs out of fuel emits the marker `(depth)`, parsed as `FDepth`, rather than a completeness downgrade hidden inside an atom. The goal serializer starts with fuel 100 and provider schemes with fuel 80; a goal whose fragment contains `FDepth` is refused outright (`fragRefusal`: "the goal exceeds the translator's depth bound"). It does not silently replace unseen structure with a trustworthy atom. This preserves the distinction between:

- a semantically opaque subtree intentionally treated as an atom;
- a subtree that was never fully inspected because a resource bound stopped the serializer.

Only the first can participate in carefully justified parametric negative reasoning, and even then only under its safety classification.

Source trail

The fragment datatype, generated Lean serializer, S-expression parser, provider metadata, and candidate-verification program are in https://github.com/VladimirReshetnikov/Leant/blob/49c8f9f100a644b5a32b3c942bc6565a3a3ba6a0/src/Leant/Synth/Fragment.hs. The module comments give the most concise current statement of the exact structural fragment and opaque-cargo policy.

Chapter checkpoint

`Frag` is Leant's semantic firewall. It exposes admitted arrows, products, sums, polymorphism, applications, and inductives; seals unsupported dependency as alpha-aware atoms; and carries explicit completeness/safety metadata. It is richer than Djinn formulas but intentionally poorer than Lean expressions.

Inductives, Recursion, Providers, and Instances

19.1 Expanding an inductive as a sum of products

A nonrecursive datatype such as

A simple finite datatype

```
inductive Choice (A B : Type) where
  | left  : A → Choice A B
  | right : B → Choice A B
```

has the structural shape $A+B$. A one-constructor record with two independent fields has the shape $A\times B$. Leant may inspect the live inductive declaration and emit constructor-tagged products and sums when all of the following are established:

- the head is an inductive with no indices (`numIndices = 0`), not recursive, not nested, not `unsafe`, and not part of a mutual block (`iv.all.length = 1`);
- the occurrence applies the family to exactly its parameters (`numParams` arguments);
- every field of every constructor, at those parameters, is an explicit, non-dependent binder (a field mentioned by a later field, or an implicit/instance field, makes the whole family fall back to an application or an atom);
- the constructor list is therefore complete by construction, which is why an `FInd`/`FParamInd` node itself never poisons a refutation; only its field fragments can;
- when every applied parameter is a proper type the occurrence is emitted as `(param-ind Family key (params ...) (ctor ...) ...)` and shared across the query, otherwise as the occurrence-local `(ind key (ctor ...) ...)`.

(These are the tests performed by `indOf` in the generated prelude, `src/Leant/Synth/Fragment.hs`.) `Bool`, `Ordering`, `Option α`, `Except ε α`, `Decidable p`, and a user `structure Pair (A B : Type)` all qualify; the golden transcript `test/synth-inductives.golden` shows each of them being constructed and eliminated.

Constructor names and arities are retained so a Djinn proof term can be lowered back to the correct Lean injections and case branches.

19.2 Why indexed and dependent-field inductives remain opaque

For `Eq a b`, the only constructor changes what can be known about the index b. For `Vector α n`, constructors return different indices. For `Exists`, a later proof field depends on the witness. Flattening these to ordinary sums/products would discard the equalities and dependencies that make their eliminators type-correct.

Leant therefore keeps such families opaque unless a dedicated representation accounts for the dependency. The policy favors an incomplete search over an unsound translation.

19.3 Recursive types and one-layer structure

A recursive declaration such as `List α` is structurally

$$1 + (\alpha \times \text{List } \alpha).$$

This equation suggests constructor introduction and case analysis, but unrestricted proof search can unfold it forever. Leant and Djex therefore retain a bounded recursive description: `recOf` accepts a recursive or nested, non-indexed, non-mutual inductive applied to its parameters, blocks the occurrence's own key so that recursive fields serialize as the matching atom, and emits `(rec complete|partial key (params ...) (ctor ...) ...)` (or `param-rec` with the exact head when all parameters are proper types); `partial` means some constructor's fields could not be serialized and was dropped.

Current behavior distinguishes:

- bounded positive recursive introduction, which can construct finite layers;
- recursive elimination, supported only under restricted Exference policies;
- general recursion or recursor invention, which is not part of ordinary structural search.

A recursive family whose inventory is partial may still produce a candidate that Lean verifies. It cannot support a complete "uninhabited" conclusion.

19.4 Providers turn the environment into premises

A synthesis problem is often impossible from local arguments alone but easy using an imported function. Leant discovers selected global constants as *providers*. A provider contributes a checked type scheme and an exact Lean global name; the engine may use it like an additional premise.

For example, a goal requiring `Demo.consume` could be solved only if that constant is in the admitted provider inventory. Provider discovery is bounded and query-directed so the search environment does not explode with every declaration in every import.

The provider query records nominal roots and a possible result head: the serializer emits `(query (roots "Prod" "Sum") (head "Sum"))`, the root namespaces of every constant the goal mentions plus the head constant of its final result. The Lean-side scanner (`providerProgram`) then keeps constants whose root namespace is one of those roots, plus every declaration made in the current session; drops compiler-generated names and recursor-style auxiliaries (`rec`, `casesOn`, `noConfusion`, …) and anything whose fully peeled result is a sort; sorts the rest shortest-name-first; ranks exact result-head matches and session declarations ahead of the pool, and conventional implementation workers (`go`, `loop`, names ending in `TR`, `Impl`, `Aux`) last; and serializes at most 80 providers with fuel 80 each, together with their leading type-binder names and instance-evidence assignments. Provider discovery runs only after the provider-free structural lane has failed to verify a candidate, or when the goal is a single opaque atom or application that a live value might inhabit. Haskell caches inventories in a twelve-entry cache keyed by the canonical query (sorted, deduplicated roots plus head) and by a *provider world* generation that advances whenever imports or user declarations change, but not when a generated `it` binding is added.

19.5 Curated library mode

The `synth-library` setting (default `on`) exposes a curated, rated collection of library functions over recursive inductives; the default table in `src/Main.hs` has seventeen entries (`List.map`, `List.append`, `List.flatten`, `List.foldr`, `List.reverse`, `List.length`, `Nat.add`, `List.replicate`, `List.foldl`, …), no `.rec` recursors, and a project file `leant.ratings` may override or extend it at startup. Whenever the goal mentions a recursive inductive, the serializer instantiates these functions at the goal's own element types and appends them as `(prem "List.map" FRAG)` offers; the driver keeps at most eight offers, runs them as antecedents of a Djinn search with a choice-point budget of 100 000, and lists their candidates ahead of the plain structural ones. This is a pragmatic way to synthesize nontrivial behavior without teaching the core engine to invent recursion.

The distinction is important:

- using a known provider is ordinary application synthesis;

- inventing the provider's recursive implementation is program synthesis at a much richer level.

A term using an admitted provider is still verified against the exact Lean goal. Negative conclusions are relative to the provider inventory and its completeness policy, not to every constant that might exist in Lean.

19.6 Provider schemes and exact instantiations

A polymorphic provider may have sort binders and class constraints. Leant's provider protocol records:

- the exact global name;
- a fragment for its source scheme;
- binder metadata aligned with visible quantifiers;
- exact or legacy bounded instantiation evidence for erased constraints;
- nominal higher-kinded arguments with admitted kind arities;
- structured class contexts only at the provider-owned boundary.

The generated Lean code asks the live instance resolver (`Lean.Meta.SynthInstance.getInstances`/`tryResolve`) for candidate instance heads, explores each under restored metavariable state, closes all related constraints, and keeps only complete ground-kinded assignments. Partial assignments do not enter the search pool. The bounds are fixed on both sides of the wire: at most six leading type binders per provider (Djex's `maximumProviderInstantiationArguments`), 32 instance heads inspected, 16 distinct assignment vectors retained per provider, kind arity at most 64, and Djex accepts at most 32 assignments per query.

This machinery allows a provider whose applicability depends on type-class evidence to be instantiated more faithfully than a flat collection of guessed type arguments.

19.7 Why instance search must be bounded and fail closed

Type-class resolution can recurse through instance premises, branch among heads, and instantiate type variables. An exhaustive global characterization may be impractical or impossible under open-world imports. Leant therefore uses explicit caps on inspected heads, assignments, argument counts, and kind arities.

Those bounds are useful for positive synthesis: any resulting term is checked. They are not silently upgraded into a proof that no other instance assignment exists. Exhaustion or omission is recorded in the completeness ledger.

19.8 Premise staging

Leant runs several search lanes in a fixed order, all under one command-wide wall-clock deadline (20 s by default, `LEANT_SYNTH_TIMEOUT=N`, `0` waits forever):

1. the *baseline* lane: the goal's own structure and hypotheses, plus the library premises when `synth-library` is on; its rendered candidates are verified first, and one verified survivor is terminal;
2. the *provider* lanes, only if the baseline verified nothing (or the goal was a provider-open atom): Djinn is given sparse prefixes of the ranked inventory, the first 1, 4, and 16 providers and then the whole bounded inventory; Exference always gets the full inventory with rank-based penalties; `both` runs both engines for the singleton and full lanes and Djinn alone in between; spellings already sent to Lean by an earlier lane are removed from later streams;
3. the *classical* fallback, only after a *sound* Djinn refutation and only when `synth-classical` is on: first excluded-middle premises `Classical.em _` for each atomic subformula (skipped when there are none or more than five), then the double-negation translation wrapped in `Classical.byContradiction` (Glivenko's theorem makes this complete for a propositional body under a quantifier prefix).

Recursive support is not a separate lane: constructor premises for recursive occurrences ride along in whichever lane is running. A conclusive negative claim must account for every lane that is part of the configured search contract. Failure in the baseline is not final if a provider lane is still permitted; that is why a sound baseline refutation is held back until the provider lanes have also failed, and why the classical routes run last.

Open-world versus closed-world reading

Lean's environment is open and extensible: a future import can add a function of the desired type. Leant's refutation can at most concern the closed structural calculus and exact provider inventory sealed for the current query. It is not a metaphysical claim that no Lean declaration could ever inhabit the type.

Chapter checkpoint

Simple complete inductives can become sums of products; indexed or dependent families remain opaque. Recursive structure is bounded. Providers make selected live declarations available as premises, with exact names and checked schemes. Type-class assignments and discovery bounds help positive search but weaken negative evidence unless exhaustiveness is established.

Chapter 20

Djinn and Exference: Two Different Search Contracts

20.1 Why Leant has two engines

No single search strategy is best for every goal. Djinn offers a logical decision procedure for a disciplined fragment. Exference offers a richer, ranked, resource-bounded expression search. Leant can run either engine or merge both result streams, but it must not merge their evidence meanings.

20.2 Djinn: proof search in LJT

Djinn uses Roy Dyckhoff's contraction-free LJT calculus for intuitionistic propositional logic. The calculus is designed to avoid unrestricted contraction and to make proof search terminate without repeatedly duplicating assumptions.

At a high level, Djinn:

1. converts a checked source type and declaration environment into an LJT formula and named assumptions;
2. applies right rules to decompose the goal;
3. applies focused left rules to exploit assumptions;
4. produces a normalized proof term with free variables corresponding to premises;
5. independently checks the proof term against the formula environment;
6. lowers the proof through constructor and provider mappings into a typed candidate representation.

The default unbudgeted depth-first mode is complete for the supported formula calculus. An optional choice-point budget changes the contract: an empty result with budget exhaustion means only "not found within the explored prefix." Leant's ordinary Djinn run (`djinnRun` in `src/Leant/Synth/Engine.hs`) sets `optionAlternatives = True`, `optionCutoff = 60` (the candidate window; reaching it prints `note: search truncated: candidate limit reached (60)` and carries no negative evidence), and `optionBudget = Nothing`; only the library-premise lane and the excluded-middle lane pass a budget of 100 000 choice points, and their negatives are discarded anyway.

Djinn can retain alternative proofs and can interleave branches under another strategy, but search strategy is distinct from logical completeness. The code records whether a bound stopped exploration.

20.3 What Djinn can decide

For an exact formula made of atoms, implication, finite conjunction/disjunction, and nominal emptiness, unbudgeted LJT search can decide intuitionistic provability. By Curry–Howard, it can produce a structural inhabitant or establish non-provability in that formula calculus.

This is not yet a Lean non-inhabitation result. Leant must additionally establish that:

- its Lean-to-fragment translation was exact for negative purposes;
- fragment-to-Djinn lowering preserved the relevant semantics;
- all premises and constructor declarations were complete;

- no bounded quantifier/provider/recursive stage remained;
- search was genuinely unbounded and exhausted.

Only then may Djinn's non-provability support a proof-backed uninhabited report.

20.4 Exference: best-first typed expression search

Exference explores typed expression states using a priority queue and heuristic scores. It combines function bindings, deconstructors, unification, constraints, rigid instantiation plans, expression checking, and search-control policies.

Its options record (`ExferenceOptions` in `exference/src-core/Language/Haskell/Exference/Core/Internal/Options.hs`) and the values Leant passes are:

- `exferenceMaximumSteps`: processed search nodes; Leant passes `rsSynthSteps`, default 4096 (`:set synth-steps N`);
- `exferenceMaximumQueueSize`: Leant fixes `Just 1024` (Djex's default is 8192) to keep memory interactive; hitting it is reported as `queue limit pruned N`;
- `exferenceMaximumDepth`: optional heuristic-depth cap, left at Djex's default (none);
- `exferenceAllowUnused`: whether a candidate may ignore an introduced binder; Leant first runs a strict lane and, only if it yields nothing, a relaxed one;
- `exferenceAllowResidualConstraints`/`exferenceConstraintDeferralSteps`: whether and how long unresolved class constraints may be deferred (defaults);
- `exferenceMultiConstructorPatterns`: enabled by Leant, so `match` over multi-constructor datatypes is generated;
- `exferenceHeuristics` and the session's rating overrides: Leant assigns provider k in discovery order the penalty $20k$, so the best-ranked live value is tried first.

Its completion status distinguishes full exhaustion from step limits, queue/depth pruning, and identifier-space exhaustion. This is good operational hygiene, but even full exhaustion is conclusive only for the sealed Exference calculus and finite environment. Leant currently treats Exference as a positive heuristic engine, not a refutation authority.

20.5 Typed candidate graphs

Modern Djex retains more than rendered source. An Exference candidate can carry a typed term graph, source type-variable hints, exact inventory identity, and fingerprints. Independent checkers validate type, scope, syntax, constraints, and source projection before the candidate enters canonical result envelopes.

Leant preserves this sidecar through grouping and verification. It enables later semantic layers, such as Length interpretation, to analyze the exact candidate structure rather than reparsing a Lean string.

20.6 Result merging

When Leant runs both engines, candidate streams may contain extensionally or textually duplicate answers. Merging must preserve:

- engine identity and search policy;
- semantic candidate identity;
- rendering variants;
- exact typed origin where available;
- notes about recursive/provider/classical routes;

- verification status.

A duplicate string from Djinn and Exference can be displayed once while still retaining distinct provenance internally. In particular, Exference must not inherit Djinn's completeness evidence merely by producing the same text.

Property	Djinn	Exference
Core strategy	contraction-free logical proof search	heuristic best-first expression search
Main strength	small exact structural formulas	broader/ranked practical candidates
Unbounded decision procedure	yes, for admitted LJT formula fragment	no general Leant refutation contract
Resource controls	alternatives, strategy, optional choice budget	steps, queue, depth, constraints, heuristics
Proof/candidate validation	independent LJT proof checker and typed lowering	independent typed expression/candidate checks
Negative authority in Leant	possible only with complete translation ledger	none

Source trail

Djinn's search contract is documented in https://github.com/VladimirReshetnikov/Djex/blob/0cf82f2aa22debb1de2c7aaa13a2abbe190550d8/djinn/src-core/Djinn/Internal/LJT.hs. Exference's validated input, priority-queue search, completion states, and projections are in `exference/src-core/Language/Haskell/Exference/Core/Internal/Exference.hs`. Leant's engine boundary is `src/Leant/Synth/Engine.hs`.

Chapter checkpoint

Djinn and Exference share source infrastructure but promise different things. Djinn can decide an exact propositional fragment when unbounded. Exference is a ranked, bounded positive search. Leant may merge candidates, never evidence authorities.

Chapter 21

Rendering Candidates Back into Lean

21.1 A proof term is not yet Lean source

Djinn returns a proof term over symbols, tuple/sum operations, and premise variables, already lowered by Djex to its shared `Expression` syntax; Exference returns a typed term graph (or, as a fallback, the same erased expression syntax). Neither is directly accepted by Lean. `renderLeanTerm` and `renderLeanTermGraphProjection` in `src/Leant/Synth/Render.hs` must reconstruct:

- lambda binders with legal, noncapturing names;
- applications and precedence;
- exact Lean global names for providers and constructors;
- product construction and splitting;
- injections and case expressions;
- impossible elimination from empty types;
- explicit versus implicit type binders;
- missing type arguments and instance arguments;
- nominal type-family distinctions erased by logical lowering.

21.2 Universe-agnostic constructor syntax

Whenever possible, the renderer uses syntax whose meaning is supplied by the expected goal:

- `⟨a, b⟩` for one-constructor product-like targets;
- dot constructors for sum injections when the expected family identifies them;
- `match` for case analysis;
- `nomatch` or empty elimination for impossible cases.

This avoids hard-coding universe-specific constructors and lets Lean recover exact parameters.

21.3 Binder reconstruction

Djinn's formula quantification does not remember every Lean surface binder. Leant records a visibility vector and walks the candidate term against it.

An explicit type binder may require an actual lambda in rendered source:

Explicit type argument

```
fun (α : Type) => ...
```

An implicit type binder may remain omitted because Lean can infer it. A provider application may require an explicit placeholder:

Explicit slot restored by a placeholder

```
someProvider _ x
```

If a quantified argument can be instantiated in more than one plausible place, Leant generates a bounded family of textual variants.

21.4 One semantic candidate, several strings

Suppose the engine has proved the structural candidate "apply provider *p* to value *x*." Depending on Lean's original binder sequence, viable renderings may include:

Illustrative variant group

```
p x
p _ x
@p _ x
```

These strings are not three independently discovered programs. They are reconstruction attempts for one semantic candidate. The verification loop tries them in deterministic order and accepts the first that elaborates. (When Djex makes a provider's vacuous type instantiation visible and live discovery recorded the binder's source name, the renderer prefers a named argument, `Demo.global («a» := Nat)`, as in `test/synth-djinn-providers.golden`; the positional `@`-form is the fallback for inventories without binder metadata.)

Grouping matters for quotas: failed variants do not consume a requested candidate slot, and multiple accepted spellings of the same semantic term do not flood the output.

21.5 Name restoration and hygiene

Search engines allocate their own binder symbols and may alias provider names. The renderer:

- restores exact global Lean names from checked maps;
- allocates fresh local names avoiding collisions with globals, atoms, and other binders;
- respects shadowing and alpha-equivalence;
- uses constructor descriptors validated against declarations;
- avoids treating opaque display text as an identifier.

The result must be syntactically valid even before type checking, but syntax validity never grants semantic authority.

21.6 Rendering failure is ordinary candidate failure

A search result can be valid in the smaller calculus yet lack a supported Lean rendering. For example, a legacy proof-term node may have no current lowering, or a quantified instantiation may be irrecoverably ambiguous. Leant drops that rendering route and records diagnostics rather than inventing unchecked syntax.

The renderer is intentionally untrusted

A renderer bug can produce malformed or ill-typed Lean source. The verification boundary converts that bug into a rejected candidate. The design does not require the renderer to be part of the logical trusted base.

Source trail

Candidate lowering, binder visibility, variant enumeration, name restoration, and universe-agnostic Lean syntax are implemented in https://github.com/VladimirReshetnikov/Leant/blob/49c8f9f100a644b5a32b3c942bc6565a3a3ba6a0/src/Leant/Synth/Render.hs.

Chapter checkpoint

Rendering is a reconstruction problem, not pretty-printing. Leant may derive several Lean strings from one semantic candidate, preserves exact names and binder hygiene, and relies on expected-type elaboration. Unsupported or wrong renderings are simply rejected.

Chapter 22

Verification and the Positive Soundness Boundary

22.1 The exact verification program

For a goal string G and rendered candidate t, Leant constructs:

Candidate verification

```
set_option autoImplicit true in
noncomputable example : (G) := (t)
```

This is exactly the string built by `candidateVerificationProgram` in `src/Leant/Synth/Fragment.hs` (on one line) and sent as an ordinary command in the user's current environment, so it sees the same imports, session declarations, and `set_option`s as the user. `noncomputable` only disables code generation. Leant accepts the variant when the backend request succeeded, the response carries no fatal error, no message of severity `error`, and no `sorry` entry; the rejection classes are `BackendRequestFailure`, `BackendFatalResponse`, `LeanErrorDiagnostic`, and `LeanContainsSorry`. Acceptance therefore requires:

- the goal to elaborate exactly as a type;
- the candidate to elaborate against that expected type;
- all names, implicit arguments, instances, coercions, and universes to resolve;
- the elaborated term to contain no unsolved metavariables or `sorry`;
- the kernel to accept the resulting declaration under safety rules.

A successful callback is wrapped as `Verified` inside Leant's verification module. That wrapper means only that this callback accepted this variant; it is not a behavioral proof, solver certificate, or completeness witness.

22.2 Lazy quota-aware checking

Candidates arrive in semantic groups. Verification proceeds lazily:

1. take the next group;
2. try its variants in deterministic order;
3. on the first accepted variant, emit one verified candidate and consume one quota slot;
4. if every variant fails, continue without consuming the quota;
5. stop once the success quota is reached or the stream ends.

The numbers are fixed in `src/Leant/Synth/Engine.hs`: a lane hands at most `synthMaxTried` $= 12$ fresh groups to Lean (24 in `both` mode, so that neither engine's standalone frontier is lost), and `synthMaxShown` $= 5$ accepted groups are displayed and bound; the excluded-middle classical lane uses a six-group frontier. `verifyCandidateGroups` in `src/Leant/Synth/Verification.hs` is the pure driver and `synthVerify` in `src/Main.hs` supplies the backend callback. This avoids eagerly compiling a potentially large candidate stream and gives users five usable answers rather than five attempts.

22.3 What positive verification guarantees

If the Lean worker accepts the command, then relative to the live environment:

$$\vdash t : G.$$

This guarantee survives errors in:

- fragment classification;
- Djex search;
- proof-term lowering;
- provider matching;
- name restoration;
- variant selection.

Those errors may cause missed candidates, wasted work, or surprising terms, but an ill-typed result should not be displayed as verified.

22.4 What positive verification does not guarantee

It does not establish that:

- the candidate has the user's intended behavior;
- the candidate is efficient, terminating under unsafe execution paths, or free of undesirable axioms;
- the candidate is unique or minimal;
- the search was complete;
- the candidate satisfies an optional semantic contract not encoded in its Lean type;
- a textual copy in another environment will still elaborate.

A type such as `List α → List α` admits many verified inhabitants. Verification proves type correctness, not intent.

22.5 Binding accepted candidates into the session

Leant retains every accepted result as a hidden session declaration: `set_option autoImplicit true in def «it!N» : (GOAL) := (TERM)`, retried with `noncomputable` if plain `def` is rejected. Survivors are bound worst-first so that the newest binding, which bare `it` denotes, is the best candidate; the labels `it1`, `it2`, … printed in the transcript are Leant-side aliases that later input may use (`#check it2`, `exact it1`) and that Leant substitutes with the mangled names (or, in prove mode, with the candidate applied to the goal's hypotheses). The binding is created through the Lean backend, giving later commands an exact environment object rather than a Haskell-side fiction. Replay and snapshot machinery preserve the command history needed to reconstruct session state.

Never reverse the trust arrow

The correct reasoning is "Lean accepted the candidate, therefore it inhabits the goal." It is not "Djex produced the candidate, therefore Lean ought to accept it." Search is advisory; verification is authoritative.

Source trail

The group-level lazy verifier and its narrow `Verified` wrapper are in https://github.com/VladimirReshetnikov/Leant/blob/49c8f9f100a644b5a32b3c942bc6565a3a3ba6a0/src/Leant/Synth/Verification.hs. The exact Lean command is generated in `src/Leant/Synth/Fragment.hs`; session binding is orchestrated by `src/Main.hs`.

Chapter checkpoint

Every displayed candidate crosses the original Lean elaboration and kernel boundary. Verification is lazy and group-aware. It proves exact type inhabitation in the live environment, while deliberately making no claim about behavior, optimality, or search completeness.

Negative Evidence and the Completeness Ledger

23.1 Why “no term found” has several meanings

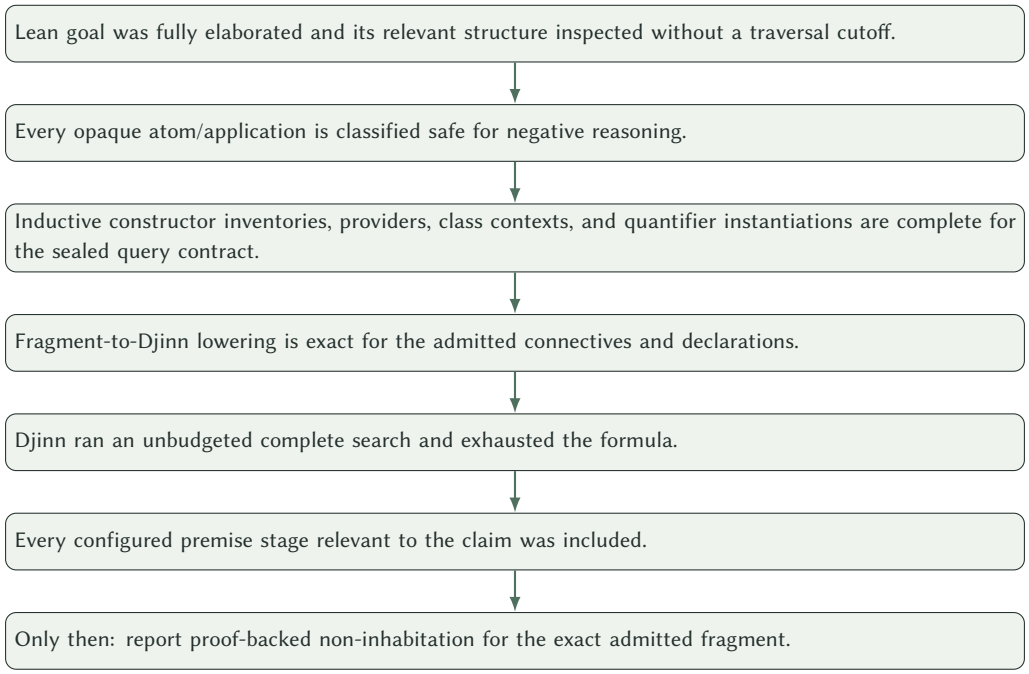
An empty candidate list can result from very different events:

- the exact structural formula is intuitionistically unprovable;
- the search budget expired;
- Exference pruned its queue or reached a depth cap;
- provider discovery omitted a useful constant;
- a quantified instantiation plan was bounded;
- recursive structure was only partially unfolded;
- hidden instance evidence was not enumerated;
- a valid engine candidate had no supported rendering;
- every rendered variant failed Lean elaboration;
- the goal contains dependent structure represented opaquely.

Only the first event, under an exact translation and complete environment contract, supports “uninhabited.” The others support weaker statements such as “no verified term was found under current bounds.”

23.2 The chain required for a strong negative result

A proof-backed negative report requires all links below:



A single failed link downgrades the wording.

23.3 Exact, safe opacity versus unsafe opacity

Opacity is not automatically fatal. If a universally quantified abstract formula is sealed as an atom and the structural calculus treats it parametrically, exhaustive failure may still be meaningful. But an opaque concrete type can hide a constructor or theorem. For example, treating `Nat` as an arbitrary atom and searching for `Nat` from no premises would return no proof, although `0` is an inhabitant.

Leant's safety bit and completeness facts distinguish these cases. Concrete constants and fixed applications generally prevent a structural absence from becoming a Lean-level refutation unless their constructor inventory is represented exactly.

23.4 Quantifier plans

Higher-rank and impredicative synthesis may require instantiating polymorphic premises. A bounded finite plan can discover useful candidates, but failure after finitely many guessed instantiations is not a proof that no instantiation works.

The completeness ledger records whether the current quantifier fragment and plan are among the exact complete cases. Ambiguous trailing instantiations that are merely rendered as variants also cannot support negative authority.

23.5 Provider-relative completeness

A closed formula may be uninhabited from local assumptions but inhabited using a global provider. Leant's query contract therefore includes the provider stages and inventories actually searched.

Even a complete provider scan over one live environment would establish only environment-relative absence. Leant uses bounded query-directed discovery, so most provider-enabled failures remain explicitly inconclusive.

23.6 Recursive and instance-related downgrades

A recursive type with bounded unfolding may hide arbitrarily deep constructor terms. A hidden instance dictionary may carry methods or data. A partial constructor inventory may omit a branch. Each is acceptable for positive search because Lean checks candidates; each blocks strong negative interpretation.

23.7 Djinn versus Exference wording

Djinn can produce an internal evidence value corresponding to proved uninhabitability only when its formula search is complete. Leant still gates that value through fragment safety.

Exference never grants Leant a refutation. Even if its queue reports natural exhaustion, the engine is used as a heuristic candidate generator under a finite environment and option set. Leant reports absence, limits, and pruning diagnostics without calling the Lean type uninhabited.

23.8 What Leant actually prints

The wordings are fixed in `report` inside `src/Main.hs`; reading them is the quickest way to know which of the three outcomes above occurred.

Message	Meaning
`provably uninhabited — no closed term of this polymorphic type exists`	Djinn's unbudgeted search returned `ProvedUninhabitable`, the fragment had no unsafe atoms and no depth marker, the provider lanes (if any ran) found nothing, and the classical routes (if enabled) also found nothing
`(constructively — a classical proof may still exist; this is not a disproof of the proposition)`	appended to the line above when the goal was serialized as `prop`; `synth-classical off` or a classical miss
`no term found within bounds (opaque atoms '...' hide structure the engine cannot analyze — not a refutation)`	Djinn exhausted its formula, but the fragment contains unsafe atoms (the first three keys are listed)
`no term found within the search bounds` followed by `note: search truncated: ...`	no candidate and no logical claim: an Exference lane, a budgeted Djinn lane, or a Djinn run truncated at the candidate window
`the engine proposed N candidate(s) but none survived Lean verification`	search succeeded, every rendered variant was rejected by Lean
`out of fragment: the goal is a single opaque atom '...'`	`fragRefusal` fired and no provider could open the atom
`the engine did not finish within Ns — no answer, not a verdict`	the command-wide deadline expired

23.9 Three negatives, worked

A sound refutation. `test/synth-basic.golden` contains

Transcript

```
λ> :synth (∀ a b : Type, a → b)
provably uninhabited — no closed term of this polymorphic type exists
```

The serializer opens the two explicit sort binders as `s0` and `s1` and emits `(goal type (query (roots ) (head)) (all "s0" (all "s1" (-> (var "s0") (var "s1")))))`: no constant is used, so the root list is empty and the peeled result head is a variable. The fragment is `FAll True "s0" (FAll True "s1" (FArr (FVar "s0") (FVar "s1")))`;

`fragUnsafeAtoms` is empty. Djinn's formula is $a \to b$ over two distinct atoms under implicit polymorphism; the unbudgeted LJT search terminates with no proof, Djex reports `ProvedUninhabitable`, and `djinnRun` returns `SynthRefuted True`. `src/Main.hs` does not print yet: it consults the provider inventory (with no roots and no session declarations the inventory is empty), then, because `synth-classical` is on, runs the excluded-middle lane with premises `Classical.em _ : s0 ∨ (s0 → ⊥)` and `s1 ∨ (s1 → ⊥)` and the double-negation lane on $\forall s_0\, s_1,\ ((s_0 \to s_1) \to \bot) \to \bot$; both fail, and only then is the sound refutation printed. Because the goal was serialized as `type`, no "constructively" qualifier follows.

A sound refutation that is not a disproof. `test/synth-prove.golden` contains

Transcript

```
λ> :set synth-classical off
synth classical: off
λ> :synth (∀ p : Prop, p ∨ ¬ p)
provably uninhabited — no closed term of this polymorphic type exists
(constructively — a classical proof may still exist; this is not a disproof of the proposition)
λ> :set synth-classical on
synth classical: on
λ> :synth (∀ p : Prop, p ∨ ¬ p)
  it1  fun _ => Classical.em _
  it2  fun _ => match Classical.em _ with | .inl x => .inl x | .inr k => .inr k
```

The fragment is `FAll True "s0" (FSum (FVar "s0") (FArr (FVar "s0") FBot))` (`Not` was lowered to an arrow into `FBot`), the goal sort is `prop`, and intuitionistic LJT correctly refuses excluded middle. With the classical fallback on, the excluded-middle lane hands Djinn the premise `Classical.em _` of type $s_0 \vee (s_0 \to \bot)$, which is the goal itself, and both survivors verify against the original Lean goal.

An unsound absence. Take the earlier example with `opaque P : Nat → Prop` in the session and ask `:synth ((∀ n : Nat, P n) → P 0)`. The serializer produces `(-> (atom unsafe "∀ (n : Nat), P n") (atom unsafe "P 0"))`: the premise is a dependent forall over a non-type domain and is sealed whole; `P 0` is a proposition-valued application whose argument is a term, so `appOf` declines and `atomOf` seals it. Djinn's formula $X \to Y$ has no proof and Djex reports `ProvedUninhabitable`, but `fragUnsafeAtoms` lists both keys, so `djinnRun` returns `SynthRefuted False`. `src/Main.hs` therefore continues into provider discovery (the roots include `Nat` and `P`; `P` itself is excluded because its peeled result is a sort, and nothing under `Nat` has type `P 0`), runs the provider lanes, and finally prints `no term found within bounds (opaque atoms '∀ (n : Nat), P n', 'P 0' hide structure the engine cannot analyze — not a refutation)` (or, should the provider lanes exhaust the 20 s deadline first, the timeout message, which is equally non-committal). The type is of course inhabited (`fun h => h 0`); the message says exactly that Leant has not shown otherwise.

A useful three-valued mental model

For a synthesis query, think in three broad outcomes:

1. **verified inhabitant**: a concrete Lean term checked;
2. **proved uninhabited in the exact admitted contract**: a rare, evidence-backed negative;
3. **unknown under current translation/search bounds**: the normal meaning of exhaustive-looking failure outside that contract.

23.10 Failure transparency

Good diagnostics report why authority was lost: unsafe opaque atoms, hidden instances, recursive partiality, bounded search, provider limits, rendering rejection, or Lean elaboration errors. Setting the environment variable `LEANT_SYNTH_DEBUG=1` makes `:synth` print `debug fragment:` (the parsed `Frag`), `debug premise:` lines for offered library premises, `debug provider:` lines for the discovered inventory, `debug N: VARIANT` for every rendered spelling sent towards Lean, `debug em outcome:` for the classical lane, and `debug metric: code=count` verification counters; together with the ordinary `note: search truncated: ...` lines these are everything needed to understand a result.

Source trail

The refutation gate is owned by https://github.com/VladimirReshetnikov/Leant/blob/49c8f9f100a644b5a32b3c942bc6565a3a3ba6a0/src/Leant/Synth/Engine.hs; fragment safety originates in `src/Leant/Synth/Fragment.hs`. Djinn's budget/exhaustion distinction is in `Djinn/Internal/LJT.hs`; Exference's completion states distinguish natural exhaustion, limits, pruning, and identifier exhaustion.

Chapter checkpoint

A negative result is a theorem about a search contract, not the absence of output. Leant needs exact translation, safe opacity, complete inventories and premise stages, and unbounded Djinn exhaustion. Otherwise the correct result is unknown or not found within bounds.

The Interactive Layer: `:synth`, `:prove`, Sessions, and Debugging

24.1 Expression-oriented interaction

Leant is designed for rapid exploration. Users can submit Lean declarations and expressions, inspect types and values, synthesize terms, enter proof workflows, and refer to previous results through `it`-style bindings.

The Haskell frontend remembers the Lean environment identifier after each accepted command. A failed speculative command need not destroy the prior state. History and snapshots support replay into a fresh backend process when necessary.

24.2 `:synth`

The synthesis command asks for whole terms inhabiting a goal. The forms accepted by `cmdSynth` in `src/Main.hs` are:

- `:synth TYPE`: synthesize inhabitants of `TYPE`, elaborated with `autoImplicit`; unresolved short variables are retried at `Type` when the universe unifier gets stuck;
- `:synth` with no argument: in prove mode, target the current goal with its accessible hypotheses as premises; otherwise target the last `sorry`;
- `:synth –behavior-mode filter – TYPE`: filter the candidates with the Length behavioral contract activated at startup;
- `:synth [–behavior-mode rank|filter] –length-contract ABSOLUTE-PATH – TYPE`: use one contract file for this command only. Both forms are rejected unless a startup configuration activated the Length assessment mode.

The session settings that affect it are exactly four, all read by `:set` (any other `:set OPT VAL` is forwarded to Lean as `set_option`):

- `:set synth-engine djinn|exference|both` (default `djinn`; case-sensitive);
- `:set synth-steps N` (Exference's step budget, default 4096, must be positive);
- `:set synth-classical on|off` (default `on`: classical candidates after a sound refutation);
- `:set synth-library on|off` (default `on`: rated library premises for recursive inductives).

Each setting given without a value prints its current state. Two environment variables complete the picture: `LEANT_SYNTH_TIMEOUT` (seconds, default 20, `0` unbounded) and `LEANT_SYNTH_DEBUG`. There is no setting for the number of results (five accepted groups), the candidate window (60), the verification frontier (12 or 24), Exference's queue bound (1024), or the provider cap (80); those are constants of the pinned revision.

The command is best suited to structurally informative types: adapters, combinators, product/sum rearrangements, polymorphic plumbing, constructor assembly, and applications of known providers.

24.3 `:prove`

Proof mode (`:prove PROP`, or bare `:prove` to resume the last `sorry`) works with goals in an interactive Lean proof state: every input line is a tactic, Leant reprints the goals, and after each step it probes a fixed list of finishing tactics (`rfl`, `trivial`,

decide, simp, omega, exact?, aesop) against the immutable proof state and prints a verified suggestion: that never enters the script by itself. Inside prove mode :goals, :undo [N], :script, :suggest, :auto, :synth, :qed [NAME], and :abort are available; :qed saves the finished script as a theorem (or a def for a non-proposition). Bare :synth wraps the current goal as ∀ (h₁ : T₁) ..., target over its accessible hypotheses, runs the ordinary pipeline, and binds each survivor applied to those hypotheses so that exact it1 closes the goal (see test/synth-prove.golden). Proof mode also benefits from ordinary Lean tactics for rewriting, induction, simplification, arithmetic, and dependent case analysis.

This division reflects the theory:

- :synth searches for an inhabitant of a comparatively self-contained target;
- :prove can transform a difficult dependent goal into simpler structural subgoals before synthesis is invoked.

A vector theorem may first require induction and index rewriting; a residual branch might then reduce to a product/function wiring problem that Djinn handles well.

24.4 Provider caches and environment identity

Provider discovery can be expensive. Leant.Synth.ProviderCache keeps a twelve-entry cache keyed by the canonical provider query (sorted, deduplicated root namespaces plus the result head, exactly what the serializer emitted) and by the current *provider world*, an integer generation that src/Main.hs advances (invalidateProviderWorld) on every accepted state-changing command except a generated it binding, and on import changes, resets, snapshot loads, undo, and backend restarts. Successful empty inventories are cached too; discovery failures are not. A cache hit is valid only because the key names the whole input of discovery.

The cache stores checked semantic records, not merely names. Reusing a string list across environments could associate a name with a different declaration or miss new instances.

24.5 Debugging a missing candidate

When an expected term does not appear, inspect the pipeline in order:

1. Did the raw goal elaborate as intended?
2. Which subtrees became arrows/products/sums, applications, inductives, or opaque atoms?
3. Was a depth or kind limit reached?
4. Were the needed providers discovered, and under what schemes?
5. Which engine ran, with what bounds?
6. Did the engine produce a semantic candidate?
7. Which Lean variants were rendered?
8. Why did Lean reject them?

This avoids blaming "search" for a failure caused by provider discovery or rendering, and avoids blaming "Lean" for a candidate never generated.

24.6 Reading source ownership

A useful source map is:

Concern	Primary Leant source
interactive state and command orchestration	`src/Main.hs`
Lean process protocol and timeouts	`src/Leant/Backend.hs`
Lean-side goal/provider serialization	`src/Leant/Synth/Fragment.hs`
fragment-to-Djex preparation and search	`src/Leant/Synth/Engine.hs`
Djex-term-to-Lean reconstruction	`src/Leant/Synth/Render.hs`
lazy group verification	`src/Leant/Synth/Verification.hs`
semantic provenance invariants	`docs/synth-internals.md`
provider inventory cache and world generations	`src/Leant/Synth/ProviderCache.hs`
session snapshots and history replay	`src/Leant/Session/Snapshot.hs`, `src/Leant/Session/Replay.hs`
user-facing commands and examples	`README.md`, `docs/Leant_Overview/Leant_Overview.tex`, the transcripts in `test/*.golden`

For Djex, begin with `README.md` and `docs/architecture.md` (with `docs/semantic-foundations.md` for the meaning of evidence and completeness), then the shared `synthesis/src/Language/Haskell/Synthesis` (`Type.hs`, `Kind.hs`, `Query.hs`, `Search.hs`), Djinn's `djinn/src-core/Djinn/Internal/LJT*.hs` and `Core.hs`, and Exference's `exference/src-core/Language/Haskell/Exference/Core/Internal/Exference.hs` and `Options.hs`.

Chapter checkpoint

The REPL layer preserves a live Lean environment and exposes synthesis as one tool among ordinary declarations and proof operations. Debugging is easiest when the pipeline stages are inspected separately. Source ownership is intentionally narrow enough that each semantic boundary has a clear home.

Part V

Worked Traces and Boundaries

Worked Structural Synthesis Traces

This chapter follows representative goals through the conceptual pipeline. The exact debug spelling may evolve, but the semantic stages are fixed by the pinned source. Every block titled "Leant transcript" below is quoted verbatim either from Leant's recorded golden transcripts under `test/` (`synth-basic.golden`, `synth-manual.golden`, ...; `test/run-tests.sh` pipes the matching `.txt` into `leant --plain` and compares) or from a live run of the pinned build; the prose around it is commentary. Verified candidates are printed as `it1`, `it2`, ... (at most five per query, `synthMaxShown` in `src/Leant/Synth/Engine.hs`) and are bound under those names in the session, bare `it` being `it1`.

25.1 Composition

Consider:

Leant query

```
:synth ((A → B → C) → (A → B) → A → C)
```

Lean elaboration

With auto-implicit variables, Lean elaborates A, B, and C as implicit sort variables and the goal as nested Π-types. The value binders are non-dependent arrows.

Fragment

The structural fragment is

$$(A \to B \to C) \to (A \to B) \to A \to C,$$

with A, B, C rigid variables. No opaque atoms, inductives, providers, instances, or bounds are needed.

Djinn formula and proof search

The formula is the same propositional implication shape. Right rules introduce f, g, and x. The atomic goal is C.

The context contains:

$$f : A \to B \to C, \qquad g : A \to B, \qquad x : A.$$

To use f, search must provide A and B. It uses x for A and applies g to x for B. The proof term is equivalent to

$$\lambda f.\lambda g.\lambda x.\ f\ x\ (g\ x).$$

Rendering and verification

The renderer chooses fresh Lean binder names, and Leant prints the Lean-verified result (`test/synth-basic.golden`):

Leant transcript (test/synth-basic.golden)

```
λ> :synth ((A → B → C) → (A → B) → A → C)
  it1  fun f g x => f x (g x)
```

Lean checks the candidate against the exact original goal. Because the fragment was exact and Djinn's unbudgeted search is complete, failure could also have had a strong logical interpretation; in this case a verified positive answer ends the story.

25.2 Swapping a product

For data:

Product swap

```
:synth (A × B → B × A)
```

or for propositions:

Conjunction swap

```
:synth (P ∧ Q → Q ∧ P)
```

both lower to a product-swap shape. The exact nominal families and result sort are retained outside the formula calculus. Djinn's proof term splits the input pair and constructs a pair in reverse order.

The renderer spells that proof with an anonymous-constructor *pattern* on the left and an anonymous constructor on the right, and the pinned build prints the same text for both queries (live run; the identical shape is recorded in `test/synth-manual.golden` for the dependent variant of Chapter 27):

Leant transcript

```
λ> :synth (A × B → B × A)
  it1  fun ⟨x, y⟩ => ⟨y, x⟩
λ> :synth (P ∧ Q → Q ∧ P)
  it1  fun ⟨x, y⟩ => ⟨y, x⟩
```

Neither `Prod.mk` nor `And.intro` is named: Lean resolves the constructor to destructure and the constructor to build from the expected type on each side, so one universe-agnostic spelling serves `Type`-valued pairs and `Prop`-valued conjunctions alike. (A projection form such as `fun p => ⟨p.2, p.1⟩` also type-checks in Lean 4.33 for both goals, but it is not what the renderer emits.) Search shares a logical shape; verification restores the actual Lean family.

25.3 Sum reassociation

Consider:

A sum transformation

```
:synth (A ⊕ (B ⊕ C) → (A ⊕ B) ⊕ C)
```

The formula becomes a constructor-tagged disjunction. A proof must case-analyze the input. The pinned renderer emits one `match` per eliminated sum, so the inner disjunction becomes a nested `match` rather than a nested pattern (live run of the pinned build):

Leant transcript

```
λ> :synth (A ⊕ (B ⊕ C) → (A ⊕ B) ⊕ C)
  it1  fun x => match x with | .inl y => .inl (.inl y) | .inr z => (match z with | .inl w => .inl (.inr w)
   ↪  | .inr x1 => .inr x1)
```

A human would more likely write three flat branches:

Hand-written equivalent (accepted by Lean 4.33)

```
fun s =>
  match s with
  | .inl a => .inl (.inl a)
  | .inr (.inl b) => .inl (.inr b)
  | .inr (.inr c) => .inr c
```

Lean accepts both, and the semantic content is identical: three leaves, each a tagged injection. Constructor descriptors preserved during lowering prevent the branches of unrelated sum-like inductives from being confused.

25.4 A local continuation pattern

The type

Continuation-shaped structural goal

```
((A → B) → A) → (A → B) → B
```

can look uninhabited at first glance, because its first premise is the antecedent of Peirce's law; yet it *is* intuitionistically inhabited: from $k : (A \to B) \to A$ and $f : A \to B$, one derives $k\ f : A$ and then $f(k\ f) : B$. Djinn discovers exactly that term (live run of the pinned build; the renderer names the binders `f` and `g`):

Leant transcript

```
λ> :synth (((A → B) → A) → (A → B) → B)
  it1  fun f g => g (f g)
```

Compare two neighbouring goals recorded in `test/synth-basic.golden`. Peirce's law itself has no constructive proof, so the Lean-checked term Leant shows for it comes from the classical fallback (`:set synth-classical on`, the default), which is entered only after the constructive search has exhausted the formula; and weakening the conclusion to `A → A` makes the first premise irrelevant:

Leant transcript (test/synth-basic.golden)

```
λ> :synth (((A → B) → A) → A)
  it1  Classical.byContradiction (fun k => k (fun f => absurd (fun _ => f (fun x => absurd (fun _ => x)
   ↪  k)) k))
λ> :synth (((A → B) → A) → (A → A))
  it1  fun _ x => x
```

The example illustrates why visual intuition can be unreliable. Systematic proof search explores how assumptions' result atoms can feed one another; Chapter 26 describes when the classical lane runs.

Chapter checkpoint

For exact arrows, products, and sums, the pipeline closely follows Curry–Howard introduction and elimination rules. Lean-specific nominal details ride alongside the formula and return during rendering. Verification confirms that the

reconstructed syntax matches the original family and binder structure.

A Complete Negative Trace

26.1 The target

Consider the polymorphic type:

Leant query

```
:synth (∀ A B : Type, A → B)
```

No assumptions, axioms, bottom values, unsafe operations, or providers are present in the closed goal.

26.2 Rigid quantification

Lean elaborates A and B as explicit type binders. Leant admits them as rigid universally quantified type variables. Rigid means that search cannot unify A with B merely to make the arrow look like identity.

The body is $A \to B$. Right introduction gives a hypothetical $x : A$ and leaves atomic goal B.

26.3 Exhaustive LJT failure

The context contains only $x : A$. There is:

- no hypothesis of type B;
- no implication ending in B;
- no product or sum whose elimination yields B;
- no constructor inventory or provider for arbitrary rigid B;
- no falsity from which to eliminate.

The LJT search space closes without a proof.

26.4 Why this failure can be conclusive

For this goal, the relevant ledger can be exact:

- only sort quantification and one ordinary arrow occur;
- the variables are abstract and rigid;
- no concrete opaque atom conceals an inhabitant;
- no indexed or recursive inductive is involved;
- no instance binder or provider stage is needed;
- Djinn can run unbudgeted.

Leant can therefore report proof-backed uninhabitation for the admitted constructive fragment, rather than merely "no term found." The recorded transcript (`test/synth-basic.golden`, which spells the binders in lower case; the capitalised query of this chapter prints the same line on the pinned build) is:

Leant transcript (test/synth-basic.golden)

```
λ> :synth (∀ a b : Type, a → b)
provably uninhabited — no closed term of this polymorphic type exists
```

Two details of the pinned `src/Main.hs` sharpen the wording. First, the classical fallback (`:set synth-classical`, on by default) runs *after* a sound constructive refutation and *before* anything is printed, so the "provably uninhabited" line also asserts that the excluded-middle and double-negation lanes produced no Lean-verified term; `test/synth-prove.golden` shows the difference on `∀ p : Prop, p ∨ ¬ p`, which is "provably uninhabited" with the fallback off and answered by `fun _ => Classical.em _` with it on. Second, when the goal is a proposition rather than a type, Leant appends the qualifier `(constructively — a classical proof may still exist; this is not a disproof of the proposition)`, because a constructive refutation of a `Prop` says nothing about its classical truth; the type-valued goal above receives no such qualifier, while `:synth (∀ p q : Prop, p → q)` on the pinned build prints both lines.

26.5 What would change the conclusion

Adding any of the following changes the problem:

Extra premises that make the target inhabitable

```
-- A direct provider:
(∀ A B : Type, (A → B) → A → B)

-- Falsity in the context:
False → (∀ A B : Type, A → B)

-- Classical choice plus a nonempty assumption for B:
∀ A B : Type, Nonempty B → A → B
```

The first returns the supplied function; the second eliminates falsity. Both are found by the pinned build (live run):

Leant transcript

```
λ> :synth (∀ A B : Type, (A → B) → A → B)
  it1  fun _ _ f => f
λ> :synth (False → (∀ A B : Type, A → B))
  it1  fun x => nomatch x
```

The third is inhabited in Lean only through the choice axiom: Lean 4.33 accepts `noncomputable example : ∀ A B : Type, Nonempty B → A → B := fun _ _ h _ => Classical.choice h` (and rejects the same term without `noncomputable`, since `Classical.choice` has no executable code). That inhabitant lies outside Leant's search calculus: `Nonempty B` is an opaque atom, and the pinned build answers `the engine proposed 10 candidate(s) but none survived Lean verification` followed by `note: search truncated: candidate limit reached (60)` — a bounded failure, not a refutation. Refutation is always relative to the exact context, and a changed context changes both the answer and the meaning of failure.

Do not confuse parametric impossibility with runtime tricks

A partial language could implement `A → B` by looping forever or throwing an unchecked exception. Lean's safe logical fragment requires total terms. Unsafe or partial executable definitions do not become proofs of arbitrary propositions.

Chapter checkpoint

The negative result works because the type translates exactly, universal variables stay rigid, no hidden structure or premise exists, and complete LJT search exhausts. This is the model case for Leant's strongest negative wording.

Dependent Cargo: What Works Without Dependent Search

27.1 Moving an opaque dependent formula

Declare:

Dependent propositions

```
opaque P : Nat → Prop
opaque Q : Prop
```

and ask for:

Structural use of dependency

```
:synth (((∀ n : Nat, P n) ∧ Q) →
        (Q ∧ (∀ n : Nat, P n)))
```

Let

$$X = \forall n : \mathrm{Nat},\ P(n).$$

Leant seals the entire X as one alpha-aware atom. The formula seen by Djinn is $(X \wedge Q) \to (Q \wedge X)$. A product-swap proof is sufficient.

The recorded transcript (`test/synth-manual.golden`, where the query is written without the outer parentheses) is:

Leant transcript (test/synth-manual.golden)

```
λ> opaque P : Nat → Prop
λ> opaque Q : Prop
λ> :synth ((∀ n : Nat, P n) ∧ Q → Q ∧ (∀ n : Nat, P n))
  it1  fun ⟨x, y⟩ => ⟨y, x⟩
```

The term never inspects the proof of X: the pattern variable `x` is bound to it and returned unchanged in the other field. (A projection spelling such as `fun h => ⟨h.2, h.1⟩` is an equally valid Lean term but is not what the renderer prints.) Lean's expected type confirms that `x` has the exact dependent proposition.

27.2 Instantiating the dependent premise does not work by mere opacity

Now ask for:

Requires dependent application

```
(∀ n : Nat, P n) → P 0
```

The obvious term is:

Human-written term

```
fun h => h 0
```

But the premise `∀ n, P n` is sealed as one atom X and the goal `P 0` as another atom Y, so the propositional formula is $X \to Y$ and no relationship between X and Y is visible. This is exactly what the pinned build does. With `LEANT_SYNTH_DEBUG=1` it prints `debug fragment: FArr (FAtom False "∀ (n : Nat), P n") (FAtom False "P 0")`, both atoms flagged unsafe (`False`) because they mention the constants `P` and `Nat`. Because the goal ends in an opaque atom, the structural lane's failure is followed by provider discovery, which pulls the live `Nat` inventory (`Nat.add`, `Nat.succ`, `Nat.zero`, ...) into a second lane; that lane does not finish within the default wall clock, and the pinned build reports (live run):

Leant transcript

```
λ> :synth ((∀ n : Nat, P n) → P 0)
the engine did not finish within 20s — no answer, not a verdict
(bounded hypothesis instantiation can widen the search a lot; set LEANT_SYNTH_TIMEOUT to another number of
 ↪  seconds, or 0 to wait indefinitely)
```

Raising `LEANT_SYNTH_TIMEOUT` to 150 seconds produced the same message: the missing rule, not the budget, is the obstacle. Moving opaque cargo is sound; opening it to perform dependent application requires a richer rule.

Leant has bounded support for selected quantified type instantiations, especially Haskell-like sort polymorphism. A value-indexed dependent forall over `Nat` is a different feature and should not be inferred from that support.

27.3 Transport is likewise invisible

Consider:

Transport target

```
∀ (α : Type) (n m : Nat),
  n = m → Vector α n → Vector α m
```

A human can write:

Human-written transport

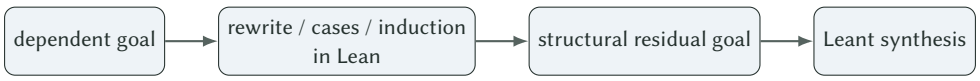
```
fun α n m h v => h ▸ v
```

The essential step eliminates equality into a type family. A propositional search that treats `n = m`, `Vector α n`, and `Vector α m` as unrelated atoms cannot synthesize it. Erasing the indices would be unsound; preserving them opaquely is safe but incomplete. On the pinned build, `:synth (∀ (α : Type) (n m : Nat), n = m → Vector α n → Vector α m)` ends with `the engine did not finish within 20s — no answer, not a verdict` (and likewise at 150 seconds): the equality and the two indexed applications are opaque atoms, and the provider lane that follows the structural failure runs out the wall clock without a verified term.

27.4 Use proof mode to expose structural subgoals

In a larger theorem, ordinary Lean tactics can perform induction or rewriting first. After substituting m for n, the remaining vector term may have exactly the expected type by definitional equality. If the residual goal is a function/product/sum wiring problem, synthesis can then help.

This is the intended collaboration:



A precise capability statement

Leant can often *transport values whose types are dependent* through surrounding structural logic. It generally cannot *reason inside those dependent types* by inventing indices, motives, equality proofs, or transports.

Chapter checkpoint

Opaque cargo supports rearrangement, storage, and application through already exposed premises. It does not reveal dependent elimination rules. Value-indexed foralls, equality transport, and indexed pattern matching remain jobs for Lean tactics or a future richer synthesis calculus.

Chapter 28

Polymorphism, Rank-N Use, and Rendering Variants

28.1 A polymorphic argument

Consider:

Rank-N target

```
((∀ α : Type, α → α) → Nat)
```

The argument f is itself polymorphic. One inhabitant is:

Candidate

```
fun f => f Nat 0
```

The type variable α is nested inside the domain of the outer function, so this is higher-rank use. To produce this particular term, search must instantiate f at `Nat` and then provide `0`. The pinned build does not have to: the goal is also inhabited without touching f, and under the default Djinn engine the first (and only) proof ignores the premise, while Exference's ranked search does instantiate it (live run):

Leant transcript

```
λ> :synth ((∀ α : Type, α → α) → Nat)
  it1  fun _ => .zero
λ> :set synth-engine exference
synth engine: exference
λ> :synth ((∀ α : Type, α → α) → Nat)
  it1  fun f => f _ (.zero)
  it2  fun f => f _ (f _ (.zero))
  it3  fun f => f _ (.succ (.zero))
  it4  fun f => f _ (f _ (f _ (.zero)))
  it5  fun f => f _ (f _ (.succ (.zero)))
note: search truncated: step limit reached, queue limit pruned 33448
note: search truncated: step limit reached, queue limit pruned 31665
```

`f _ (.zero)` is the higher-rank use: the renderer leaves the type argument as `_`, Lean's unifier resolves it to `Nat` from the expected type, and `.zero` is the constructor of the recursive `Nat` occurrence. Under `:set synth-engine both` the Djinn term leads and the Exference terms follow as `it2`–`it5`. The recorded goldens contain the same mechanism where the premise is the only route to the result: in `test/synth-manual.golden`, `:synth ((∀ p : Prop, p → p) → Q → Q)` answers `fun f => f _` (instantiating `p := Q`), and after `axiom Wrap : Type 1 → Type` the impredicative query `:synth ((∀ a : Type 1, Wrap a) → Wrap (∀ b : Type, b → b))` answers `fun x => x _`.

28.2 How the type crosses the boundary

Leant classifies the outer binder as a value arrow. The domain retains an explicit `ForallType` over a rigid type variable, with body $\alpha \to \alpha$. Djex's higher-rank machinery and rigid instantiation plan may expose an application of f at an admitted ground type.

The natural-number result requires a constructor or provider. With the recursive `Nat` declaration represented sufficiently for positive introduction, zero is available: under `LEANT_SYNTH_DEBUG=1` the occurrence appears as `FParamRec True "Nat" "Nat" [] [("Nat.zero",[]),("Nat.succ",[FAtom False "Nat"])]`, a complete two-constructor inventory, which is why `.zero` can be introduced without using f. Positive success is safe even if recursive completeness is bounded, because Lean verifies the concrete term.

28.3 Why a renderer may need variants

The original binder `∀ α : Type, ...` is explicit. A source scheme written with braces would be implicit:

Implicit polymorphic argument

```
({α : Type} → α → α) → Nat
```

Depending on the exact source binder, a candidate application might need `f Nat 0`, `f 0`, or `f (α := Nat) 0`. The engine's abstract application does not by itself encode every surface choice. Leant's side metadata and bounded variant enumerator reconstruct candidates and let Lean choose the valid one. The pinned renderer (`src/Leant/Synth/Render.hs`) uses three concrete spellings, all visible in the goldens: an explicit `∀` slot of a local premise becomes a `_` argument (`f _` above); an implicit slot is skipped unless the engine itself chose the type, in which case a local head is exposed with `@` (`fun _ x => @x (∀ (a0_0 : Type _), a0_0 → a0_0)` for the goal `(∀ x : Type, x → x) → ({a : Type 1} → Demo.Token) → Demo.Token` in `test/synth-quantified-provider.golden`); and a global provider receives a named argument (`Demo.global («a» := Nat)` in `test/synth-djinn-providers.golden`). On the pinned build the implicit-binder variant of this chapter's goal, `:synth (({α : Type} → α → α) → Nat)`, answers `fun _ => .zero` under Djinn, exactly like the explicit one.

28.4 Ambiguity weakens negative evidence

If positive search finds a semantic application but every reconstruction variant fails, no verified candidate is shown. Conversely, a bounded instantiation plan that finds no semantic application does not prove none exists. Higher-rank positive support and higher-rank complete refutation are different capabilities.

Chapter checkpoint

Djex's source language can represent selected higher-rank type schemes even though it is not dependently typed. Leant records binder visibility and may enumerate application spellings. Positive candidates are verified; bounded instantiation and reconstruction prevent broad negative conclusions.

Provider-Guided and Library-Guided Synthesis

29.1 A global conversion function

Imagine a live Lean environment containing:

Environment declarations

```
axiom A : Type
axiom B : Type
axiom convert : A → B
```

(`axiom` rather than `opaque`: Lean 4.33 rejects `opaque convert : A → B` with `failed to synthesize 'Inhabited' or 'Nonempty' instance for A → B`, because a body-less `opaque` needs an inhabitant to compile against; the fixtures in `test/` use `axiom` for the same reason.)

A query for `A → B` has no closed structural inhabitant if A and B are unrelated atoms. If provider discovery admits `convert`, the search environment gains a premise of exactly that type, and the pinned build answers with the provider itself, eta-reduced (live run):

Leant transcript

```
λ> :synth (A → B)
  it1  convert
```

The candidate's origin records that `convert` was a live provider, not a local lambda binder. The renderer restores its exact global name; Lean resolves the declaration in the current environment and checks the application. `test/synth-djinn-providers.golden` records the same mechanism for a nullary provider (`:synth Demo.Seed` answers `Demo.seedValue`) and for a two-step chain (`:synth (Demo.Input Demo.Index → Demo.Output Demo.Index)` answers `fun x => Demo.consume (Demo.produce x)`).

29.2 Polymorphic providers

Suppose:

A polymorphic provider

```
opaque mapBox {α β : Type} : (α → β) → Box α → Box β
```

For a goal `(A → B) → Box A → Box B`, provider matching must instantiate $\alpha := A$ and $\beta := B$. The exact type-constructor applications `Box A` and `Box B` must retain both nominal head and arguments; flattening them to unrelated text or a single `Box` atom would break unification.

On the pinned build, with `axiom Box : Type → Type` and `axiom mapBox {α β : Type} : (α → β) → Box α → Box β` declared, the explicitly quantified query answers at once, and Lean infers the implicit type arguments (live run):

Leant transcript

```
λ> :synth (∀ A B : Type, (A → B) → Box A → Box B)
  it1  fun _ _ z0 z1 => mapBox z0 z1
  it2  fun _ _ f x => mapBox (fun y => y) (mapBox f x)
note: search truncated: candidate limit reached (60)
```

The auto-bound spelling `:synth ((A → B) → Box A → Box B)`, by contrast, ran out the wall clock on the same build (`the engine did not finish within 20s — no answer, not a verdict`, and again at 150 seconds): the auto-bound variables enter the search Sort-polymorphic, and the bounded hypothesis-instantiation plan that widens is the one the timeout message warns about. Writing the type binders explicitly is the practical remedy.

29.3 Class-constrained providers

A provider might require `[C α]`. Lean's provider scanner can ask the live instance resolver for complete assignments and preserve structured exact context metadata. If a valid assignment exists, a rendered term may leave the dictionary implicit. `test/synth-djinn-providers.golden` shows the shape: after `class Demo.C (a : Type) : Prop where witness : True`, `instance : Demo.C Nat := ⟨True.intro⟩`, and `axiom Demo.global {a : Type} [Demo.C a] : Demo.Token`, the query `:synth (Nat → Demo.Token)` under Djinn answers `fun _ => Demo.global («a» := Nat)`. The type argument is spelled out because the scanner found the assignment `a := Nat`; the `[Demo.C Nat]` dictionary is left to Lean's instance resolution.

Failure to find one within the bounded scan does not prove the provider unusable in every possible elaboration context. The completeness ledger records that limitation.

29.4 Library recursion as a provider

Suppose a user wants a list transformation that can be expressed by an existing combinator such as `List.map`, `List.foldr`, or a curated recursor. Library mode (`:set synth-library`, on by default) can make the combinator a premise. Search then synthesizes its arguments rather than inventing recursion. `test/synth-library.golden` records both settings:

Leant transcript (test/synth-library.golden)

```
λ> :synth ((a → b) → List a → List b)
  it1  fun f x => List.map f x
  it2  fun f x => List.reverse (List.map f x)
  it3  fun f x => List.map (fun y => y) (List.map f x)
  it4  fun f x => List.map (fun y => y) (List.reverse (List.map f x))
  it5  fun f x => List.append (List.map f x) (List.map f x)
note: search truncated: candidate limit reached (60)
λ> :synth ((a → b → b) → b → List a → b)
  it1  fun _ x _ => x
  it2  fun f x y => List.foldr f x y
  it3  fun f x y => List.foldl (fun _ a => f a x) x y
note: search truncated: candidate limit reached (60)
```

Leant transcript (test/synth-library.golden, continued)

```
λ> :set synth-library off
synth library: off
λ> :synth ((a → b) → List a → List b)
  it1  fun _ _ => .nil
```

With the library premises removed, the only thing the structural engine can build for a `List b` is the constructor `.nil`: type-correct, verified, and useless. Every candidate in the first block is likewise type-correct; only `it1` is the map a user

probably wanted. That gap between "has the type" and "does what was meant" is the subject of Chapter 30.

This decomposition is powerful:

$$\text{recursive algorithm invention} \quad \Longrightarrow \quad \text{application synthesis over a trusted recursive combinator.}$$

The final term remains ordinary Lean code, and its provider use is checked.

29.5 Why results depend on configuration

Provider and library settings change the search environment, so they change both positive candidates and the meaning of failure. A query that is uninhabited in pure intuitionistic structure may become inhabited after enabling a provider, a classical theorem, or a recursor.

A reproducible synthesis report should therefore include:

- Lean environment/import state;
- provider and library modes;
- engine and resource settings;
- quantifier/recursive policies;
- exact verified candidate.

Chapter checkpoint

Providers bridge imported Lean functionality into the search calculus. Exact names, schemes, type applications, constraints, and environment identity are preserved. Curated recursion converts hard recursive invention into ordinary combinator application. Configuration is part of the query contract.

The Optional Length/Z3 Behavioral Layer

30.1 Why types are not enough

For `List α → List α`, structural synthesis returns several verified terms. On the pinned build (live run, default settings):

Leant transcript

```
λ> :synth (List α → List α)
  it1  fun x => x
  it2  fun x => List.reverse x
  it3  fun x => List.append x x
  it4  fun x => List.map (fun a => a) x
  it5  fun x => List.flatMap (fun _ => x) x
note: search truncated: candidate limit reached (60)
```

Suppose the user additionally wants output length to equal input length. `it1`, `it2`, and `it4` satisfy it; `it3` doubles the length and `it5` squares it (and `fun _ => .nil`, which the library-free search of Chapter 29 produces, sends everything to zero). That property is not encoded in the type, so ordinary Lean verification cannot separate these candidates.

Djex contains an experimental semantic Length layer for a narrowly defined class of list-spine and product problems. Leant can optionally use it to assess already type-correct Exference candidates.

30.2 The authority order

The order is essential:

1. synthesize a candidate through the ordinary engine;
2. render and verify it against the exact Lean type;
3. retain its exact typed candidate graph and inventory sidecar;
4. seal a supported Length contract and target;
5. symbolically interpret the graph under an explicit provider-law basis;
6. encode the resulting bounded arithmetic problem as canonical QF_LIA SMT-LIB;
7. run Z3 through a resource-bounded process/session boundary;
8. independently replay any model or claimed bounded-positive result against the original checked semantic problem;
9. issue only the narrow receipt or ranking decision authorized by that replay.

Z3 never makes an ill-typed term well typed. Lean verification happened first. Conversely, Lean type verification does not prove the Length postcondition; that is a separate, explicitly bounded semantic claim.

30.3 Sealed contracts and typed graphs

The behavioral layer does not reparse the rendered Lean string. It consumes an engine-owned typed term graph tied to:

- the exact synthesis session and inventory;
- provider semantics admitted for Length interpretation;
- the normalized target type and contract;
- the candidate graph fingerprint;
- resource and domain policies.

This prevents a textually identical term from inheriting a receipt produced under another inventory or contract.

30.4 Symbolic length interpretation

Supported term-graph nodes are interpreted as arithmetic expressions over input lengths. A provider such as identity may have law $\ell_{out} = \ell_{in}$; constant empty has $\ell_{out} = 0$; append has $\ell_{out} = \ell_1 + \ell_2$ under a sealed provider law.

Unsupported nodes fail closed. The interpreter does not guess semantics from function names. Provider laws belong to the checked session inventory.

30.5 SMT solving and replay

The arithmetic problem is encoded in quantifier-free linear integer arithmetic. A satisfiable counterexample query may produce concrete natural input lengths violating the postcondition. Djex re-evaluates those inputs through its own checked semantic problem before returning a validated counterexample.

An unsatisfiable solver response is not automatically a proof. Bounded-positive receipts require the exact applicable domain and replay policy recorded by the current API. The semantic documentation repeatedly emphasizes that raw solver status, bytes, or cached samples grant no independent authority.

30.6 What the layer can and cannot say

It can help rank or filter candidates under a supported length contract and bounded domain. It cannot by itself establish:

- extensional equality of arbitrary Lean functions;
- element preservation or permutation;
- correctness outside the sealed domain;
- semantics of unsupported providers;
- a kernel theorem unless a separate proof-producing bridge is built.

Behavioral checks are orthogonal to type checking

A candidate may be type-correct and behaviorally rejected; another may be behaviorally acceptable under a bounded contract but still rely on axioms or providers the user dislikes. Leant keeps these judgments separate so the user can see which property was established by which mechanism.

30.7 Activation, modes, and the rejection rule in the pinned tree

The layer is off unless a startup policy is activated, and even then it changes the ordering of candidates by default and their presence only on explicit request. The facts below are those of `docs/length-ranking.md` and `src/Main.hs` at `49c8f9f100a6` (the current tree, inspected for this guide, is the same on every point):

- **Activation.** `leant --length-ranking-config /absolute/path.json` reads one versionless configuration file at startup (root member `rankingDomain` is `"scalar"` or `"binary-product"`; an executable SHA-256 expectation is required unless `--length-ranking-allow-unpinned` is given; `--length-ranking-config-timeout` bounds only the file load, default 5 000 ms). Without it, ranking is the lazy identity and no worker is ever launched. File acquisition is POSIX-only; a Windows build fails closed at startup.
- **Eligibility.** Only candidates with typed Exference authority are assessed. The default `djinn` engine supplies no typed graph, so under it every candidate is retained with a payload-free preparation refusal; use `:set synth-engine exference` or `both`.
- **Command grammar.** `:synth TYPE` ranks with the activated startup contract. The option-bearing forms are `:synth [--behavior-mode rank|filter] [--length-contract ABSOLUTE-PATH] -- TYPE`: the mode, if present, precedes the contract, and the standalone `--` is mandatory once either option appears. A contract file supplies a law for one command; it never supplies execution authority, and both filter mode and a contract path are refused before any file IO when no startup policy is active (`finite-spine Length request rejected before IO: ...`).
- **Ranking never prunes.** In `rank` mode a replayed counterexample demotes its candidate behind the retained ones, but the candidate is still shown and still bound as `itN`, with the counterexample note beneath it.
- **Only a replayed counterexample rejects.** In `--behavior-mode filter`, the closed taxonomy of `src/Leant/Synth/Length/Selection.hs` retains preparation refusals, unassessed candidates, raw `sat`/`unsat`/`unknown` statuses, finite-box receipts, and applicable-domain receipts; the only rejecting report is a counterexample that Djex has independently re-evaluated against the candidate's own checked problem. Rejected occurrences are printed as separate `rejected` rows and receive no `itN` binding.
- **Command-local bank and progressive batches.** A filter run opens one counterexample bank per `:synth` command, reuses it across that command's lanes and across at most two ordered batches drawn from one lazy engine outcome (a second batch is requested only after no verification or complete all-rejection), and forgets it afterwards: nothing is retained in the session state, history, or snapshots. Every replayed sample must still be re-evaluated against the later candidate before it counts.
- **Symbols.** Each eligible candidate becomes a canonical QF_LIA query whose input symbols are `djex_length_input_0`, `djex_length_input_1`, … in source-position order (internal spellings chosen by Djex's Length translator, not a public API), asserted nonnegative, followed by the bad-state assertion, `check-sat`, and `get-value` over exactly those symbols.

What is *not* in the pinned tree, and should not be read into it: counterexample-directed generation of new candidates, filling a survivor quota across lanes, sound pruning of typed prefixes, a session-persistent bank, further behavioral domains, or Lean-checked proof artifacts.

Source trail

The complete reference for this stage is Leant's `docs/length-ranking.md` (startup configuration schema, command grammar, retention/rejection taxonomy, progressive batching, presentation notes); `docs/synth-internals.md` summarizes the semantic-origin and typed-candidate invariants it relies on, and the companion guide `docs/Z3_for_Leant_and_Djex/Z3_for_Leant_and_Djex.tex` teaches the SMT background and traces the same stage module by module. The Djex guide and semantic foundations at the pinned submodule revision describe typed candidates, sealed contracts, QF_LIA queries, bounded Z3 sessions, and independent replay. The standalone Djex main revision inspected here differs from the vendored one by a single documentation-only commit; later refinements should not be attributed to Leant until its submodule advances.

Chapter checkpoint

The Length layer is post-type-check behavioral analysis over sealed typed candidates. It uses Z3 as a bounded reasoning component and independently replays results. Its receipts have narrow contract-specific authority and are not Lean proofs of general program correctness.

Chapter 31

A Reliable Mental Model for Using and Extending Leant

31.1 Ask four questions about every result

When Leant shows a candidate or a failure, ask:

1. **What did Lean elaborate?** Surface notation and auto-implicit variables may hide the exact goal.
2. **What did the fragment expose?** Which pieces became logic, applications, inductives, providers, or opaque atoms?
3. **What did the engine prove or search?** Djinn formula proof, Exference bounded candidate, or both?
4. **What did the final checker authorize?** Lean type acceptance, negative completeness evidence, or a separate behavioral receipt?

These questions prevent most category errors.

31.2 Read the message literally

Leant's final line for a `:synth` query is chosen by `src/Main.hs` from a small fixed vocabulary, and each phrase was written to answer the four questions above. The short version, which Appendix D expands into a table with next steps:

- `itN TERM` rows: Lean accepted the exact term against the exact goal; nothing more. A trailing `note: search truncated: ...` means the engine stopped at a bound, so absent alternatives are not evidence.
- `provably uninhabited — no closed term of this polymorphic type exists`: an exact translation, a complete Djinn search, and (by default) an empty classical retry. On a `Prop` goal the second line `(constructively — ...)` reminds you that this is not a disproof.
- `no term found within bounds (opaque atoms '...' hide structure ...)` and `no term found within the search bounds`: unknown, and the message names why (unsafe atoms, or bounded search).
- `the engine proposed N candidate(s) but none survived Lean verification`: the search side succeeded, the rendering or elaboration side did not; the negative is about the reconstruction route.
- `the engine did not finish within Ns — no answer, not a verdict`: a wall clock, not a search state; `LEANT_SYNTH_TIMEOUT` changes it.
- `out of fragment: ...` and `cannot translate this goal:`: the query never reached an engine.

31.3 Classify goals before searching

A useful manual classification is:

Pure structural
: Arrows, products, sums, polymorphic plumbing. Excellent `:synth` targets.

Provider structural
: Requires a known library function but little semantic reasoning. Enable provider/library search.

Opaque dependent cargo
: Dependent formulas are only moved or stored. Often works.

Dependent elimination
: Requires instantiation at a value, indexed cases, transport, or a motive. Use Lean tactics/proof mode; synthesis may help later.

Recursive invention
: Requires choosing a recursion scheme and branch behavior. Prefer curated combinators or explicit design.

Behaviorally underdetermined
: Many inhabitants share the type. Add a theorem, refinement, examples, or a supported semantic contract.

31.4 Extend at the correct layer

New functionality belongs in different places depending on its semantic role:

- recognize another exact Lean connective: serializer/fragment policy;
- support another Haskell-like type form: Djex shared source representation and both engine adapters;
- add a proof-search rule: Djinn calculus, proof terms, and proof checker;
- add a heuristic expression form: Exference nodes, unification, checking, and ranking;
- reconstruct a known engine term in Lean: renderer plus verification tests;
- strengthen negative claims: completeness ledger and exactness proof, not only more search;
- add a behavioral property: typed graph semantics, sealed contract, solver/certificate/replay boundary;
- support general dependent synthesis: a new calculus and term representation, not an extra opaque-atom case.

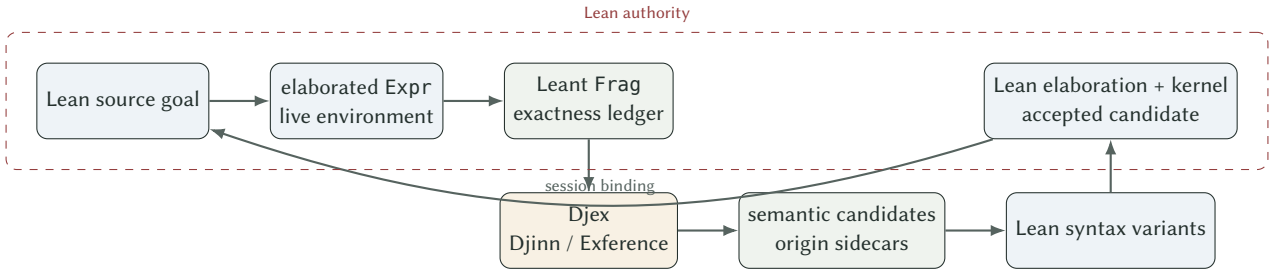
31.5 Prefer fail-closed boundaries

At every translation, ask what happens when metadata is missing or a bound is exceeded. The robust answer is usually:

- keep the subtree opaque for positive transport;
- mark negative evidence unsafe or incomplete;
- reject malformed protocol data;
- drop an unrenderable candidate;
- let Lean reject ambiguous syntax;
- report bounded failure as unknown;
- refuse unsupported behavioral interpretation.

This style may miss answers, but it prevents an optimization or convenience path from acquiring proof authority accidentally.

31.6 The compact architectural picture



The dashed region is not saying the entire Lean implementation is formally verified. It indicates where the final type-authority operation occurs. Djex and the Haskell controller are outside that authority and are made safe for positive answers by the return path through Lean.

Chapter checkpoint

Classify the goal, inspect the fragment, distinguish engine contracts, and identify the final authority. Extend the layer that owns the missing semantics. Preserve fail-closed behavior when exactness cannot be established. This mental model is sufficient to navigate both the codebase and user-facing results.

Conclusion

Lean's apparent complexity comes from one coherent decision: the type language is expressive enough for types to mention values, propositions to be types, and proofs to be programs. The Calculus of Constructions supplies the dependent function core; inductive families supply data, recursion, equality, and induction; universes and termination keep the system consistent; elaboration lets humans write concise source; the kernel reduces the result to a small checked expression language.

Leant becomes straightforward once that stack is visible. It does not replace Lean's type theory with Djex. It asks Lean to project a goal into an explicitly smaller structural language. Djinn can decide an exact intuitionistic fragment; Exference can search more flexibly under resource bounds. Unsupported dependent structure is retained as opaque cargo, not erased. Candidate terms return through a renderer that is allowed to be wrong because Lean checks every answer. Negative claims are rarer because they need a complete translation and search ledger. Optional behavioral analysis remains a separate, sealed judgment.

That architecture yields a useful division of labor:

Component	Responsibility
Lean elaborator	interpret concise source in the exact live environment
Lean kernel	authorize typing, universes, conversion, inductives, and safety
Leant fragment	state what structure is exposed and what completeness is lost
Djinn	construct or refute proofs in an exact propositional calculus
Exference	search ranked typed expressions under explicit limits
Leant renderer	reconstruct candidate Lean syntax and bounded variants
Verification loop	return every positive answer to Lean
Behavioral layer	assess a separate sealed property without borrowing type authority

The shortest durable summary is therefore:

Rich source, restricted search, authoritative return

Leant is useful not because Djex understands all of Lean, but because Leant knows exactly when it does not, preserves that boundary as data, and sends every proposed inhabitant back to Lean.

Lean Syntax and Logic Cheat Sheet

A.1 Declarations

Form	Meaning
`def f (x : A) : B := t`	transparent named definition checked at type B
`theorem h : P := p`	opaque checked proof of proposition P
`example : T := t`	check a temporary unnamed declaration
`opaque c : T`	introduce a constant whose body is hidden from reduction; without an explicit `:= body`, Lean requires an `Inhabited` or `Nonempty` instance for T (`opaque c : Nat` is accepted; `opaque f : A → B` for abstract `A`, `B` without instances is rejected)
`opaque c : T := t`	opaque constant with a stated body; `c` still does not unfold to `t`
`abbrev N := T`	reducible definition, unfolded eagerly by elaboration and instance search
`noncomputable def f ... := t`	definition that may use `Classical.choice`; no executable code is generated
`variable (x : A)`	declare a section-wide binder that later declarations may mention
`universe u`	declare a universe-level name for use in `Sort u` / `Type u`
`deriving Repr, DecidableEq`	ask Lean to generate standard instances for a declaration
`axiom a : T`	assume a constant of type T without a body
`inductive I ... where ...`	declare an inductive family and constructors
`structure S ... where ...`	one-constructor inductive with named fields
`class C ... where ...`	structure designated for instance search
`instance : C A := ...`	register a term for type-class resolution
`namespace N ... end N`	qualify enclosed names
`open N`	make namespace members available for resolution/printing
`import M`	load a compiled module and its environment declarations

A.2 Binders

Syntax	Kind	Caller behavior
`(x : A)`	explicit	normally supplied
`{x : A}`	implicit	normally inferred
`{{x : A}}`	strict implicit	inferred only once a later explicit argument is supplied (Lean also accepts the Unicode brackets U+2983/U+2984 around `x : A`)
`[x : C]`	instance implicit	synthesized by type-class resolution
`∀ x : A, B x`	dependent binder	proposition/function according to codomain sort

A.3 Term forms

Syntax	Reading
`fun x => t`	lambda abstraction
`f x y`	left-associated application

Syntax	Reading
`A → B → C`	right-associated function type
`let x := a; t`	local definition
`match s with | ...`	pattern matching, elaborated to recursors
`⟨a, b⟩`	anonymous constructor selected by expected type
`p.1, p.2, p.field`	projections
`_`	elaboration metavariable / placeholder
`@f`	expose normally implicit arguments of f at the surface
`f (α := A)`	named explicit assignment to an implicit parameter
`by ...`	tactic script constructing a term
`rfl`	reflexivity term/tactic, succeeds up to definitional equality
`h ▸ x`	transport/rewrite x along equality h
`nomatch h`	eliminate an impossible hypothesis `h` (e.g. of type `False` or `Empty`) with no cases
`h.elim, absurd hp hn`	`False.elim`/`Empty.elim` of `h`; `absurd` takes a proof and its negation and inhabits anything
`.inl a, .none, .zero`	dot-constructor resolved from the expected type (Leant's renderer uses this spelling)
`⟨w, proof⟩`	anonymous constructor for `∃`, `Subtype`, and structures alike
`show T from t`	state the expected type of a subterm explicitly

All forms in these tables were checked against Lean 4.33.0 with core only.

A.4 Logical forms

Lean	Constructor/eliminator intuition	Search shape
`P → Q`	lambda / application	implication
`P ∧ Q`	pair / projections	conjunction
`P ∨ Q`	injection / cases	disjunction
`True`	unique constructor	top
`False`	no constructor / eliminate anywhere	bottom
`¬ P`	function $P \to$ False	implication to bottom
`∀ x : A, P x`	dependent lambda/application	full dependency; only selected fragments exposed
`∃ x : A, P x`	witness + proof / dependent cases	dependent inductive in `Prop`
`x = y`	`Eq.refl` / transport	indexed inductive
`A × B`, `A ⊕ B`	pair / projections; injection / cases	product / sum, same search shape as `∧` / `∨`
`Unit`, `Empty`	`()` / eliminate anywhere	top / bottom for data
`Decidable P`	`isTrue h` / `isFalse h`	two-constructor inductive; `:synth` builds and cases on it
`Nonempty A`	`Nonempty.intro` / `Classical.choice`	opaque atom for `:synth`

A.5 Inspection commands useful around Leant

Command	Purpose
`#check t`	elaborate t and print its type
`#eval t`	compile/evaluate a computable expression
`#reduce t`	reduce an expression through the elaborator/kernel reduction facilities
`#print name`	print a declaration

Command	Purpose
`#print axioms name`	report axiomatic dependencies
`set_option pp.all true`	expose more elaborated detail in pretty printing
`set_option trace.Meta.synthInstance true`	inspect type-class search (version/options permitting)

Compact Formal Rules

This appendix collects a schematic core sufficient to decode the theoretical discussion. It is not a complete formal specification of Lean's kernel, inductive checker, quotient primitives, safety classes, or optimized conversion algorithm.

B.1 Contexts and sorts

Context extension

$$\frac{\Gamma \ \mathrm{ctx} \qquad \Gamma \vdash A : \mathrm{Sort}\ u \qquad x \notin \Gamma}{\Gamma, x : A \ \mathrm{ctx}}$$

Sort formation

$$\Gamma \vdash \mathrm{Sort}\ u : \mathrm{Sort}(u+1).$$

Lean additionally validates that every named universe parameter is declared and that no level metavariable reaches the kernel.

B.2 Dependent products

Π formation

$$\frac{\Gamma \vdash A : \mathrm{Sort}\ u \qquad \Gamma, x : A \vdash B : \mathrm{Sort}\ v}{\Gamma \vdash \Pi(x : A), B : \mathrm{Sort}(\mathrm{imax}(u, v))}$$

Lambda

$$\frac{\Gamma, x : A \vdash t : B}{\Gamma \vdash \lambda(x : A), t : \Pi(x : A), B}$$

Application

$$\frac{\Gamma \vdash f : \Pi(x : A), B \qquad \Gamma \vdash a : A}{\Gamma \vdash f\ a : B[a/x]}$$

B.3 Local definitions

Let

$$\frac{\Gamma \vdash A : \mathrm{Sort}\ u \qquad \Gamma \vdash a : A \qquad \Gamma, x : A \vdash t : B}{\Gamma \vdash (\mathrm{let}\ x : A := a;\ t) : B[a/x]}$$

Zeta reduction makes the let expression definitionally equal to $t[a/x]$.

B.4 Conversion

Definitional conversion

$$\frac{\Gamma \vdash t : A \qquad A \equiv B \qquad \Gamma \vdash B : \mathrm{Sort}\ u}{\Gamma \vdash t : B}$$

Lean's implemented $\equiv$ includes reduction and congruence, proof irrelevance, function eta expansion, eta for suitable nonrecursive structures, literal reduction, and environment-guided lazy unfolding.

B.5 Dependent pairs

A schematic Σ rule is:

Σ introduction

$$\frac{\Gamma \vdash a : A \qquad \Gamma \vdash b : B[a/x]}{\Gamma \vdash \langle a, b \rangle : \Sigma(x : A), B}$$

Lean implements `Sigma` as an inductive family rather than a primitive `Expr` constructor. Its eliminator is therefore generated by the inductive machinery.

B.6 Equality elimination

A schematic dependent eliminator is:

$$\mathrm{Eq.rec} : \Pi\{\alpha : \mathrm{Sort}\ u\}\{a : \alpha\}(P : \Pi b : \alpha, a = b \to \mathrm{Sort}\ v), P\ a\ \mathrm{rfl} \to \Pi\{b : \alpha\}(h : a = b), P\ b\ h.$$

Simpler transport is derived by choosing a motive that ignores the equality proof. Pattern matching and rewriting elaborate to applications of such eliminators.

B.7 Inductive declarations

A full rule must check at least:

- constructor types are well formed in declared universes;
- constructor results target the declared inductive family with correct parameters;
- recursive occurrences satisfy positivity;
- universe and elimination constraints are respected;

- mutual blocks are coherent;
- generated recursors type check.

Lean's kernel compiles the accepted block into ordinary constants and recursors. User pattern matching is then elaborated to those constants.

Lean-to-Leant Translation Reference

Lean shape	Leant/Djex treatment	Consequence
non-dependent $A \to B$	`FArr`, then function/implication	fully structural when children exact
`And P Q`	product/conjunction plus nominal restoration	Djinn splits/builds pair
`Prod A B`, `PProd A B`	Lean product fragment plus nominal restoration	same logical search shape, exact expected-type check
`Or P Q`, `Sum A B`, `PSum A B`	tagged sum	injections and case analysis
`Iff P Q`	pair of implications	exact logical expansion
`Not P`	$P \to \bot$	exact definitional/logical expansion
true/unit families	top/unit	immediate constructor
false/empty families	nominal bottom	no introduction; elimination if premise exists
sort-bound forall	`FAll`, rigid Djex forall	selected polymorphism and bounded rank-N use
non-dependent value forall	arrow	ordinary lambda/application
value-dependent forall	opaque atom unless dedicated case	can be moved; not generally instantiated
instance-implicit forall	`FInst` or exact provider context	positive elaboration possible; negative evidence guarded
type-valued nominal application	`FApp` retaining head and args	higher-kinded unification/provider matching
term-indexed application	opaque identity	no index-sensitive reasoning
complete nonrecursive nonindexed inductive	constructor-tagged sum of products	construction and case analysis
indexed inductive	opaque	no dependent constructor refinement
constructor with dependent field	opaque	no dependent pair elimination
recursive inductive	bounded recursive fragment	limited positive support; completeness guarded
incomplete declaration inventory	partial fragment/opaque downgrade	positive candidates checked; no strong negative
traversal fuel exhausted	depth marker/completeness downgrade	failure means unknown
concrete unsupported constant	unsafe opaque atom for refutation	may hide a known inhabitant
abstract universally quantified subtree	possibly refutation-safe opaque atom	parametric structural reasoning may remain complete

The pinned `Frag` constructors

The Haskell side of the boundary is the `Frag` type in `src/Leant/Synth/Fragment.hs` (lines 139–202 at `49c8f9f100a6`). Its module header states what the Lean-side serializer decomposes: `→`; `And`/`Prod`/`PProd`; `Or`/`Sum`/`PSum`; `Iff` and `Not` (lowered by the serializer to a pair of arrows and an arrow to bottom); `False`/`Empty`/`PEmpty`; `True`/`Unit`/`PUnit`; non-dependent and sort-quantified foralls. Everything else becomes an opaque atom keyed by its pretty-printed spelling. `LEANT_SYNTH_DEBUG=1` prints the parsed value as `debug fragment: ...`.

Constructor	Meaning
`FArr Frag Frag`	non-dependent arrow / implication
`FProd Frag Frag`	`And`/`PProd`-style pair; `FLeanProd Frag Frag` is a saturated canonical `Prod` (same search shape, exact expected-type check)
`FSum Frag Frag`	`Or`/`Sum`/`PSum`
`FTop, FBot`	unit-like and empty-like families
`FAll Bool String Frag`	forall over a sort; the flag is `True` for an explicit binder (a lambda in the rendered term) and `False` for an implicit one
`FInst String Frag`	non-dependent instance-implicit binder; the dictionary is ignored by search, bound as a wildcard by the renderer
`FExactContext String [ExactContextArgument] Frag`	exact class-constraint context of a provider scheme or instance assignment (semantic evidence, not pretty text)
`FVar String`	opaque type variable (auto-implicit or opened binder)
`FAtom Bool String`	opaque atom; the flag is the refutation-safety bit (`False` once the atom mentions any constant)
`FApp Bool String AppHead [Frag]`	proper-type application retaining head (bound variable or rigid Lean constant) and arguments
`FParamInd, FInd`	non-recursive inductive occurrence with its complete constructor inventory (parametric family, or occurrence-local form)
`FParamRec Bool ..., FRec Bool ...`	recursive occurrence with a bounded constructor description; the flag records whether every Lean constructor was seen
`FDepth`	the translator's depth bound was reached; `fragRefusal` then answers `the goal exceeds the translator's depth bound`

`fragRefusal` in the same file also refuses a goal that is a single opaque atom (`:synth handles →/×/⊕/∀/⊥/⊤ over opaque variables; dependent or non-propositional structure is transported, never analyzed`) or a single opaque type application, unless provider discovery can open it; `src/Main.hs` prints these as `out of fragment: ...`.

Interpreting Leant Outcomes

The first table lists the messages the pinned `src/Main.hs` and `src/Leant/Synth/Engine.hs` actually print at the end of a `:synth` query (each phrase was checked against the source and, where marked, against a recorded golden or a live run), what each one establishes, what it does not, and what to try next. The second table restates the same discipline for the internal evidence states behind those messages.

Messages, meaning, and next steps

Message	Meaning	Does not mean	Try next
`it1 TERM` (… `it5`)	Lean elaborated `TERM` against the exact goal in the live environment; it is bound as `itN` (`it` = `it1`)	intended behavior, efficiency, uniqueness, or that the list is complete	`#eval it1 ...`, `#check it2`; `:set synth-engine both` for Exference's ranked alternatives
`note: search truncated: candidate limit reached (60)` / `step limit reached` / `choice-point limit reached` / `queue limit pruned N` / `depth limit pruned N` / `identifier space exhausted`	the engine stopped at a bound (`candidateWindow = 60`, `:set synth-steps`, Djinn's choice points, Exference's queue/depth caps)	anything about candidates outside the explored prefix	`:set synth-steps N` (default 4096); rephrase the goal; enable a provider or library premise
`note: exference: search truncated: ...`	under `:set synth-engine both`, the Exference lane was bounded while Djinn's result stands	the Djinn candidates are affected	as above
`provably uninhabited — no closed term of this polymorphic type exists`	exact translation, refutation-safe atoms, unbudgeted Djinn exhaustion, and (with `synth-classical on`, the default) an empty classical retry (goldens: `∀ a b : Type, a → b`)	that an import, axiom, or provider outside the admitted contract could not inhabit the type	re-read the goal; add the premise you believe is missing and ask again
`(constructively — a classical proof may still exist; this is not a disproof of the proposition)`	printed under the previous line for `Prop` goals only	disproof of the proposition	`:set synth-classical on` if it was off; otherwise prove it classically by hand
`no term found within bounds (opaque atoms '...' hide structure the engine cannot analyze — not a refutation)`	Djinn exhausted the formula, but an atom mentions a constant, so absence is not sound	uninhabitation	expose the structure (define the constant, unfold, or move to `:prove`)
`no term found within the search bounds`	no lane produced a verified term and no lane could refute	uninhabitation	raise `:set synth-steps`; try `:set synth-engine both`; add binders explicitly

Message	Meaning	Does not mean	Try next
`the engine proposed N candidate(s) but none survived Lean verification`	semantic candidates existed; every rendered variant was rejected by Lean (live: `∀ A B : Type, Nonempty B → A → B`)	uninhabitation; a Lean bug	`LEANT_SYNTH_DEBUG=1` to see the fragment and variants; add explicit type annotations
`the engine did not finish within Ns — no answer, not a verdict`	the wall clock (`LEANT_SYNTH_TIMEOUT`, default 20) expired, typically inside bounded hypothesis instantiation or a provider lane	search exhaustion	`LEANT_SYNTH_TIMEOUT=0` or a larger number; quantify the type variables explicitly (`∀ A B : Type, ...`)
`out of fragment: the goal is a single opaque atom '...' / ... single opaque type application '...' / ... exceeds the translator's depth bound`	`fragRefusal` in `Fragment.hs`: no structure reached an engine and no provider could open the goal	anything about inhabitation	state a structural goal; declare a provider; use `:prove`
`cannot translate this goal:` + Lean errors; `(retrying with a b : Type — Sort-polymorphic elaboration was stuck); hint: annotate the variables' types yourself`	the goal did not elaborate, or auto-bound variables were retried at `Type`	anything about inhabitation	fix the Lean error; write `:synth (∀ a b : Type, ...)`
`synthesis engine error: ...`	a Djex-side failure (diagnostic text follows)	anything about inhabitation	report with `LEANT_SYNTH_DEBUG=1` output
`(synthesizing with hypotheses p q h hp as premises)`	inside `:prove`, the visible hypotheses were added as premises and `itN` splices name them	that the term is closed	`exact it1`
`rejected TERM` + note	only under `--behavior-mode filter`: an independently replayed Length counterexample; the row is shown but not bound	ill-typedness; a general semantic verdict; anything under `rank` mode	inspect the note's input lengths; tighten the contract
`warning: finite-spine Length behavioral assessment preserved all verified candidates: ...`	the Length stage failed closed and changed nothing	anything about the candidates' behavior	see `docs/length-ranking.md`
`finite-spine Length request rejected before IO: ...`	filter mode or a contract path was requested without an activated startup policy	anything about the goal	start Leant with `--length-ranking-config` (POSIX only at this commit)

Evidence states behind the messages

Observed outcome	What is authorized	What is not authorized
Lean-verified candidate	the displayed term inhabits the exact goal in the live environment	intended behavior, efficiency, uniqueness, completeness
Djinn proof before Lean verification	a proof term exists in the translated formula calculus	valid Lean syntax or exact Lean typing

Observed outcome	What is authorized	What is not authorized
all render variants rejected	this reconstruction route produced no accepted term	source type uninhabited
Exference step limit	no candidate in explored prefix	search exhaustion or uninhabitation
Exference natural exhaustion	no candidate in sealed finite Exference search state	general Lean uninhabitation
Djinn budget exhausted	no proof in explored choice prefix	formula unprovable
unbounded Djinn formula exhaustion	translated exact formula unprovable	Lean goal uninhabited unless completeness ledger also passes
proof-backed Leant uninhabited report	no inhabitant in the exact admitted query contract	no future import/provider could ever inhabit the type
unsafe opaque/completeness warning	positive search remains usable; negative is downgraded	conclusive absence
validated Length counterexample	exact retained candidate violates sealed contract on replayed bounded input	ill-typedness or general semantic failure
bounded-positive Length receipt	no violation under the exact supported domain/contract and replay policy	general Lean theorem outside that contract

Glossary

Alpha-equivalence
Equality up to consistent renaming of bound variables.

Atom
An indivisible proposition/type symbol in a smaller search calculus.

Auto-bound implicit
A variable Lean binds automatically when it is mentioned but not declared (Lean's `autoImplicit`); `:synth (a → a → a)` relies on it, and `:synth` retries stuck Sort-polymorphic variables at `Type`.

Binder information
Lean metadata marking a binder explicit, implicit, strict implicit, or instance implicit.

Candidate group
One semantic candidate together with bounded alternative Lean renderings.

Candidate window
The at most 60 distinct engine candidates one `:synth` lane retains before verification (`candidateWindow`); at most five verified ones are shown and bound (`synthMaxShown`).

Classical fallback
The excluded-middle and double-negation lanes `:synth` runs after a sound constructive refutation when `synth.classical` is on (the default); their terms mention `Classical.em` or `Classical.byContradiction`.

Calculus of Constructions (CoC)
A dependent lambda calculus supporting all axes of the lambda cube.

Calculus of Inductive Constructions (CIC)
A CoC-family theory extended with inductive families and eliminators.

Completeness
The property that exhaustive search finds a proof whenever one exists in the stated calculus and environment.

Completeness ledger
Leant metadata recording whether translation, inventories, instantiations, and search justify a negative result.

Context
Ordered local declarations under which a term is typed.

Definitional equality
Equality recognized by the type checker through computation and built-in conversion rules.

Dependent function / Π-type
A function type whose codomain may mention the input.

Dependent pair / Σ-type
A pair whose second component type may mention the first component.

Elaboration
Translation from concise Lean syntax to fully explicit expressions using expected types, metavariables, unification, coercions, and instances.

Environment
The checked global map from names to declarations and metadata.

Exference
Djex's heuristic best-first typed expression search engine.

Fragment
Leant's frontend-owned representation of admitted searchable structure plus opacity and completeness metadata.

Impredicative
Permitting quantification over a universe while the result remains in that same universe; Lean's `Prop` is impredicative.

Indexed family
A type family whose constructors may return different indices, such as `Vector α n`.

Inhabitant
A term having a given type.

Instance implicit
A binder whose argument is synthesized by Lean's type-class resolver.

Lane
One engine run of `:synth` over one premise set: the structural lane, the library lane, provider lanes, and the classical lanes; a verified candidate in an earlier lane stops the later ones.

Length contract
A passive JSON law over the list-spine lengths of a candidate's inputs and result, assessed by the optional Z3-backed stage; *rank* mode reorders, *filter* mode may omit only replayed counterexamples.

Library premise
A curated recursive combinator (`List.map`, `List.foldr`, …) that `synth-library on` (the default) adds as a premise when the goal mentions the corresponding recursive inductive.

LJT
Dyckhoff's contraction-free sequent calculus for intuitionistic propositional logic, used by Djinn.

Metavariable
A temporary elaboration-time unknown with a context and expected type.

Motive
The dependent result family supplied to an inductive recursor or dependent match.

Opaque cargo
A dependent or unsupported type retained as one alpha-aware atom so it can be transported structurally without internal analysis.

Proof irrelevance
Kernel principle identifying all proofs of the same proposition for definitional equality.

Provider
A selected live Lean declaration made available to synthesis as a checked premise.

Recursor
The primitive eliminator generated for an inductive family, supporting recursion and induction.

Refutation safety
Leant's classification of whether hidden fragment structure can be ignored when interpreting exhaustive failure.

Rigid variable
A universally quantified variable that unification may not instantiate.

Semantic origin
Sidecar data tying a candidate to its exact goal, environment, inventory, search policy, and typed provenance.

Soundness
Here, principally the property that every accepted candidate truly has the requested type.

Transport
: Converting a term between dependent types using an equality proof.

Universe
: A type whose elements are types; Lean represents universes as `Sort u`.

Weak-head normal form
: Reduction sufficient to expose a term's outer constructor without fully normalizing every subterm.

Pinned Source Reading Guide

F.1 Leant at `49c8f9f100a6`

`README.md`
User-facing synthesis examples, engine modes, dependent cargo, inductive and negative-result policy.

`docs/Leant_Overview/Leant_Overview.tex`
The user manual (LuaLaTeX, Unicode-safe transcript style); at `49c8f9f100a6` itself the file was still `docs/Leant.tex`, and it moved to this location in the docs-only successor commits.

`docs/synth-internals.md`
Semantic-origin, typed-candidate, verification, and behavioral-layer invariants.

`docs/length-ranking.md`
Complete reference for the Length stage: startup configuration, `:synth` option grammar, retention/rejection taxonomy, progressive batching, presentation.

`docs/Z3_Behavioral_Synthesis_Proposal/`
The proposal the implemented Level-1/bounded Level-2 slice is measured against; `docs/Z3_for_Leant_and_Djex/` (added in the docs-only successor) is the beginner's guide to the same stage.

`test/*.txt`, `test/*.golden`, `test/run-tests.sh`
The recorded `:synth` transcripts quoted throughout Part V and the runner that regenerates them.

`src/Leant/Synth/Fragment.hs`
Lean-generated serializer, `Frag`, provider protocol, parser, and candidate verification command.

`src/Leant/Synth/Engine.hs`
Fragment-to-Djex boundary, search staging, candidate groups, and refutation gate.

`src/Leant/Synth/Render.hs`
Lean syntax reconstruction, binder restoration, names, and variants.

`src/Leant/Synth/Verification.hs`
Lazy group verification and accepted-candidate quota.

`src/Leant/Backend.hs`
Lean REPL process, JSON protocol, timeout, and stderr management.

`src/Main.hs`
Interactive state, commands, sessions, providers, proof mode, and Length integration; the exact outcome messages of Appendix D.

`src/Leant/Options.hs`
Command-line options, including `--length-ranking-config` and its two companions.

`src/Leant/Synth/Length/`
`Command.hs` (the `:synth` option grammar), `Integration.hs` (request authorization and the per-command assessment context), `Selection.hs` and `Ranking.hs` (retention/rejection taxonomy), `Presentation.hs` (candidate and rejection notes), `Handoff.hs` (typed-origin eligibility).

`src/Leant/Synth/BehavioralSelection.hs`
The generic occurrence-sealed partition that filter mode uses; it checks membership and order, never behavior.

Repository root: https://github.com/VladimirReshetnikov/Leant/tree/49c8f9f100a644b5a32b3c942bc6565a3a3ba6a0

F.2 Djex

Leant vendors `7d65eba6f753`; the standalone main snapshot inspected for current architecture is `0cf82f2aa22d`.

`README.md`
Engine overview, shared semantic layer, search contracts, and optional Length/Z3 architecture.

`synthesis/src/Language/Haskell/Synthesis/Type.hs`
Shared Haskell-like type representation and binder/substitution invariants.

`synthesis/src/Language/Haskell/Synthesis/Kind.hs`
Shared kind tree and ground-kind sealing.

`djinn/src-core/Djinn/Internal/LJTFormula.hs`
Propositional formulas, opaque source-type atoms, and proof terms.

`djinn/src-core/Djinn/Internal/LJT.hs`
LJT search, strategies, budgets, proof production, and normalization.

`exference/src-core/Language/Haskell/Exference/Core/Internal/Exference.hs`
Validated best-first search, options, completion states, and typed results.

`synthesis/src/Language/Haskell/Synthesis/Semantic/Length.hs` and `Semantic/Length/`
Public Length API: problems, contracts, evaluation, counterexample banks, SMT-LIB execution.

`synthesis/internal/Language/Haskell/Synthesis/Internal/Semantic/Length/SMTLib.hs`
The QF_LIA translator that chooses the internal symbol spellings `djex_length_input_0`, `djex_length_input_1`, … and seals each query.

Standalone root: https://github.com/VladimirReshetnikov/Djex/tree/0cf82f2aa22debb1de2c7aaa13a2abbe190550d8

Vendored revision: https://github.com/VladimirReshetnikov/Djex/tree/7d65eba6f753727ae7b3d734fb4a7e00f1f44aa7

F.3 Lean 4 at `79c4cd09fdab`

`src/Lean/Level.lean`
Universe-level syntax and operations.

`src/Lean/Expr.lean`
Core expression constructors, binder information, locally nameless representation, literals, metadata, and projections.

`src/Lean/Declaration.lean`
Declaration categories, inductive metadata, recursor compilation, safety, and reducibility hints.

`src/kernel/type_checker.cpp`
Kernel type inference, Π universe calculation, application substitution, weak-head normalization, definitional equality, proof irrelevance, eta, and declaration safety checks.

The three `.lean` files ship with the installed toolchain (an elan toolchain keeps them under `src/lean/Lean/`); `src/kernel/type_checker.cpp` is a Lean-repository path that exists only in a source checkout at the pinned commit, since the toolchain ships the kernel compiled.

Repository root: https://github.com/leanprover/lean4/tree/79c4cd09fdab845aa6066bcbb09a663876601027

Bibliography

[1] Lean Developers, *The Lean Language Reference*, official online reference manual. https://lean-lang.org/doc/reference/latest/

[2] Jeremy Avigad, Leonardo de Moura, Soonho Kong, Sebastian Ullrich, and contributors, *Theorem Proving in Lean 4*, official online textbook. https://lean-lang.org/theorem_proving_in_lean4/

[3] Leonardo de Moura and Sebastian Ullrich, "The Lean 4 Theorem Prover and Programming Language," in *Automated Deduction – CADE 28*, LNCS 12699, 2021, pp. 625–635.

[4] Thierry Coquand and Gérard Huet, "The Calculus of Constructions," *Information and Computation*, 76(2–3), 1988, pp. 95–120.

[5] Henk Barendregt, "Introduction to Generalised Type Systems," *Journal of Functional Programming*, 1(2), 1991, pp. 125–154.

[6] Roy Dyckhoff, "Contraction-Free Sequent Calculi for Intuitionistic Logic," *Journal of Symbolic Logic*, 57(3), 1992, pp. 795–807.

[7] Peter Dybjer, "Inductive Families," *Formal Aspects of Computing*, 6(4), 1994, pp. 440–465.

[8] Christine Paulin-Mohring, "Inductive Definitions in the System Coq: Rules and Properties," in *Typed Lambda Calculi and Applications*, LNCS 664, 1993, pp. 328–345.

[9] The Leant contributors, *Leant*, source repository, pinned at commit `49c8f9f100a6`. https://github.com/VladimirReshetnikov/Leant/tree/49c8f9f100a644b5a32b3c942bc6565a3a3ba6a0

[10] The Djex contributors, building on Lennart Augustsson's Djinn and Lennart Spitzner's Exference, *Djex*, source repository, Leant-vendored revision `7d65eba6f753` and inspected standalone revision `0cf82f2aa22d`. https://github.com/VladimirReshetnikov/Djex

[11] Lean Developers, *Lean 4*, source repository, pinned at commit `79c4cd09fdab`. https://github.com/leanprover/lean4/tree/79c4cd09fdab845aa6066bcbb09a663876601027